



媒体与认知

Media and Cognition

清华大学电子工程系

王生进 李亚利

Email: liyali13@tsinghua.edu.cn

第6讲 深度学习之序列建模

6.1 序列建模概述

6.2 循环神经网络

6.3 TRANSFORMER

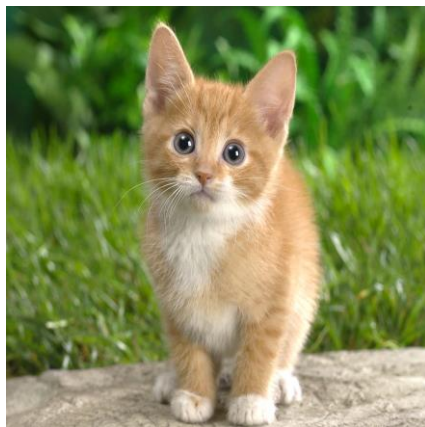
6.4 应用与展望

6.1 概述

➤ 基于神经网络的模式识别

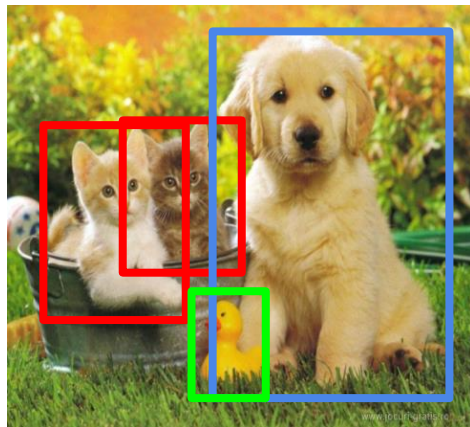
- ◆ 模式识别：图像分类、目标检测、图像分割等

Classification



CAT

Object Detection



CAT, DOG, DUCK

Instance Segmentation

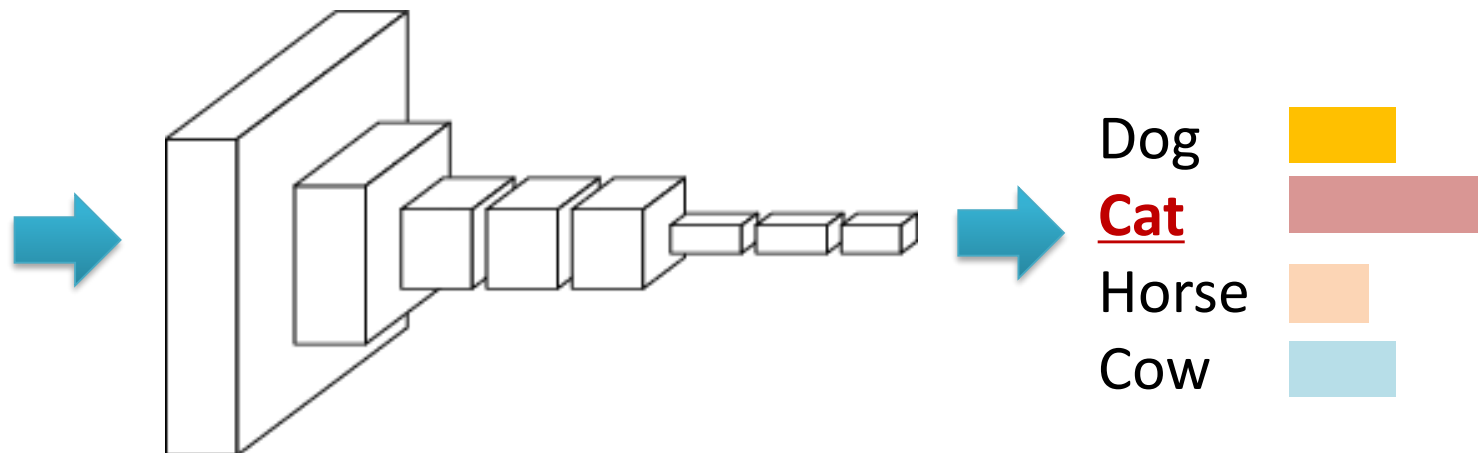
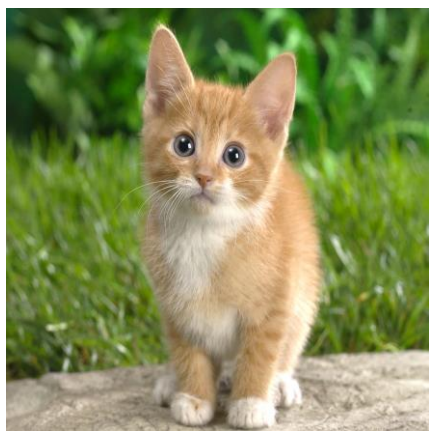


CAT, DOG, DUCK

6.1 概述

➤ 基于神经网络的模式识别

◆ 模式识别：图像分类、目标检测、图像分割等



6.1 概述

➤ 基于神经网络的模式识别

◆ 序列识别：自动语音识别ASR、手写文字识别等

手写文字识别

Foreign Minister.

FOREIGN MINISTER.

自动语音识别ASR
Speech recognition



THE SOUND OF

机器翻译

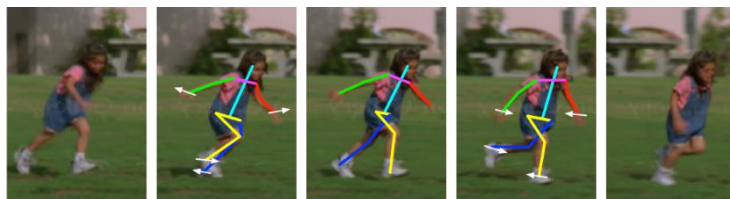
Machine translation

我是电子工程
系的学生。

I am a student from
the Department of
Electronic Engineering.

行为识别

Activity recognition

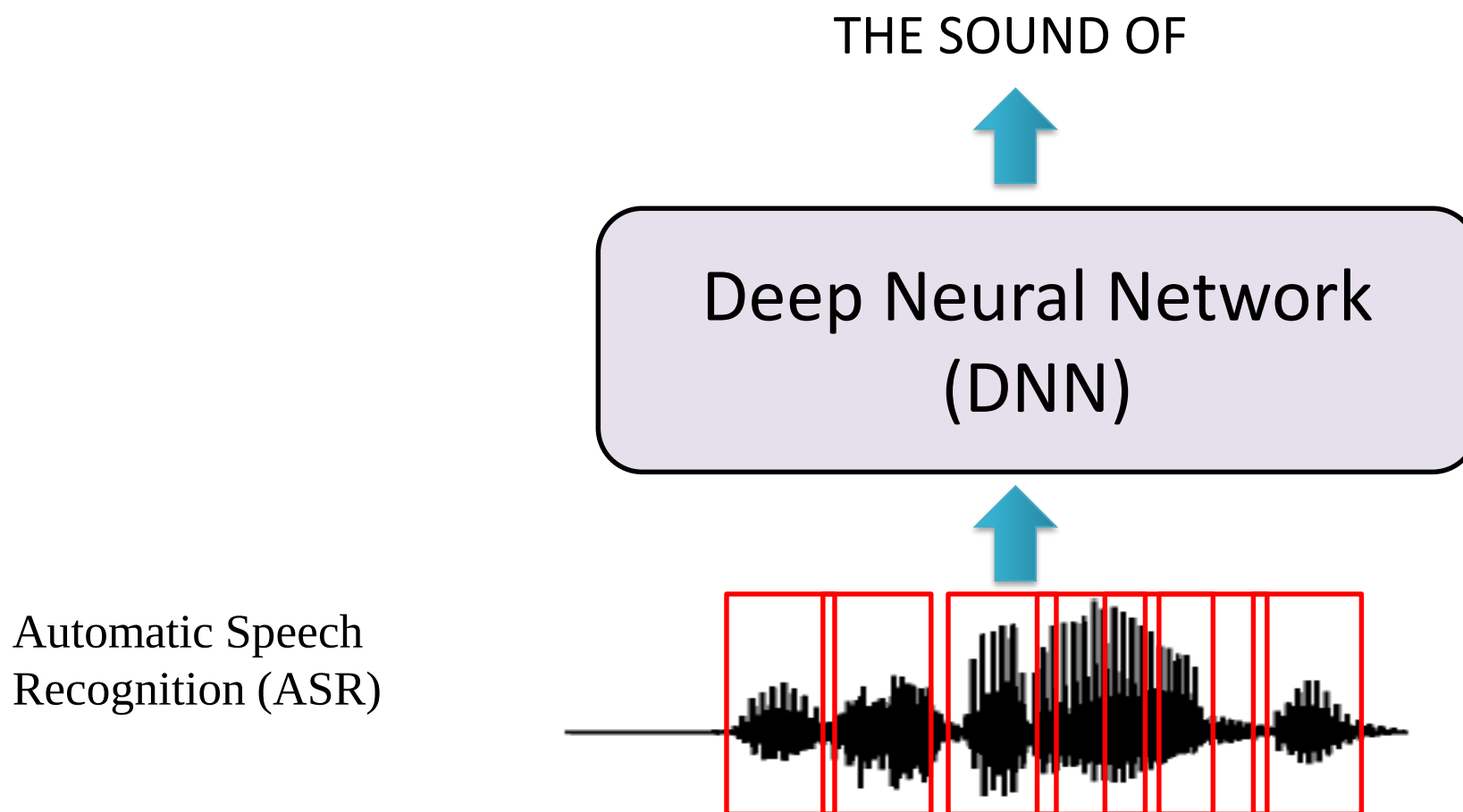


Running

6.1 概述

➤ 基于神经网络的模式识别

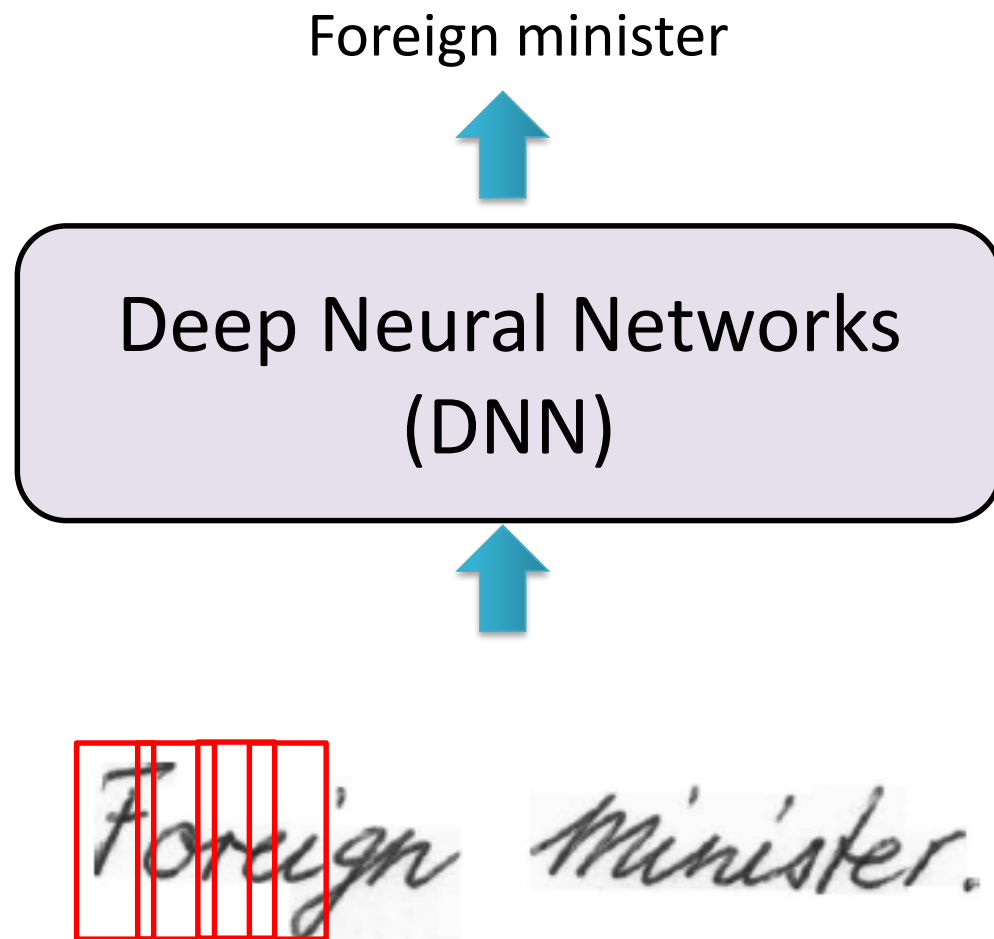
◆ 序列识别：自动语音识别ASR



6.1 概述

➤ 基于神经网络的模式识别

◆ 序列识别：手写文字识别



6.1 概述

➤ 基于神经网络的模式识别

◆ 序列识别：机器翻译

I am a student from the Department of Electronic Engineering.



Deep Neural Networks
(DNN)



我是电子工程系的学生。

6.1 概述

➤ 基于神经网络的模式识别

◆ 序列识别：词性标记

I (noun) saw (verb) a (art) saw (noun) .



Deep Neural Networks
(DNN)

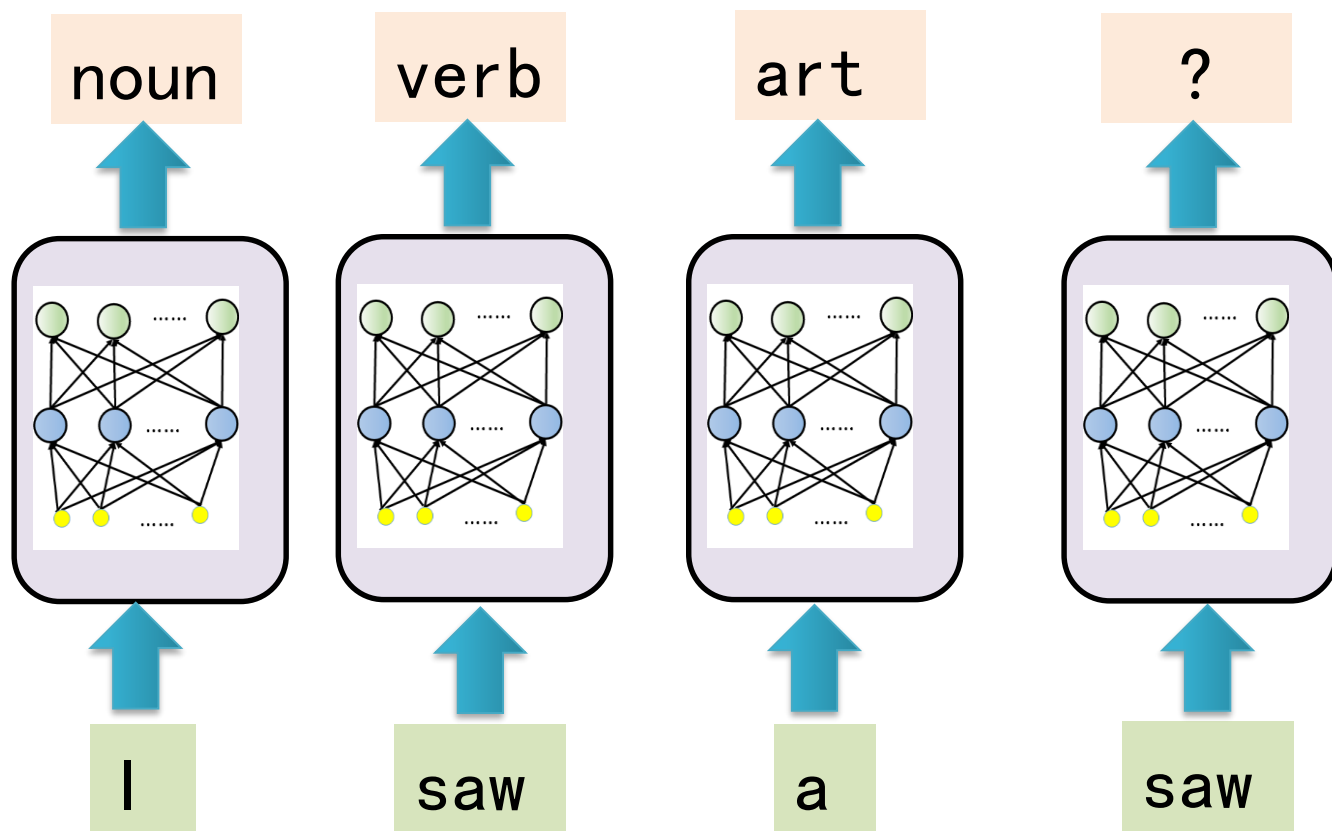


I saw a saw.

6.1 概述

► 利用全连接前馈神经网络进行序列建模

◆ 例：词性标记

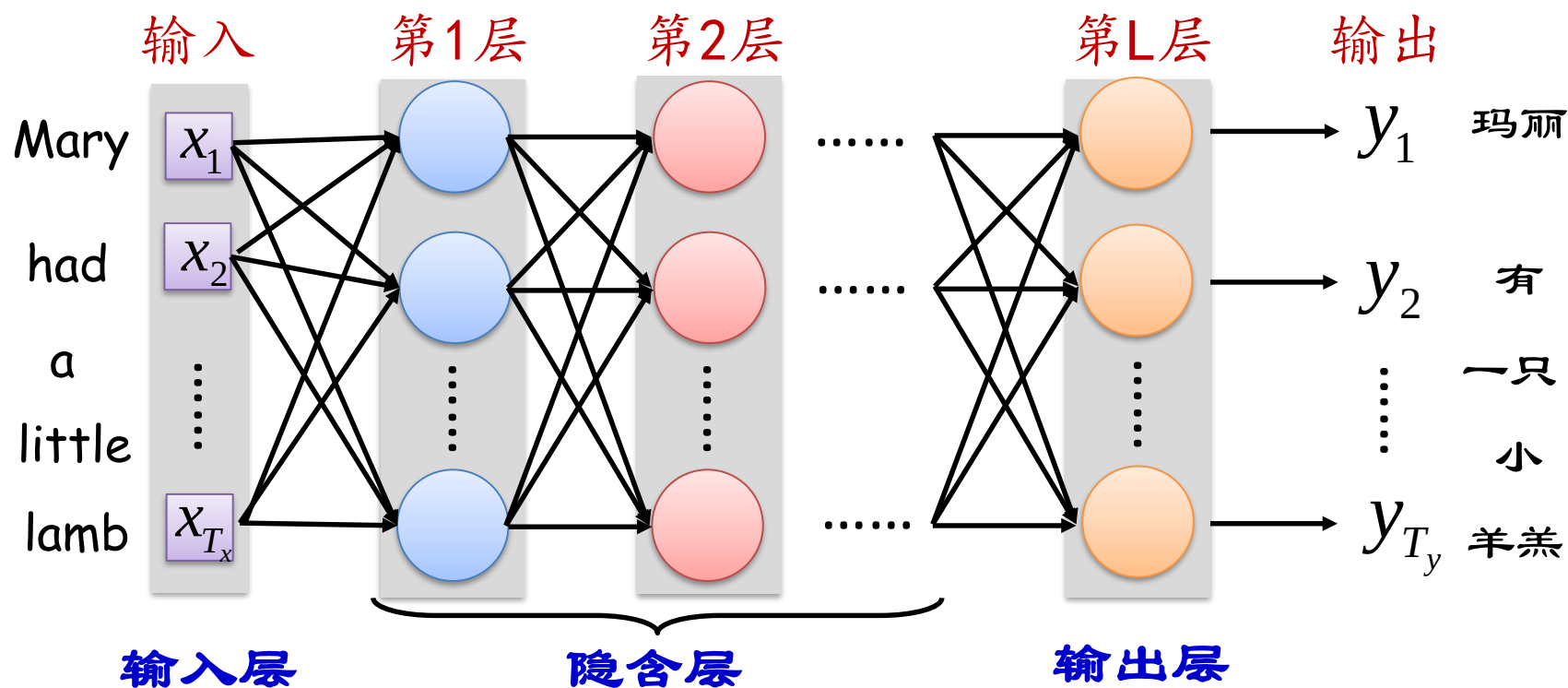


问题：未建模序列中
不同位置输入的关联；

6.1 概述

► 利用全连接前馈神经网络进行序列建模

◆ 例：机器翻译



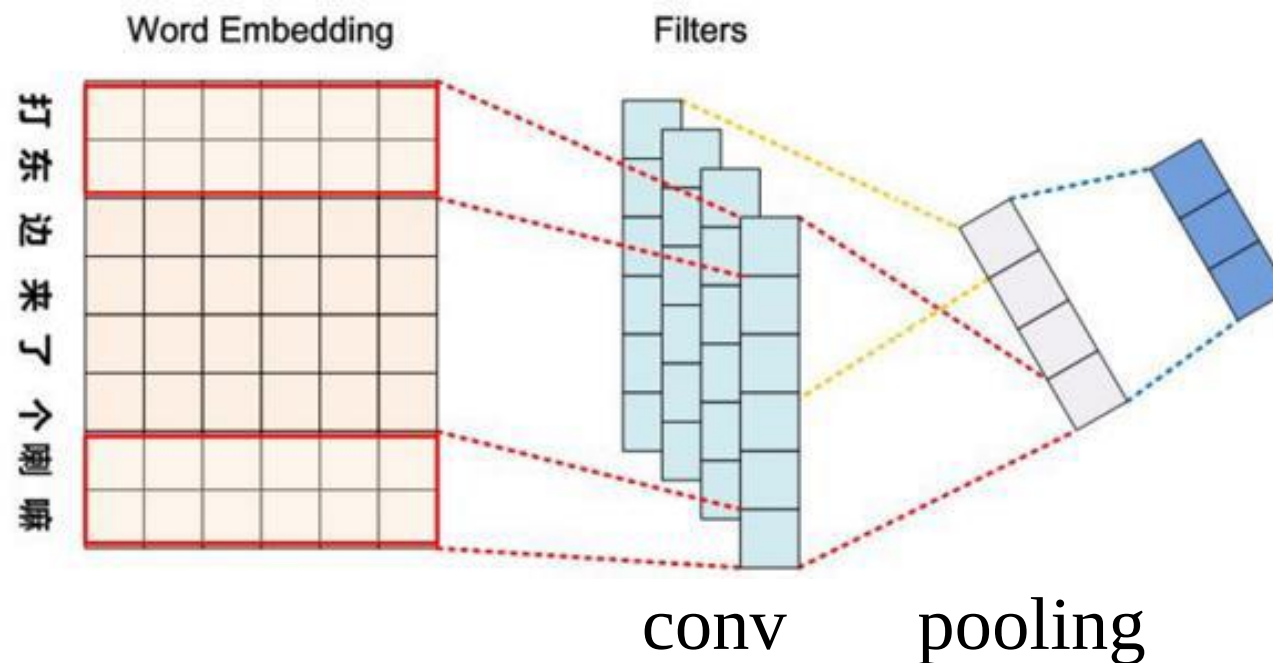
问题：1. 序列数据的输入与输出不定长；
2. 序列中不同位置的特征不共享。

6.1 概述

► 利用卷积神经网络处理时序数据

问题： 1. 序列数据的输入与输出不定长；
2. 序列中不同位置的特征不共享。

◆ 解决思路：采用Max-pooling处理变长序列



第6讲 深度学习之序列建模

6.1 序列建模概述

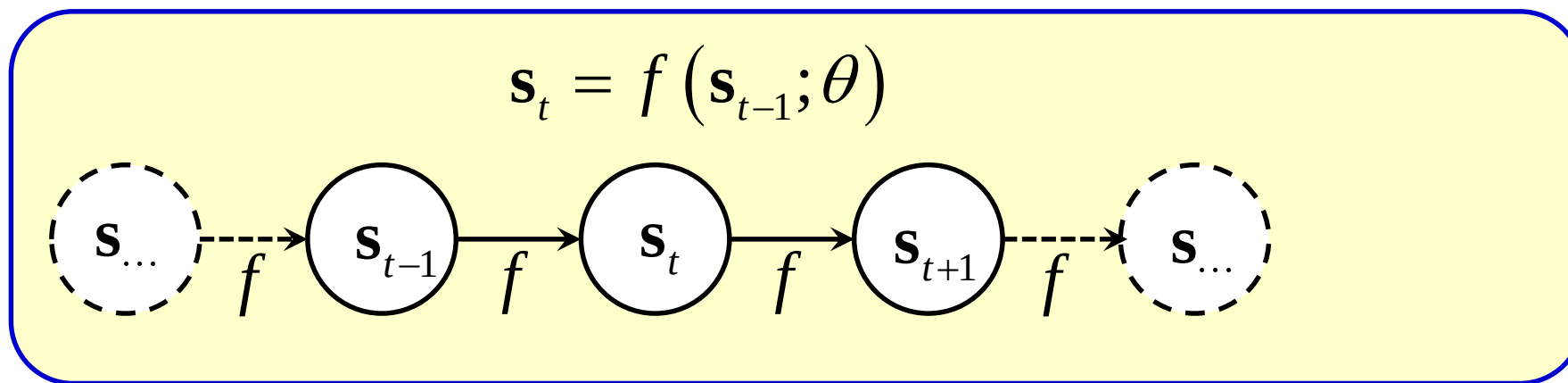
6.2 循环神经网络

6.3 TRANSFORMER

6.4 应用与展望

引子

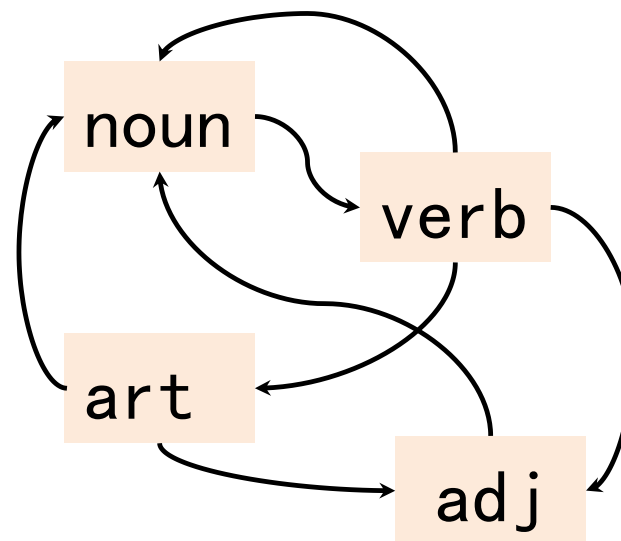
► 动态系统与序列建模



$\mathbf{s}_t$ 为动态系统的状态

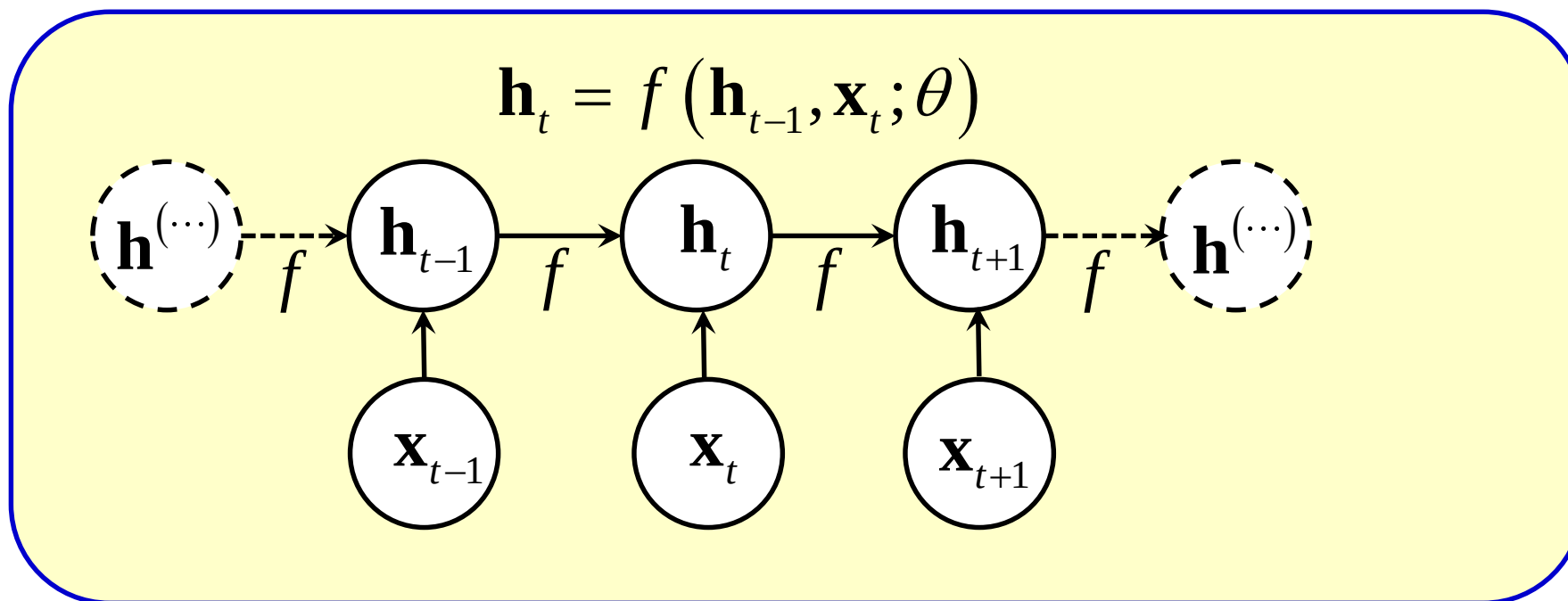
$$\mathbf{s}_t = f(\mathbf{s}_{t-1}; \theta)$$

带参数的状态转换模型



引子

► 动态系统与序列建模



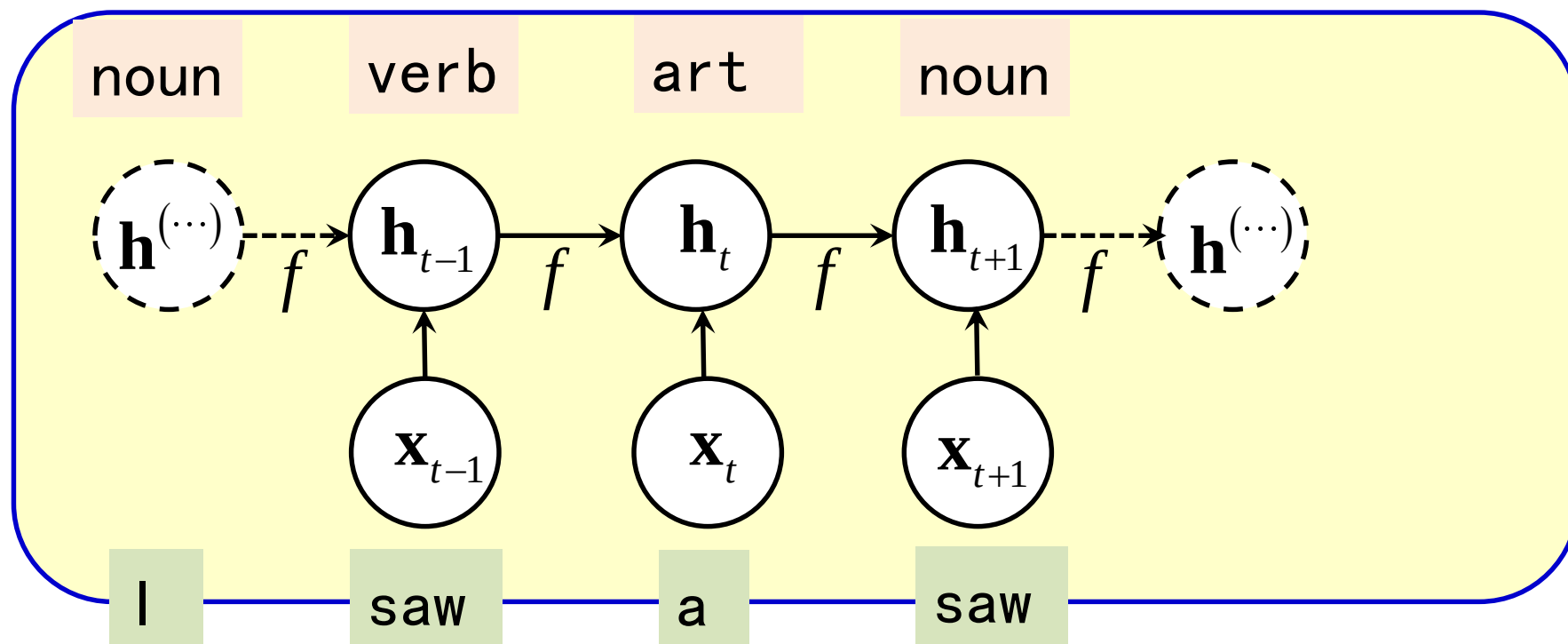
$\mathbf{h}_t$ 为系统的状态

$\mathbf{x}_t$ 为某时刻的系统输入

引子

► 动态系统与序列建模

◆ 例：词性标记



h_t 为系统的状态

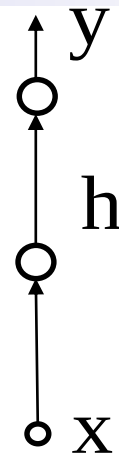
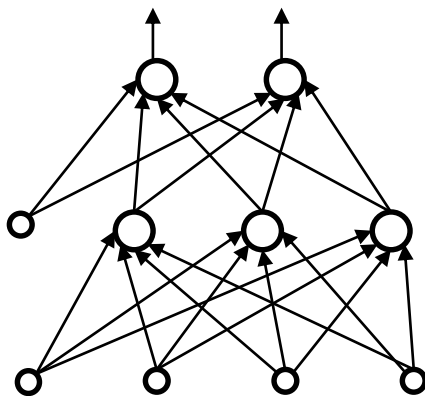
x_t 为某时刻的系统输入

$$h_t = f(h_{t-1}, x_t; \theta)$$

神经网络的结构

➤ 前馈神经网络

- ◆ 结构为有向无环图
- ◆ 隐含层 $h = f(\mathbf{w}^T \mathbf{x})$
- ◆ 处理网格化数据

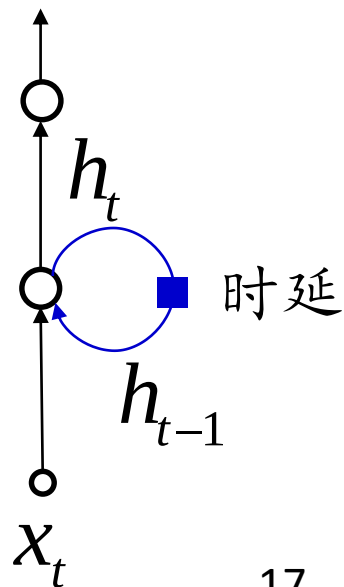
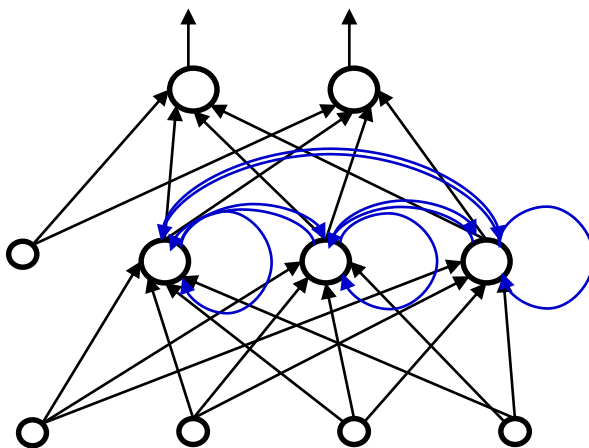


➤ 循环神经网络

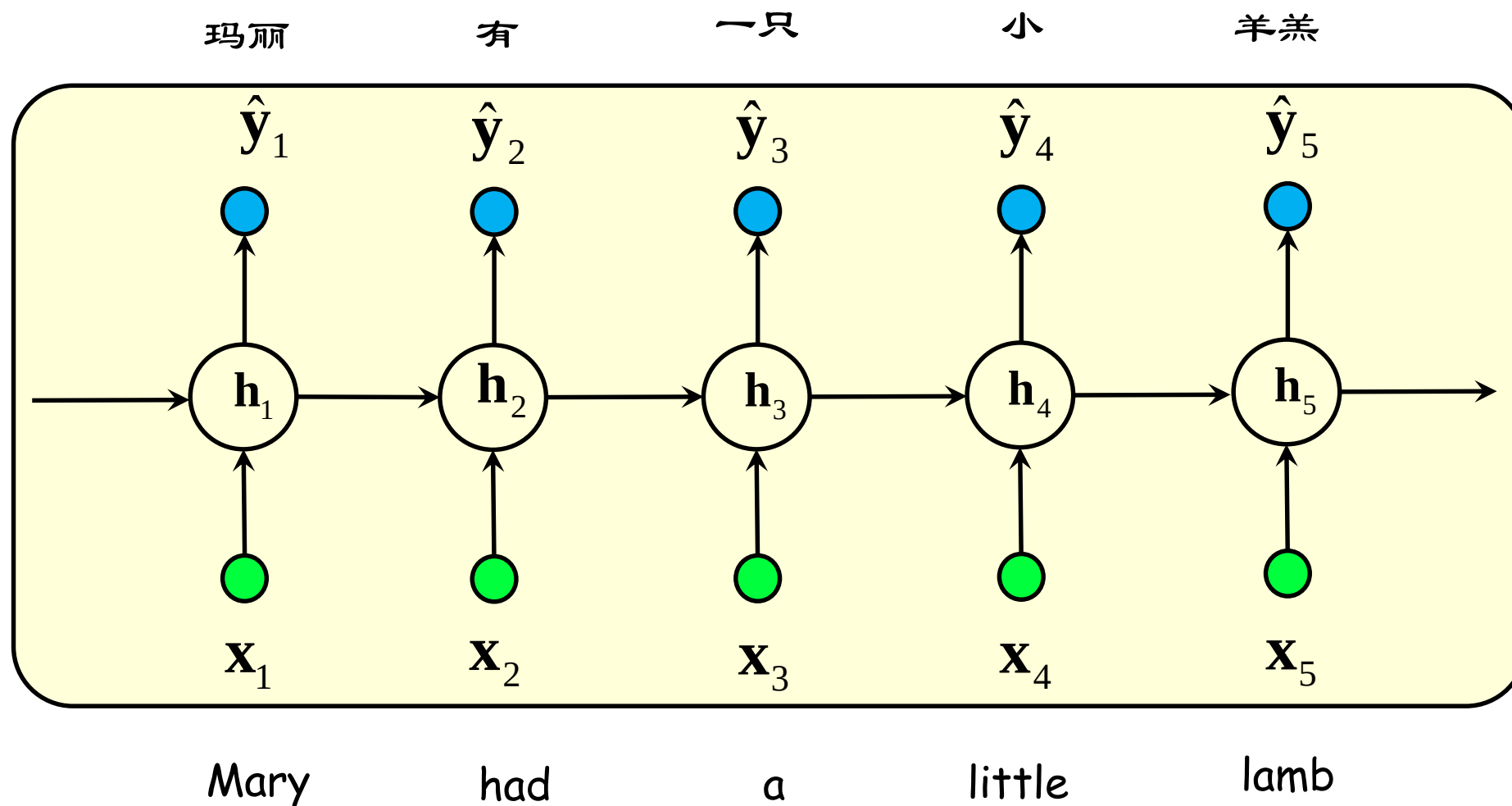
- ◆ 反馈环路的引入

$$h_t = f(\mathbf{w}_x^T \mathbf{x}_t + \mathbf{w}_h^T h_{t-1})$$

- ◆ 处理序列化数据



6.2 循环神经网络



6.2 循环神经网络

► 输入：词向量

◆ 简单方式：One-hot编码

Vocabulary

a
Aaron
⋮
and
⋮
Mary
⋮
zulu

Mary

$$\begin{bmatrix} 0 \\ \vdots \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

had

$$\begin{bmatrix} 0 \\ \vdots \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

a

$$\begin{bmatrix} 1 \\ \vdots \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

little

...

lamb

...

6.2 循环神经网络

► 输入：词向量

◆ 简单方式：One-hot编码

中文词组

啊
爱
⋮
玛
丽
⋮
羊
⋮
组
织

玛丽

$$\begin{bmatrix} 0 \\ \vdots \\ 1 \\ 0 \\ \vdots \end{bmatrix}$$

有

$$\begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ \vdots \end{bmatrix}$$

一只

$$\begin{bmatrix} 0 \\ \vdots \\ 0 \\ \vdots \\ 1 \end{bmatrix}$$

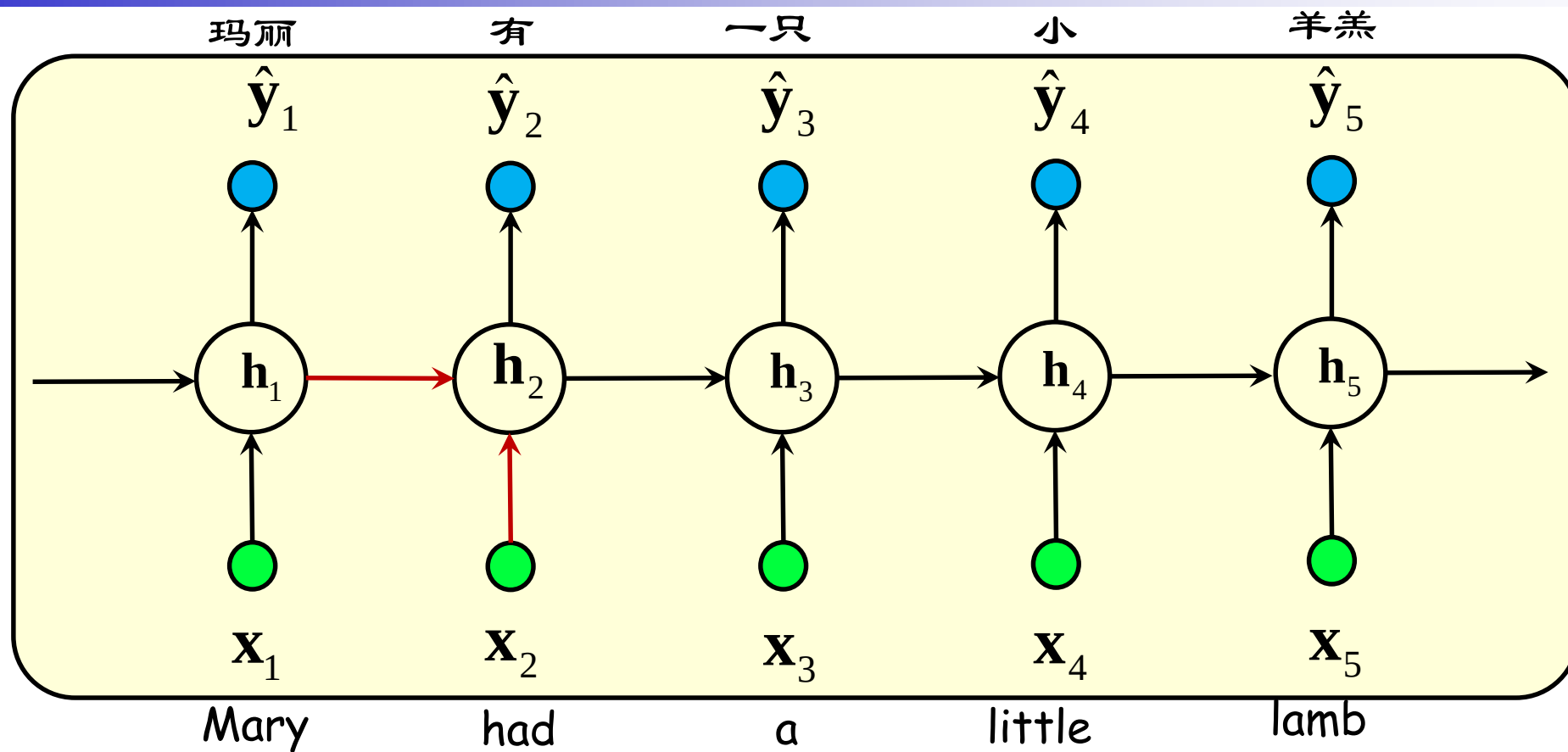
小

...

羊羔

...

6.2 循环神经网络：基本单元



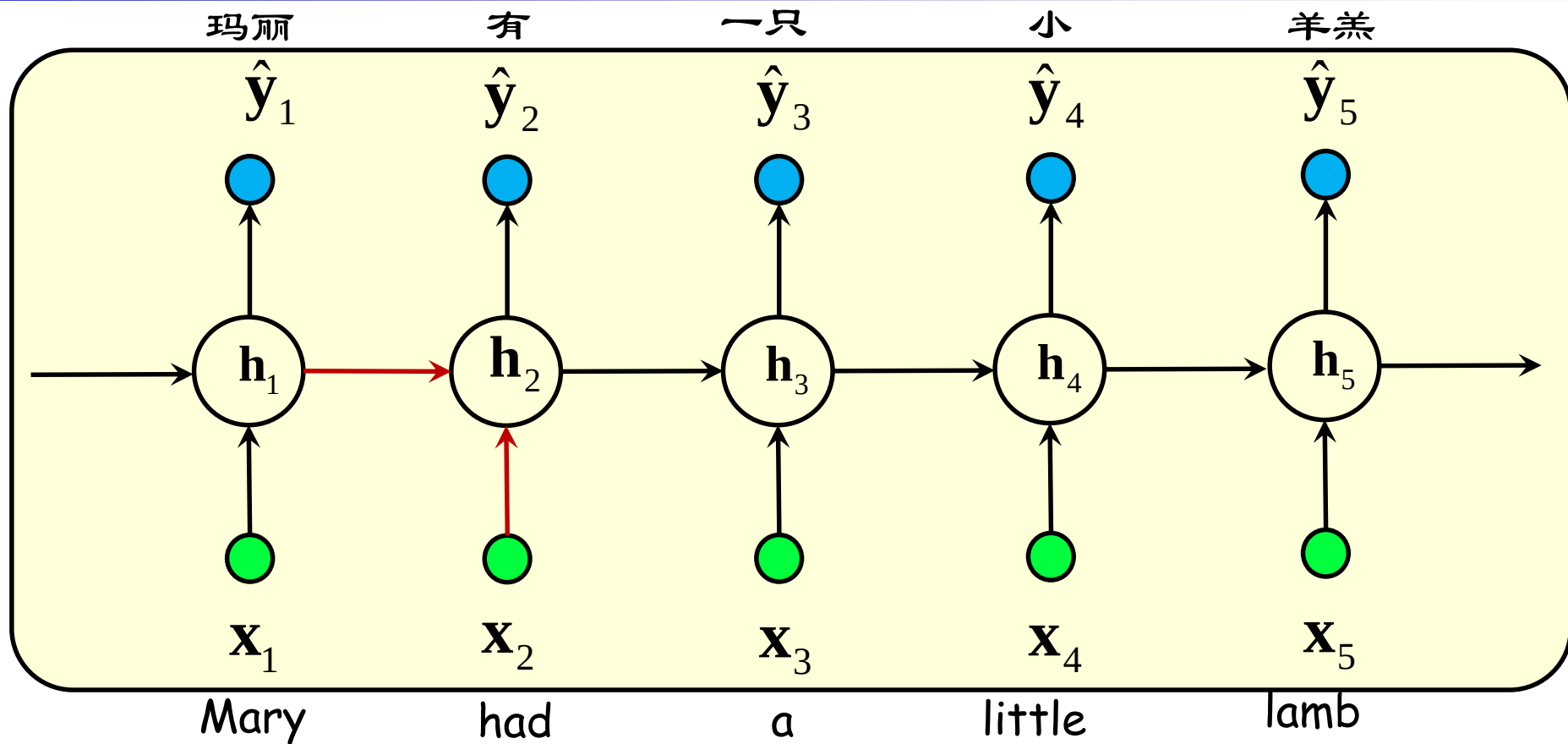
$$\mathbf{h}_1 = f(\mathbf{h}_0, \mathbf{x}_1; \theta_f), \quad \hat{\mathbf{y}}_1 = g(\mathbf{h}_1; \theta_g)$$

$$\mathbf{h}_2 = f(\mathbf{h}_1, \mathbf{x}_2; \theta_f), \quad \hat{\mathbf{y}}_2 = g(\mathbf{h}_2; \theta_g)$$

...

时序参数共享

6.2 循环神经网络：基本单元



$$\mathbf{h}_1 = f(\mathbf{h}_0, \mathbf{x}_1; \theta_f), \quad \hat{\mathbf{y}}_1 = g(\mathbf{h}_1; \theta_g)$$

$$\mathbf{h}_2 = f(\mathbf{h}_1, \mathbf{x}_2; \theta_f), \quad \hat{\mathbf{y}}_2 = g(\mathbf{h}_2; \theta_g)$$

...

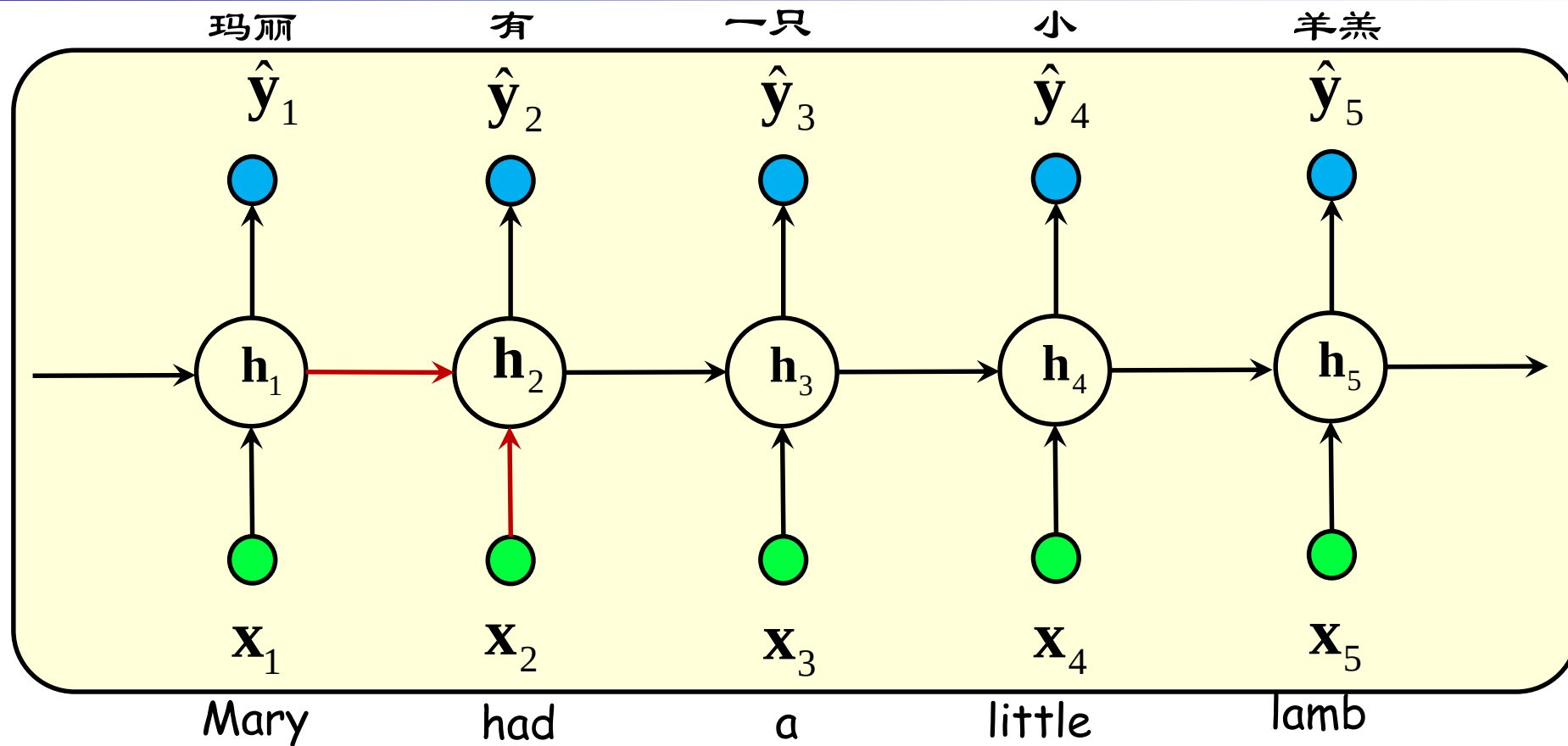
时序参数共享



$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \theta_f)$$

$$\hat{\mathbf{y}}_t = g(\mathbf{h}_t; \theta_g)$$

6.2 循环神经网络：基本单元



$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \theta_f)$$

$$\hat{\mathbf{y}}_t = g(\mathbf{h}_t; \theta_g)$$

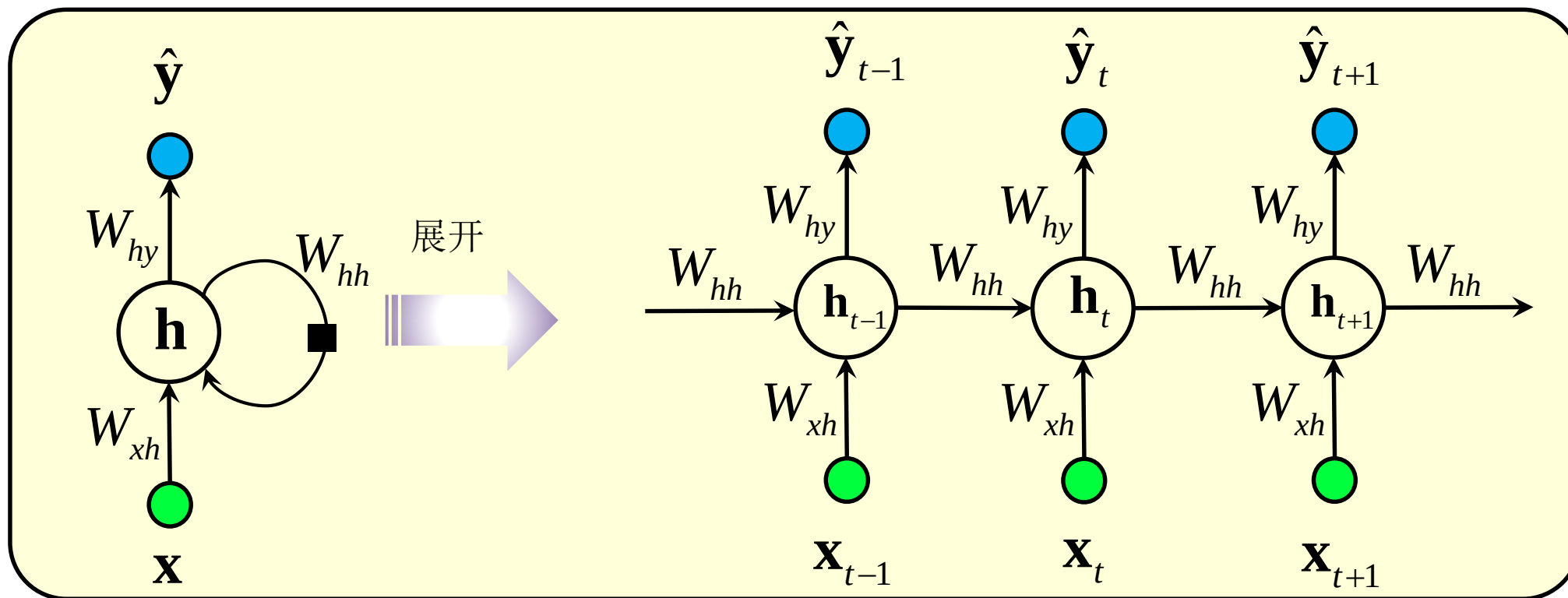


$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \mathbf{W}_{hy} \mathbf{h}_t + \mathbf{b}_y$$

可为其他
激活函数

6.2 循环神经网络：基本单元



$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \mathbf{W}_{hy} \mathbf{h}_t + \mathbf{b}_y$$

$$\mathbf{h}_t = \tanh(\mathbf{W}_h [\mathbf{h}_t, \mathbf{x}_t] + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \mathbf{W}_{hy} \mathbf{h}_t + \mathbf{b}_y$$

输出序列

1.56

2.73

=

0.1

+

1.0

2.0

×

0.537 0.462

=

0.1

+

1.0

2.0

×

0.860 0.884

0.0 0.0

0.0 0.0 1.0

×

0.1 0.2
0.3 0.4
0.5 0.6

+

0.1 -0.1

0.537 0.462

0.537 0.462 2.0

×

0.1 0.2
0.3 0.4
0.5 0.6

+

0.1 -0.1

输入序列

1

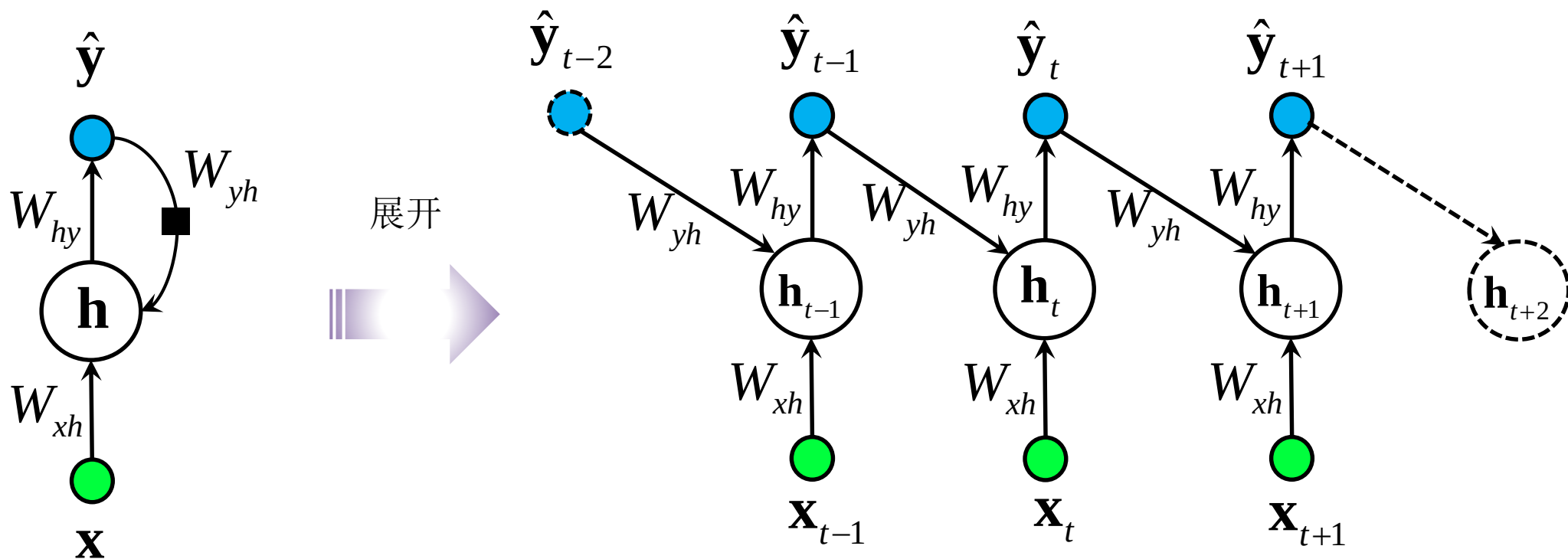
2

时刻

$t = 1$

$t = 2$

6.2 循环神经网络：变体

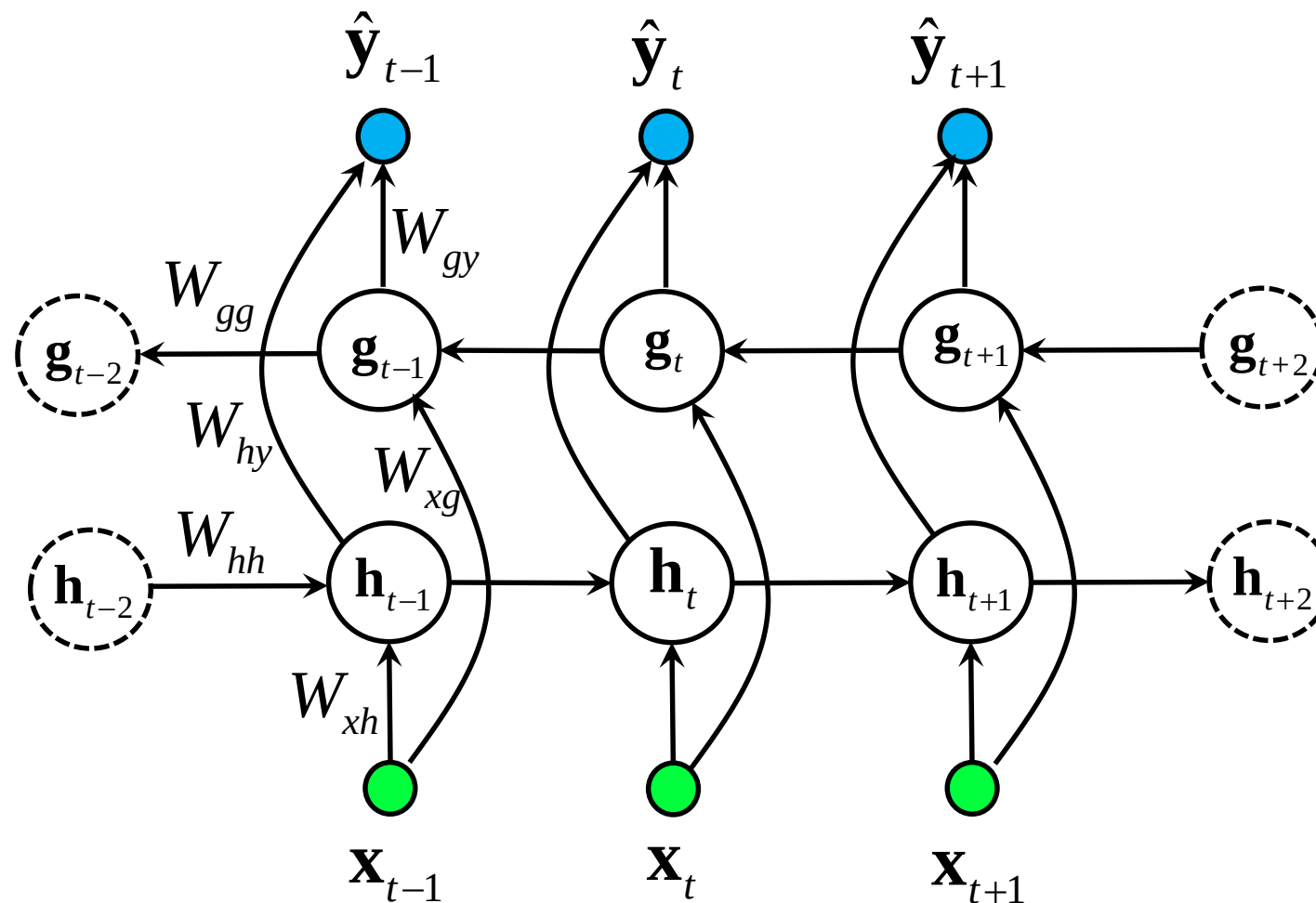


6.2 循环神经网络：拓展

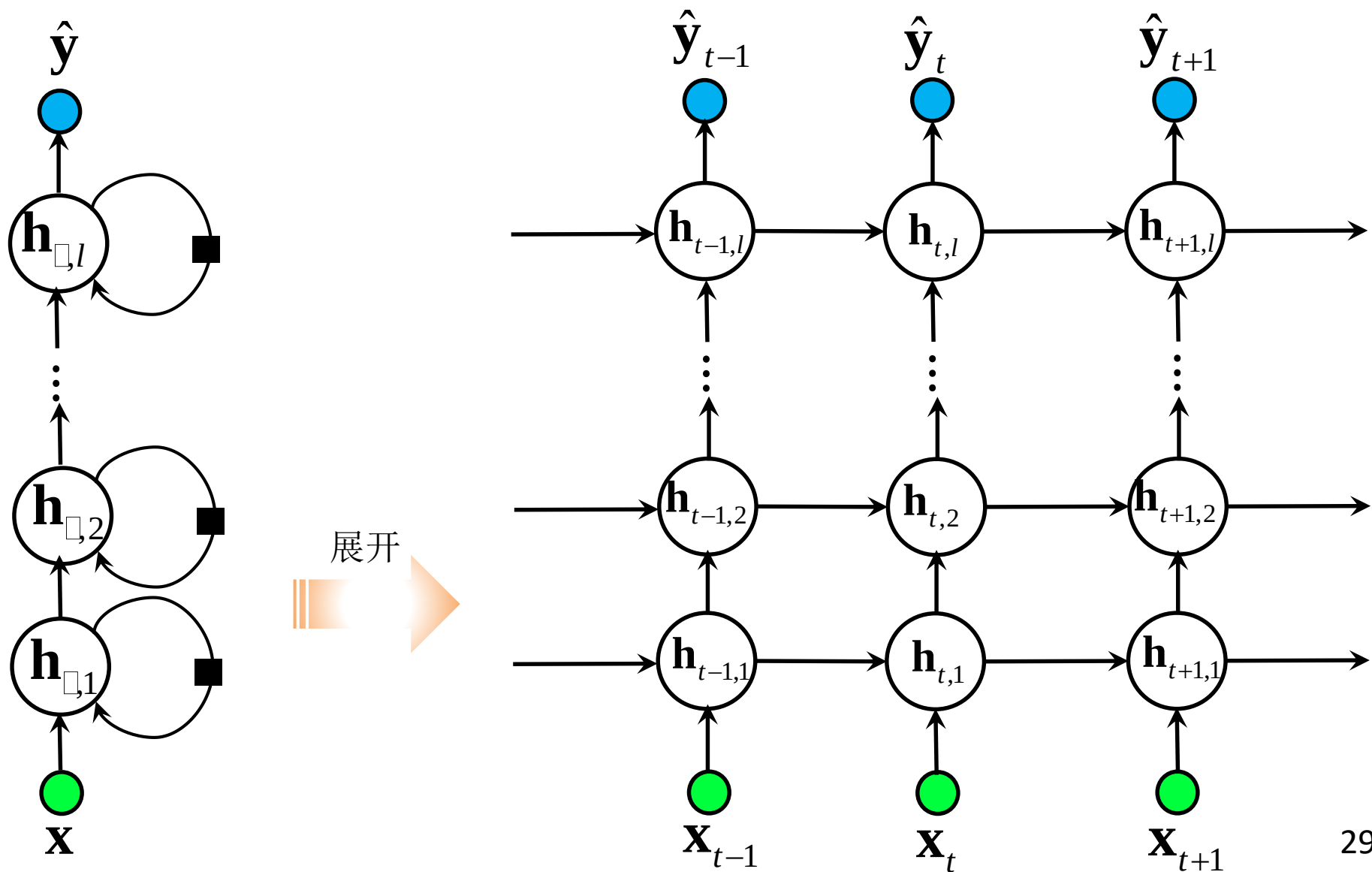
► 双向RNN

Donald Trump is the 45th
President of the USA.

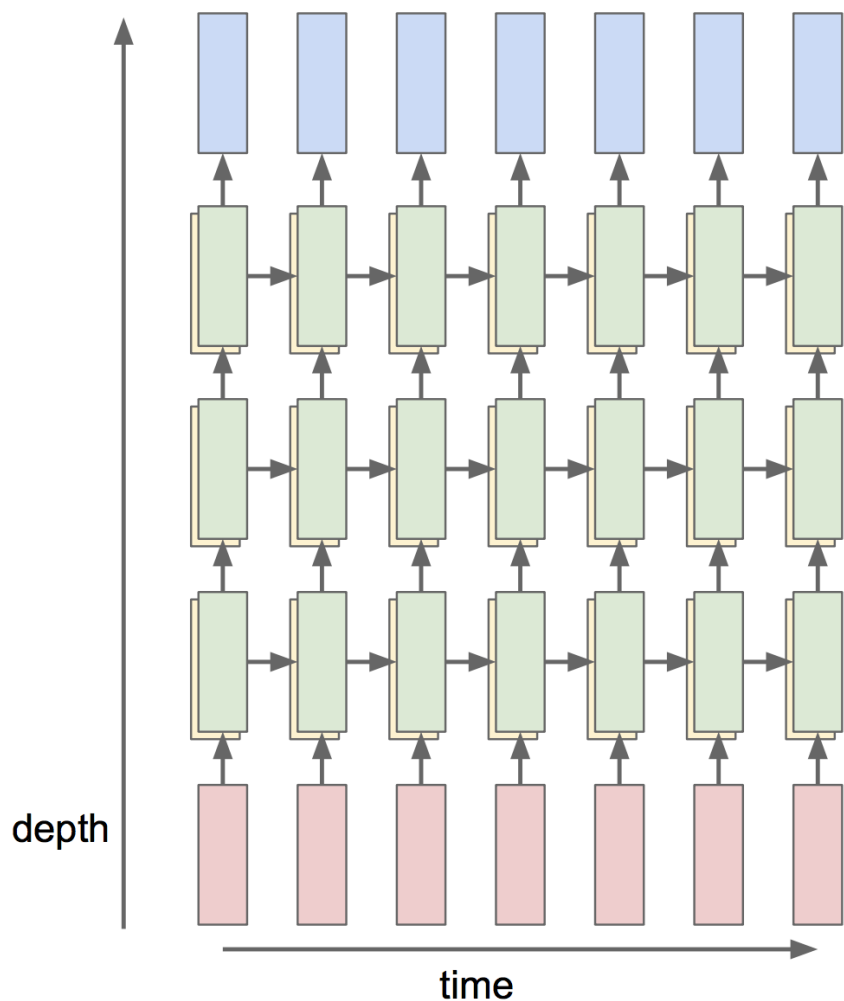
Donald duck is one of history's
most famous cartoon characters.



6.2 深度循环神经网络



6.2 深度循环神经网络



同时刻来自前一层的输入

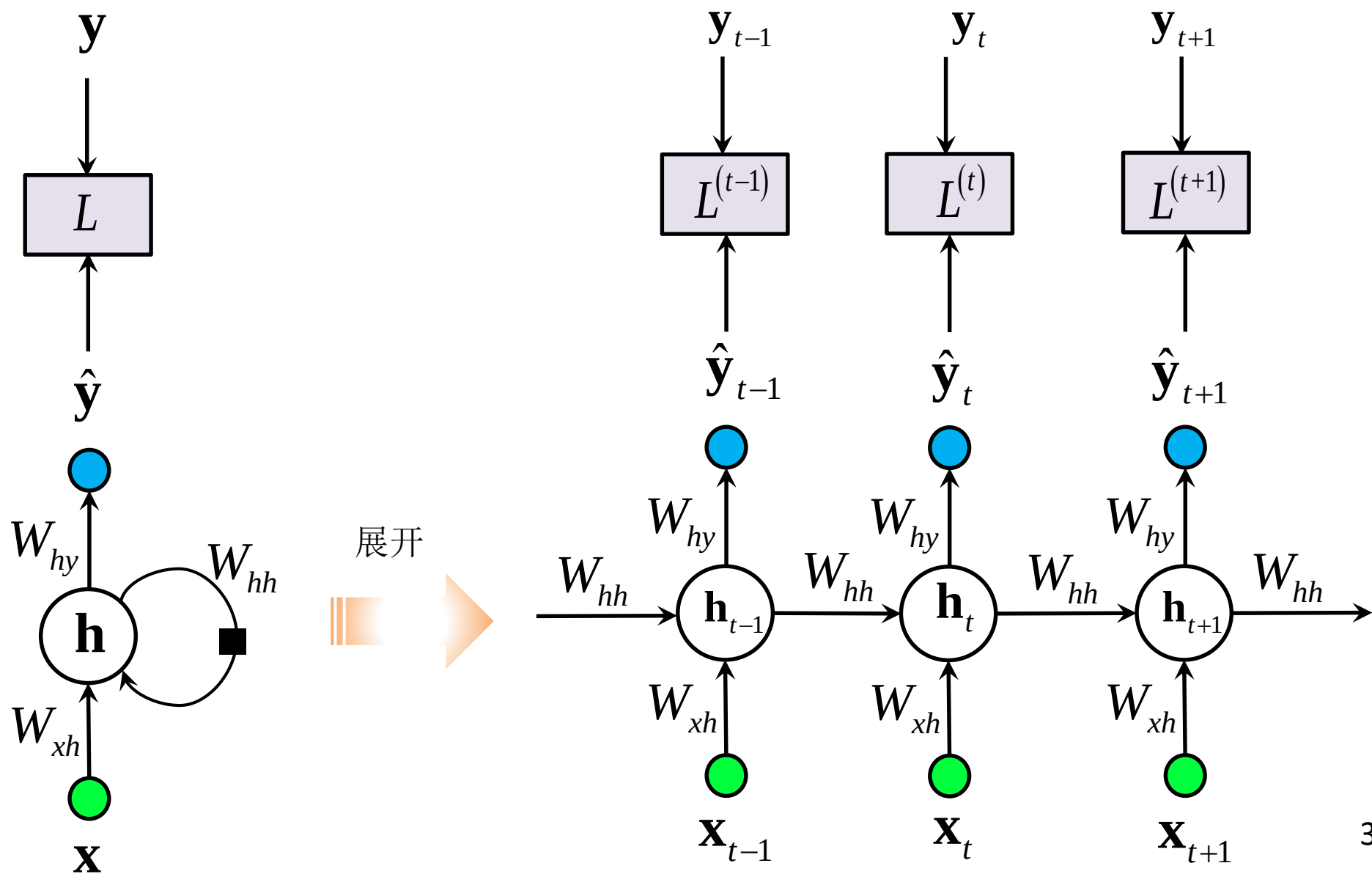
$$h_{t,l} = \tanh W^l \begin{pmatrix} h_{t,l-1} \\ h_{t-1,l} \end{pmatrix}$$

前一时刻来自同一层的状态

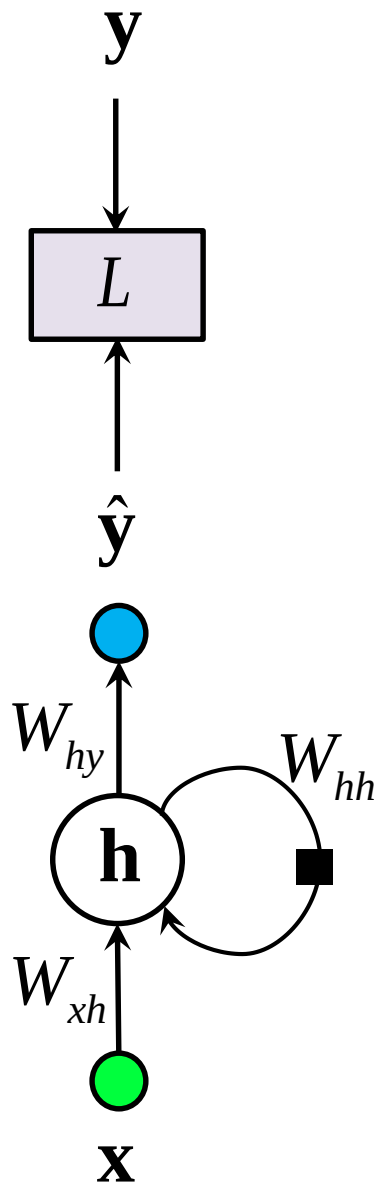
$$h \in \mathbb{R}^d, W^l \in \mathbb{R}^{d \times 2d}$$

多个RNN单元堆叠形成深度循环神经网络(deep RNN),或者称为多层RNN(multi-layers RNN).

6.2 循环神经网络：优化学习



6.2 循环神经网络：优化学习



$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

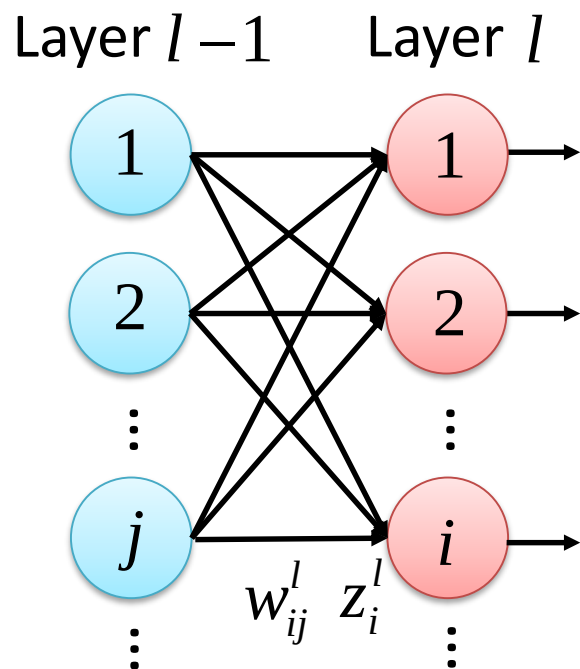
$$\mathbf{o}_t = \text{softmax}(\hat{\mathbf{y}}_t)$$

$$L(\{\mathbf{x}_1, \dots, \mathbf{x}_\tau\}, \{\mathbf{y}_1, \dots, \mathbf{y}_\tau\}) = \sum_t L^t$$

$$= -\sum_t \log p_{\text{model}}(\mathbf{y}_t | \{\mathbf{x}_1, \dots, \mathbf{x}_t\})$$

6.2 循环神经网络：优化学习

回顾BP算法：矩阵化表达



$$\begin{cases} a_j^{l-1} & l > 1 \\ x_j & l = 1 \end{cases}$$

前向传播

$$\begin{aligned} z^1 &= W^1 x + b^1 \\ a^1 &= A(z^1) \\ &\dots\dots\dots \\ z^{l-1} &= W^{l-1} a^{l-2} + b^{l-1} \\ a^{l-1} &= A(z^{l-1}) \\ &\dots\dots\dots \end{aligned}$$

$$\frac{\partial L}{\partial w_{ij}^l} = \frac{\partial z_i^l}{\partial w_{ij}^l} \frac{\partial L}{\partial z_i^l}$$

误差信号

$$\delta_i^l$$

误差反向传播

$$\begin{aligned} \delta^l &= A'(z^l) \circ \nabla L(\hat{y}) \\ \delta^{l-1} &= A'(z^{l-1}) \circ (W^L)^T \delta^l \\ &\dots\dots\dots \end{aligned}$$

$$\nabla_{w^l} L = \begin{cases} \delta_l (a_{l-1})^T & l > 1 \\ \delta_l (x)^T & l = 1 \end{cases}$$

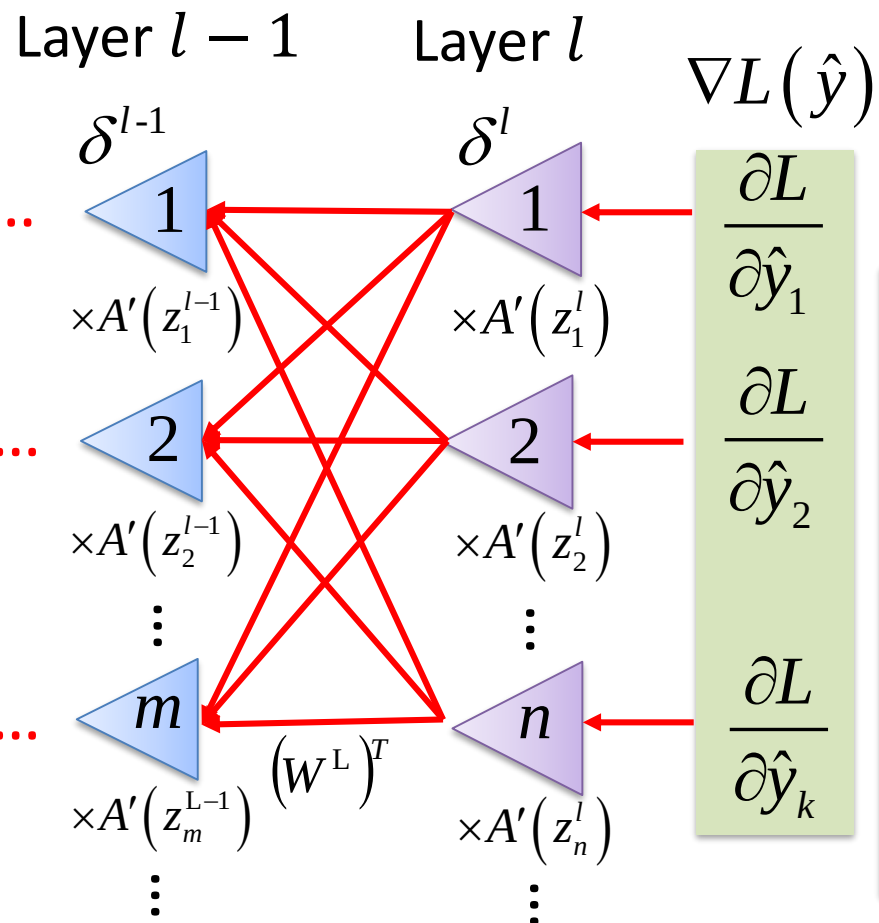
6.2 循环神经网络：优化学习

回顾BP算法：矩阵化表达

$$\frac{\partial L}{\partial w_{ij}^l} = \frac{\partial z_i^l}{\partial w_{ij}^l} \frac{\partial L}{\partial z_i^l}$$

误差信号

$$\delta_i^l$$



$$\begin{cases} a_j^{l-1} & l > 1 \\ x_j & l = 1 \end{cases}$$

前向传播

$$\begin{aligned} z^1 &= W^1 x + b^1 \\ a^1 &= A(z^1) \\ &\dots\dots\dots \\ z^{l-1} &= W^{l-1} a^{l-2} + b^{l-1} \\ a^{l-1} &= A(z^{l-1}) \\ &\dots\dots\dots \end{aligned}$$

误差反向传播

$$\begin{aligned} \delta^l &= A'(z^l) \circ \nabla L(\hat{y}) \\ \delta^{l-1} &= A'(z^{l-1}) \circ (W^L)^T \delta^l \\ &\dots\dots\dots \end{aligned}$$

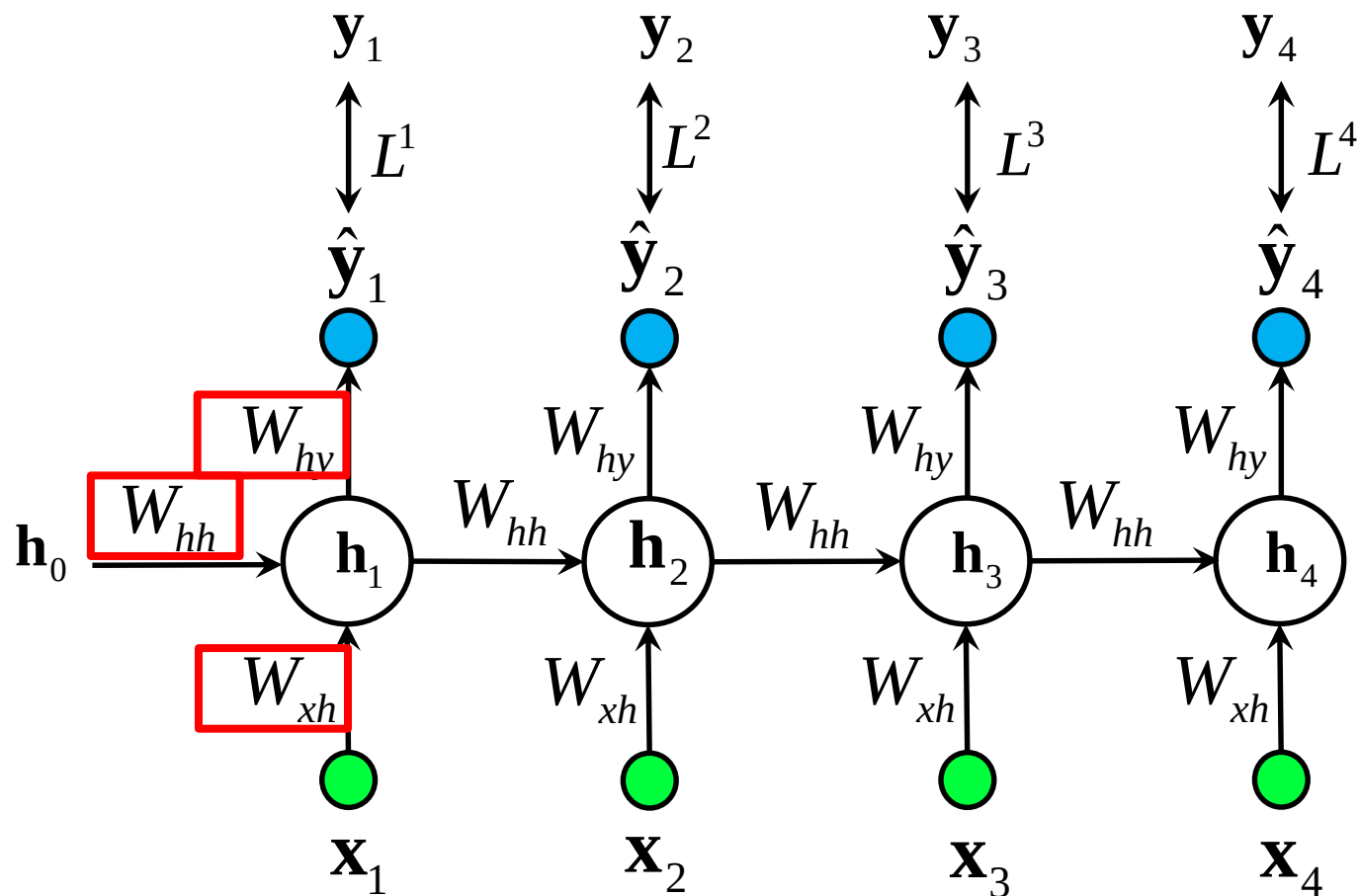
$$\nabla_{w^l} L = \begin{cases} \delta_l (a_{l-1})^T & l > 1 \\ \delta_l (x)^T & l = 1 \end{cases}$$

6.2 循环神经网络：优化学习

Backpropagation through time (BPTT)

$$L = \sum_{t=1}^T L^t, L^t = -\log(o^t)$$
$$o^t = \text{Softmax}(\hat{y}^t)$$

$$L = \frac{1}{2} \sum_{t=1}^{T_y} \|y^t - \hat{y}^t\|^2$$
$$L^t = \frac{1}{2} \|y^t - \hat{y}^t\|^2$$



网络参数梯度下降 W_{xh}, W_{hy}, W_{hh}

$$\Rightarrow w \leftarrow w - \eta \partial L / \partial w$$

6.2 循环神经网络：优化学习

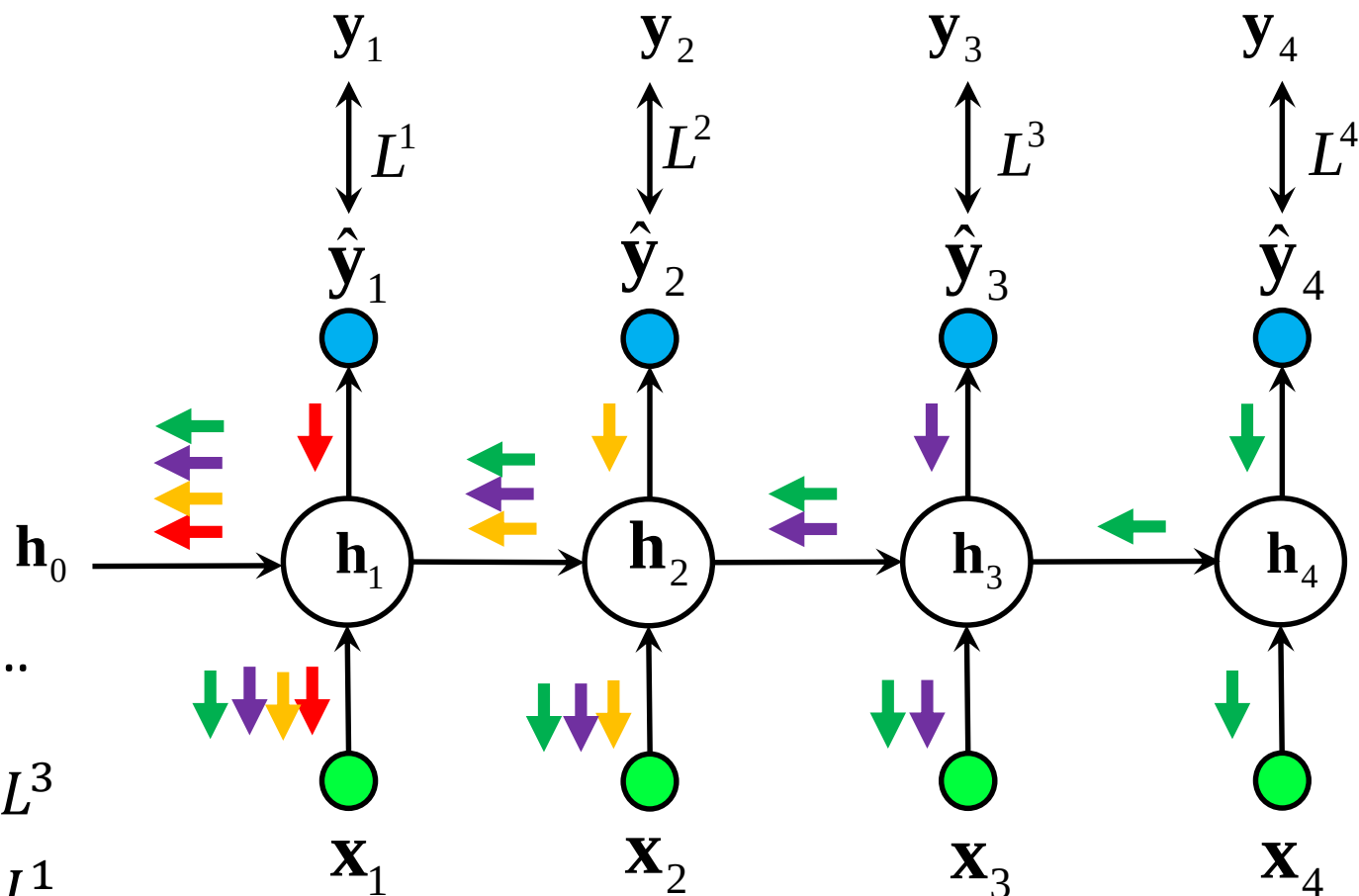
误差沿时间反传
Backpropagation
through Time (BPTT)

前向传播

误差反
向传播

计算 $h_1, h_2, h_3, h_4, \dots$

→ For L^4 → For L^3
→ For L^2 → For L^1



6.2 循环神经网络：优化学习

误差沿时间反传
Backpropagation
through Time (BPTT)

误差反
向传播

→ For L^4

→ For L^3

→ For L^2

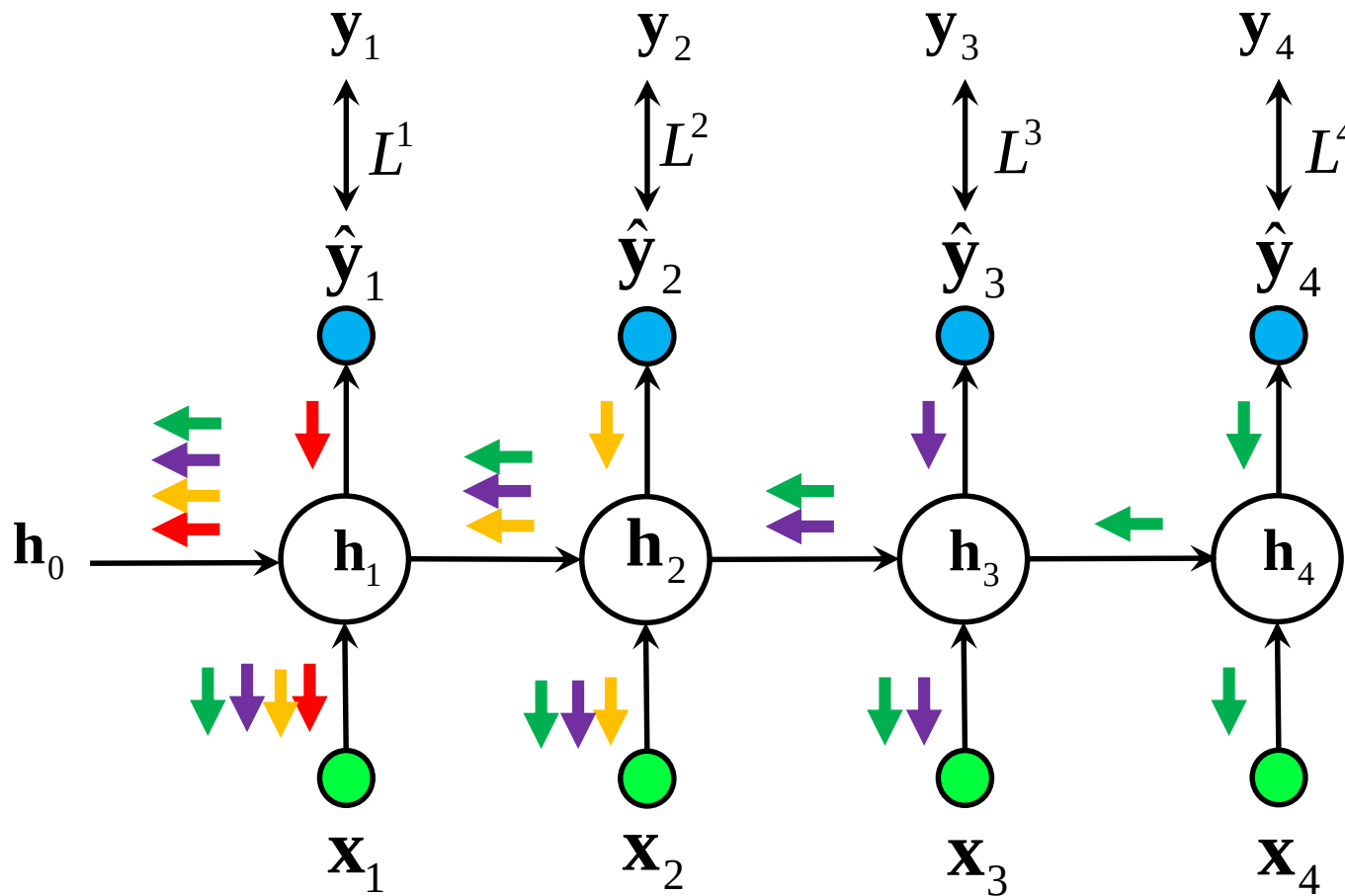
→ For L^1

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

$$\mathbf{o}_t = \text{softmax}(\hat{\mathbf{y}}_t)$$

$$\frac{\partial L}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^T \frac{\partial L^t}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^T \frac{\partial L^t}{\partial \hat{\mathbf{y}}_t} \frac{\partial \hat{\mathbf{y}}_t}{\partial \mathbf{W}_{hy}}$$



6.2 循环神经网络：优化学习

误差沿时间反传
Backpropagation
through Time (BPTT)

误差反
向传播

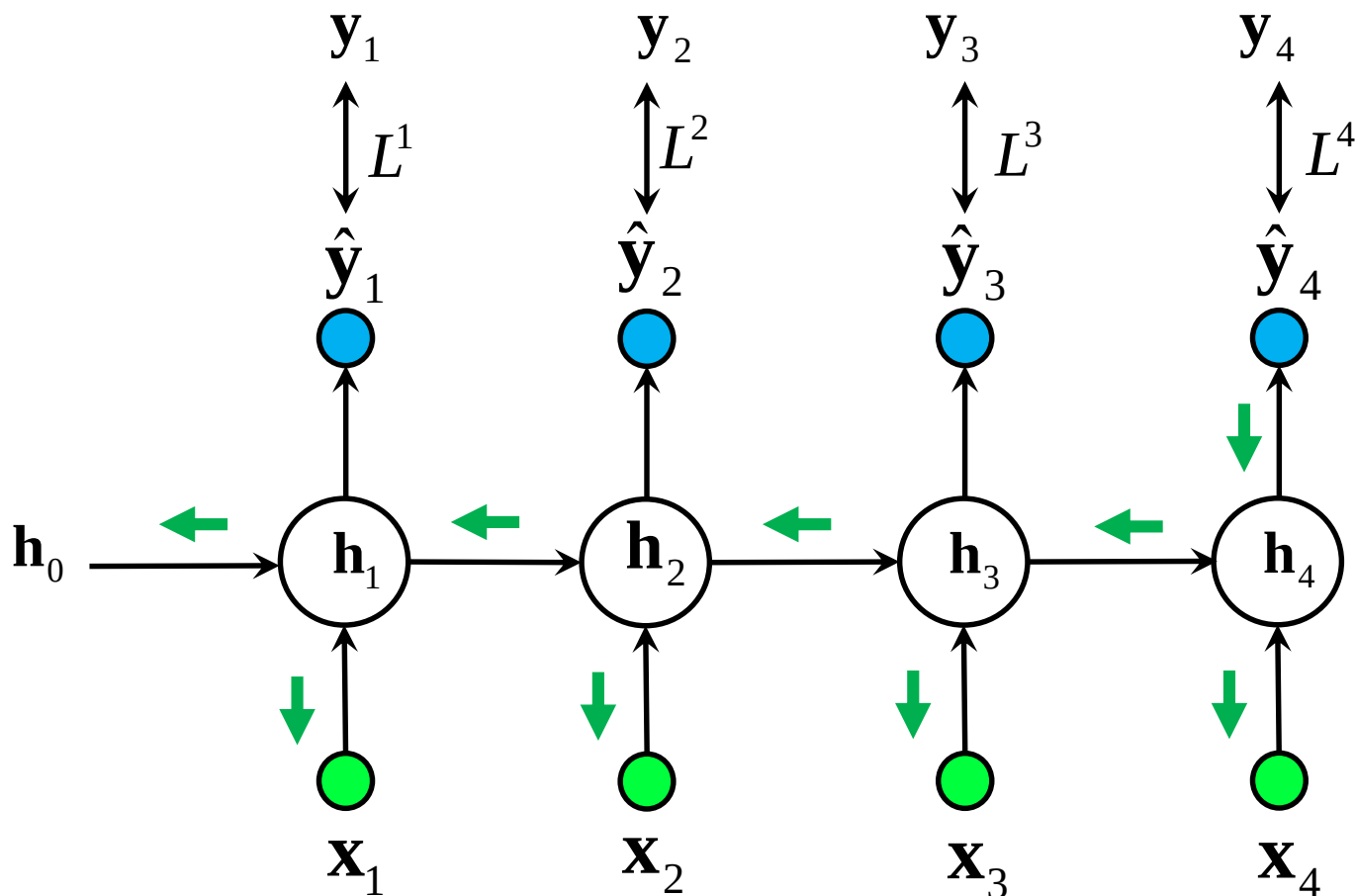
→ For L^4

$$\frac{\partial L^4}{\partial h_4} = \frac{\partial L^4}{\partial \hat{y}_4} \frac{\partial \hat{y}_4}{\partial h_4}$$

$$\frac{\partial L^4}{\partial h_3} = \frac{\partial L^4}{\partial \hat{y}_4} \frac{\partial \hat{y}_4}{\partial h_4} \frac{\partial h_4}{\partial h_3}$$

$$\frac{\partial L^4}{\partial h_2} = \frac{\partial L^4}{\partial \hat{y}_4} \frac{\partial \hat{y}_4}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2}$$

$$\frac{\partial L^4}{\partial h_1} = \frac{\partial L^4}{\partial \hat{y}_4} \frac{\partial \hat{y}_4}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1}$$



...

6.2 循环神经网络：优化学习

误差沿时间反传
Backpropagation
through Time (BPTT)

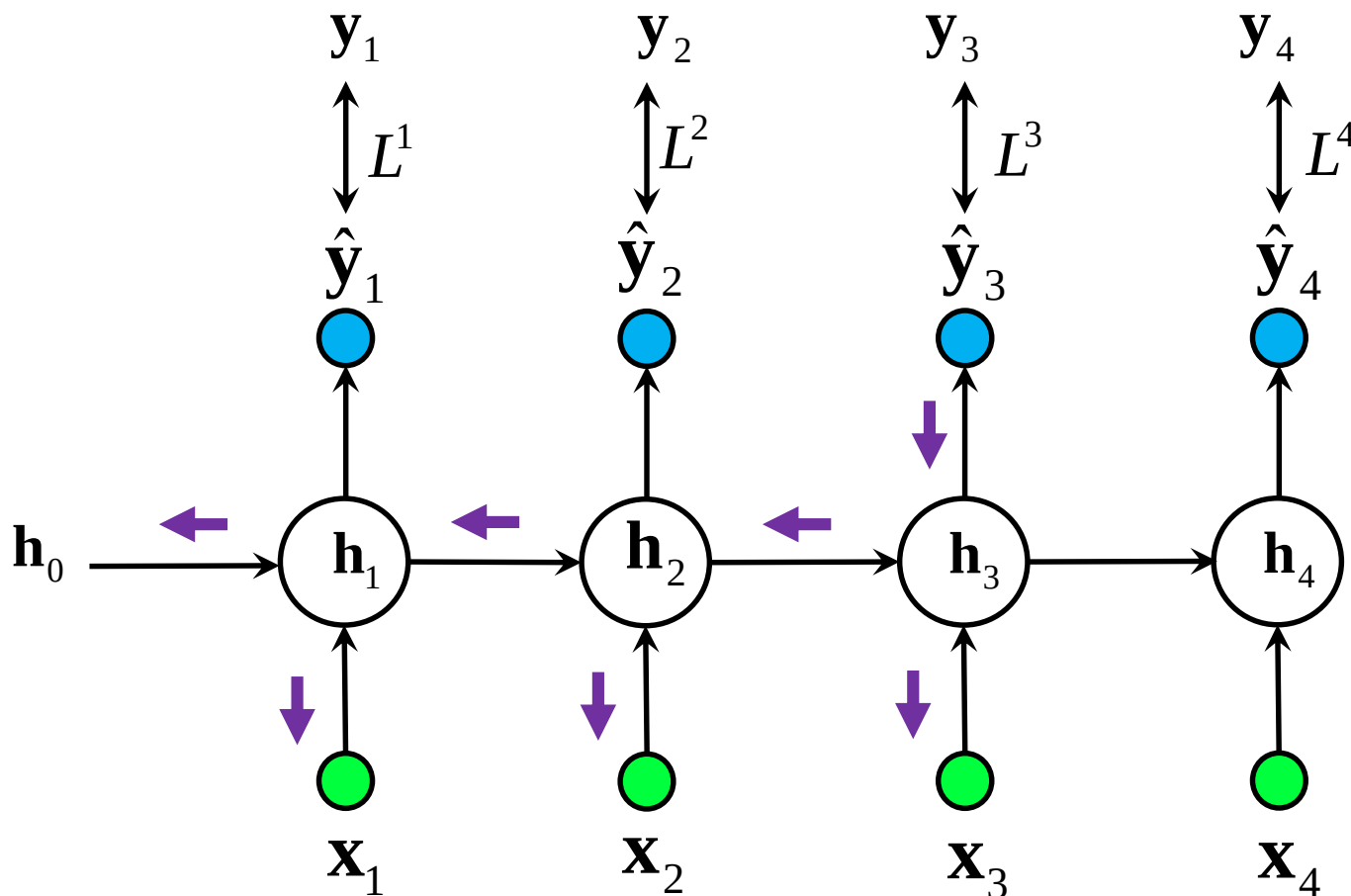
误差反
向传播

→ For L^3

$$\frac{\partial L^3}{\partial h_3} = \frac{\partial L^3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3}$$

$$\frac{\partial L^3}{\partial h_2} = \frac{\partial L^3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \frac{\partial h_3}{\partial h_2}$$

$$\frac{\partial L^3}{\partial h_1} = \frac{\partial L^3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1}$$



6.2 循环神经网络：优化学习

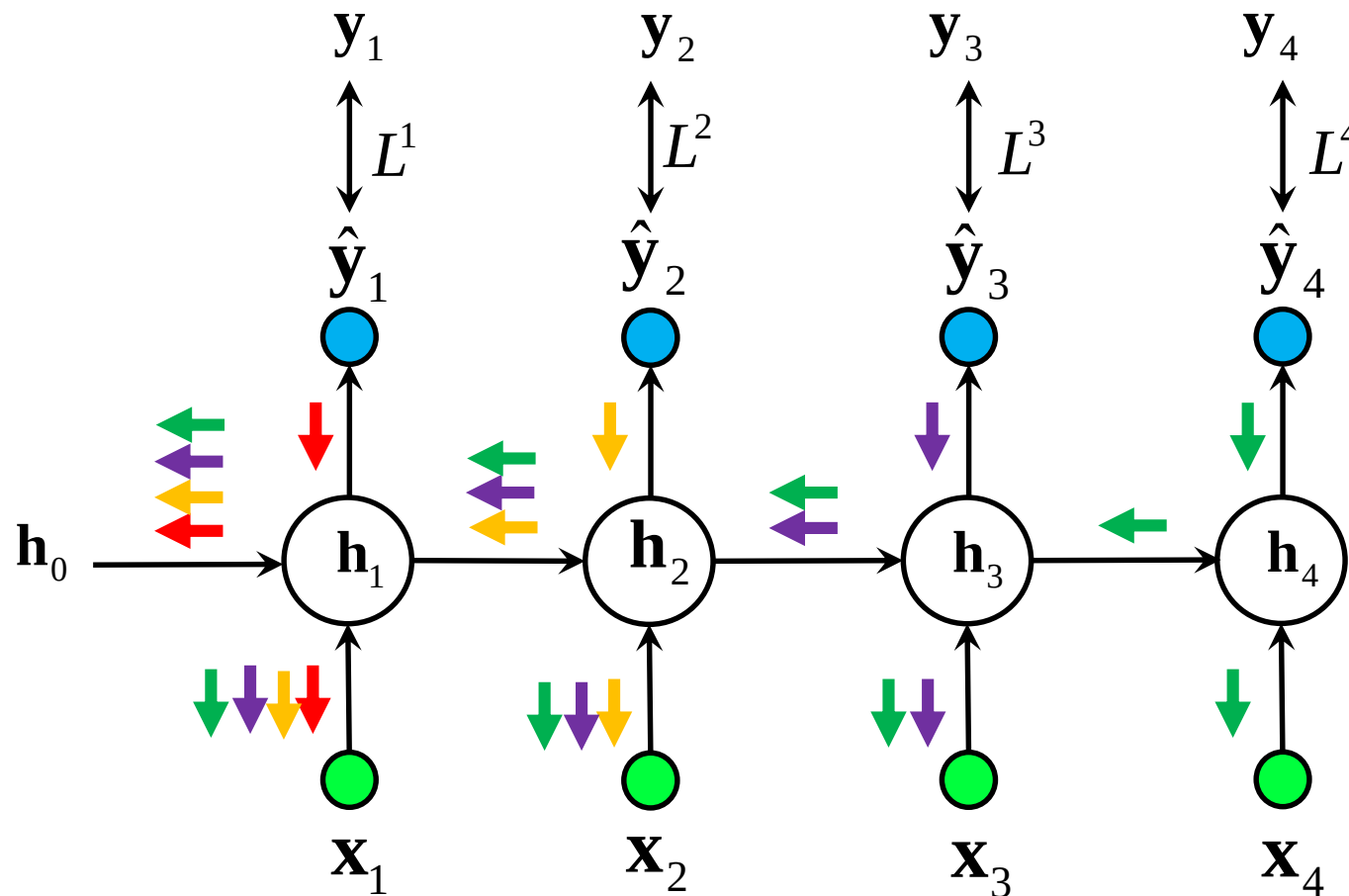
误差沿时间反传
Backpropagation
through Time (BPTT)

误差反向传播

→ For L^4 → For L^3
→ For L^2 → For L^1

$$\frac{\partial L^4}{\partial W_{hh}} = \frac{\partial L^4}{\partial h_4} \frac{\partial h_4}{\partial W_{hh}} + \frac{\partial L^4}{\partial h_3} \frac{\partial h_3}{\partial W_{hh}} + \dots \frac{\partial L^4}{\partial h_1} \frac{\partial h_1}{\partial W_{hh}}$$
$$\frac{\partial L^3}{\partial W_{hh}} = \frac{\partial L^3}{\partial h_3} \frac{\partial h_3}{\partial W_{hh}} + \dots \frac{\partial L^3}{\partial h_1} \frac{\partial h_1}{\partial W_{hh}}$$

...



$$\frac{\partial L^t}{\partial W_{hh}} = \sum_{k=1}^t \frac{\partial L^t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

6.2 循环神经网络：优化学习

误差沿时间反传
Backpropagation
through Time (BPTT)

误差反
向传播

→ For L^4 → For L^3
→ For L^2 → For L^1

$$\frac{\partial L^t}{\partial W_{hh}} = \sum_{k=1}^t \frac{\partial L^t}{\partial \hat{y}^t} \frac{\partial \hat{y}^t}{\partial h^t} \frac{\partial h^t}{\partial h^k} \frac{\partial h^k}{\partial W_{hh}}$$

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \mathbf{W}_{hy} \mathbf{h}_t + \mathbf{b}_y$$

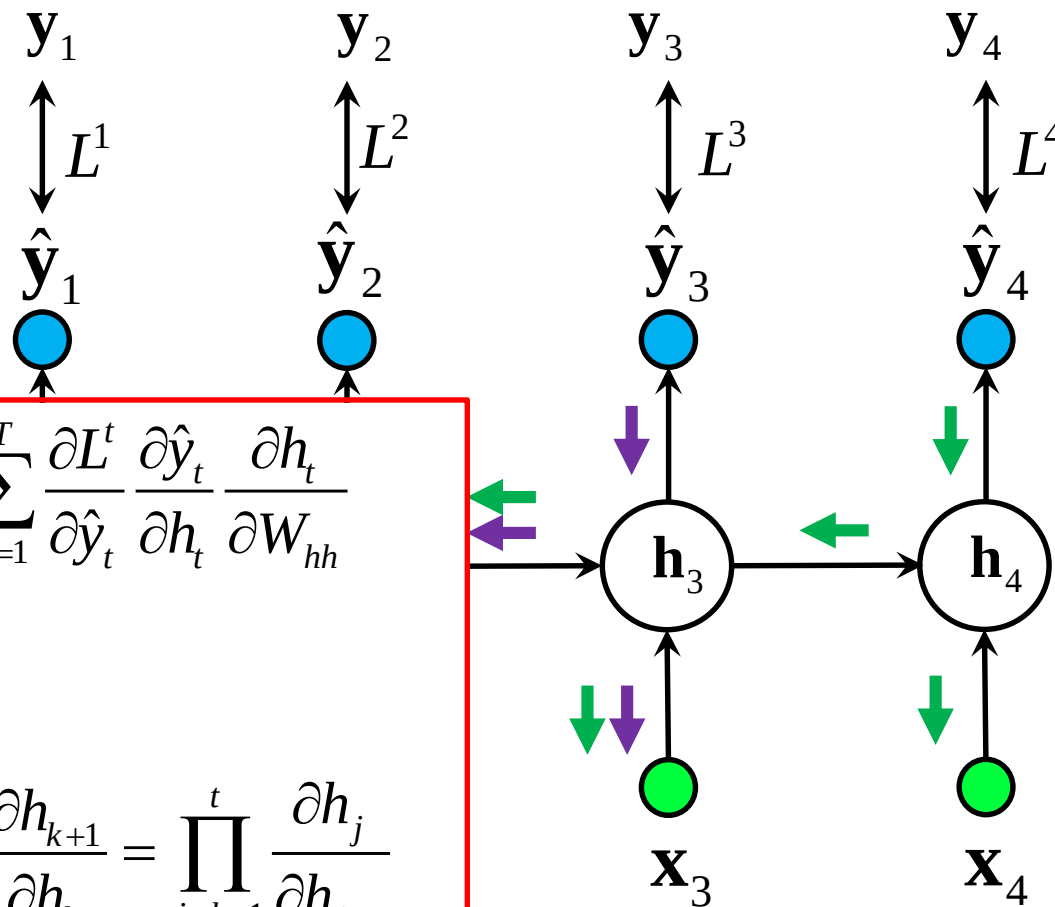
$$\mathbf{o}_t = \text{softmax}(\hat{\mathbf{y}}_t)$$

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L^t}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L^t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}}$$

$$\frac{\partial h_t}{\partial W_{hh}} = \sum_{k=1}^t \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

$$\frac{\partial h_t}{\partial h_k} = \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \dots \frac{\partial h_{k+1}}{\partial h_k} = \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}}$$

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial L^t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_{hh}}$$



6.2 循环神经网络：优化学习

误差沿时间反传
Backpropagation
through Time (BPTT)

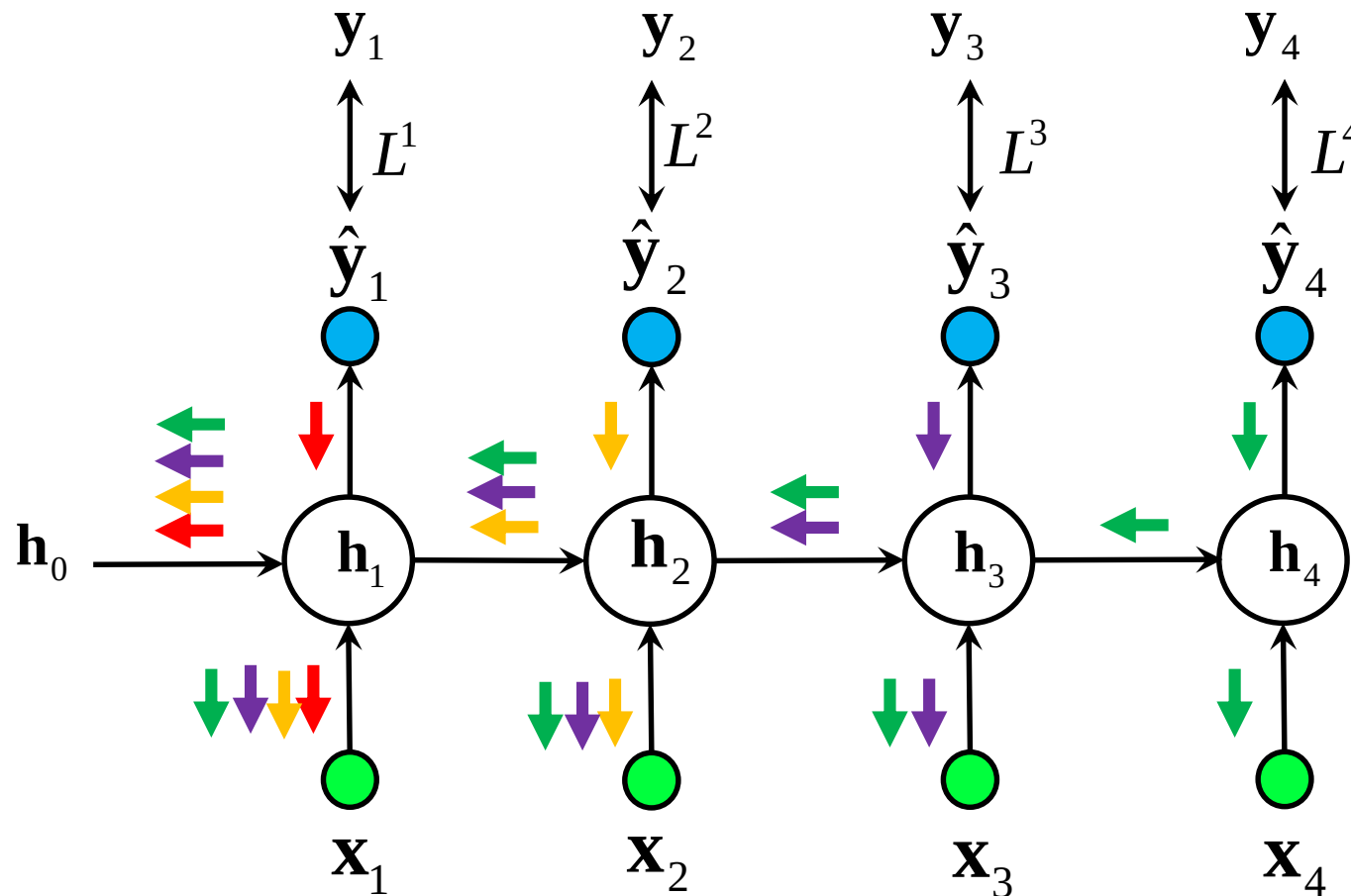
误差反
向传播

→ For L^4 → For L^3
→ For L^2 → For L^1

$$\frac{\partial L^4}{\partial W_{xh}} = \frac{\partial L^4}{\partial h_4} \frac{\partial h_4}{\partial W_{xh}} + \frac{\partial L^4}{\partial h_3} \frac{\partial h_3}{\partial W_{xh}} + \dots \frac{\partial L^4}{\partial h_1} \frac{\partial h_1}{\partial W_{xh}}$$

$$\frac{\partial L^3}{\partial W_{xh}} = \frac{\partial L^3}{\partial h_3} \frac{\partial h_3}{\partial W_{xh}} + \dots \frac{\partial L^3}{\partial h_1} \frac{\partial h_1}{\partial W_{xh}}$$

...



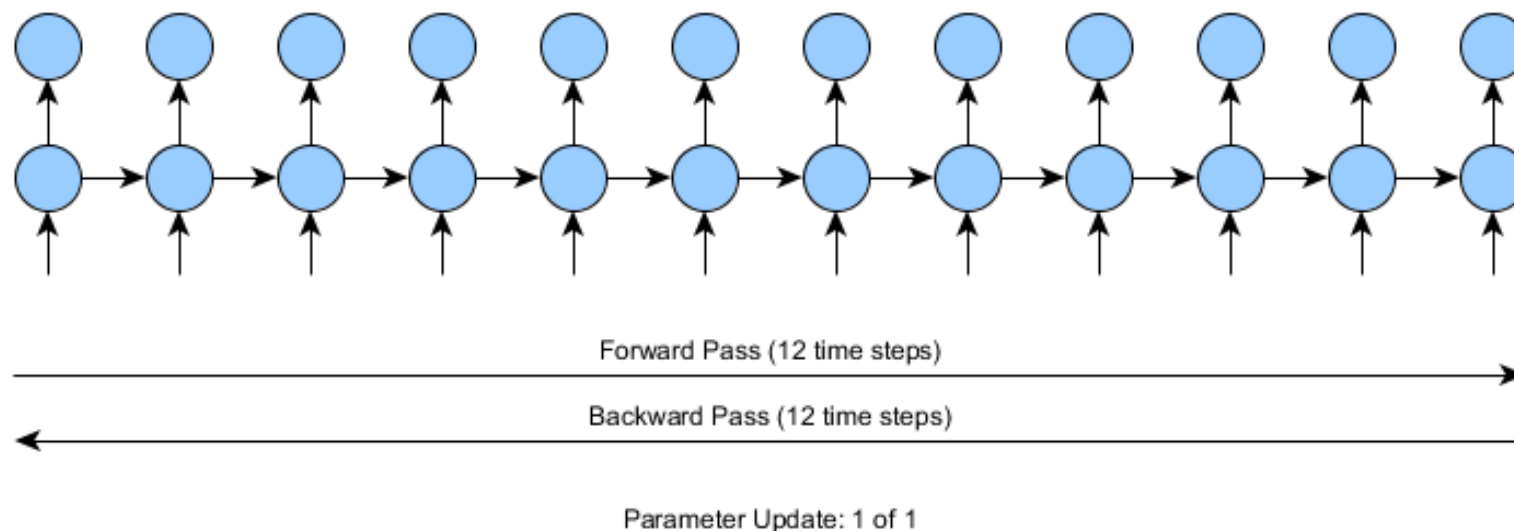
$$\frac{\partial L^t}{\partial W_{xh}} = \sum_{k=1}^t \frac{\partial L^t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_{xh}}$$

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial L^t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_{xh}}$$

6.2 循环神经网络：训练学习

➤ Truncated BPTT

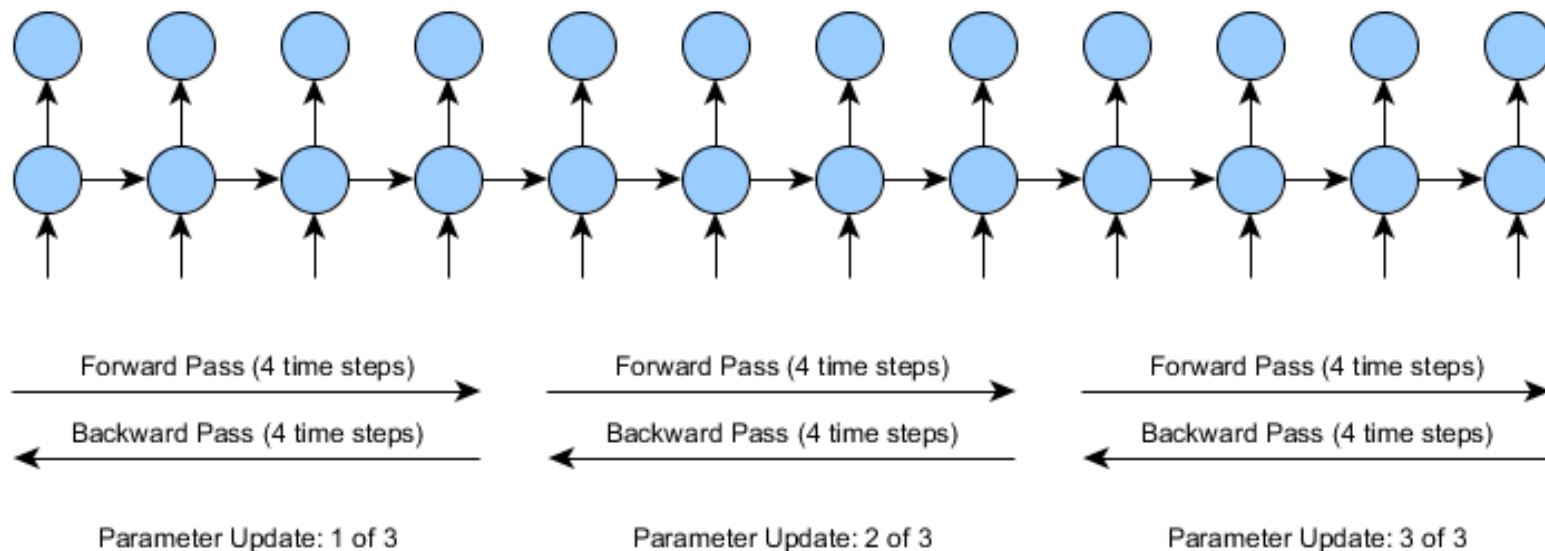
- ◆ 问题：我们需要把误差反传到最初时刻吗？
- ◆ 理论上需要，但实际上我们采用时间截断的BPTT，误差仅在有限时间内反传
- ◆ 另外，时序数据通常包含多个较短的序列数据



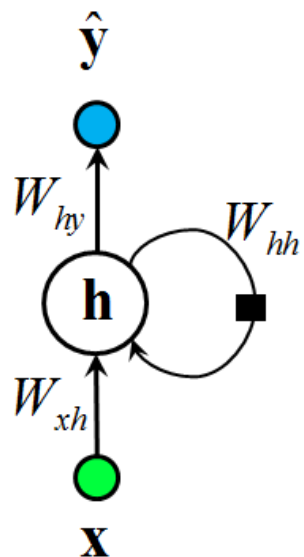
6.2 循环神经网络：训练学习

➤ Truncated BPTT

- ◆ 问题：我们需要把误差反传到最初时刻吗？
- ◆ 理论上需要，但实际上我们采用时间截断的BPTT，误差仅在有限时间内反传
- ◆ 另外，时序数据通常包含多个较短的序列数据



6.2 循环神经网络：梯度消失与梯度爆炸



$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

$$\mathbf{o}_t = \text{softmax}(\hat{\mathbf{y}}_t)$$

$$\frac{\partial h_j}{\partial h_{j-1}} = \tanh'(\dots) \cdot W_{hh}$$

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^T \frac{\partial L^t}{\partial W_{hy}} = \sum_{t=1}^T \frac{\partial L^t}{\partial \hat{\mathbf{y}}_t} \frac{\partial \hat{\mathbf{y}}_t}{\partial W_{hy}}$$

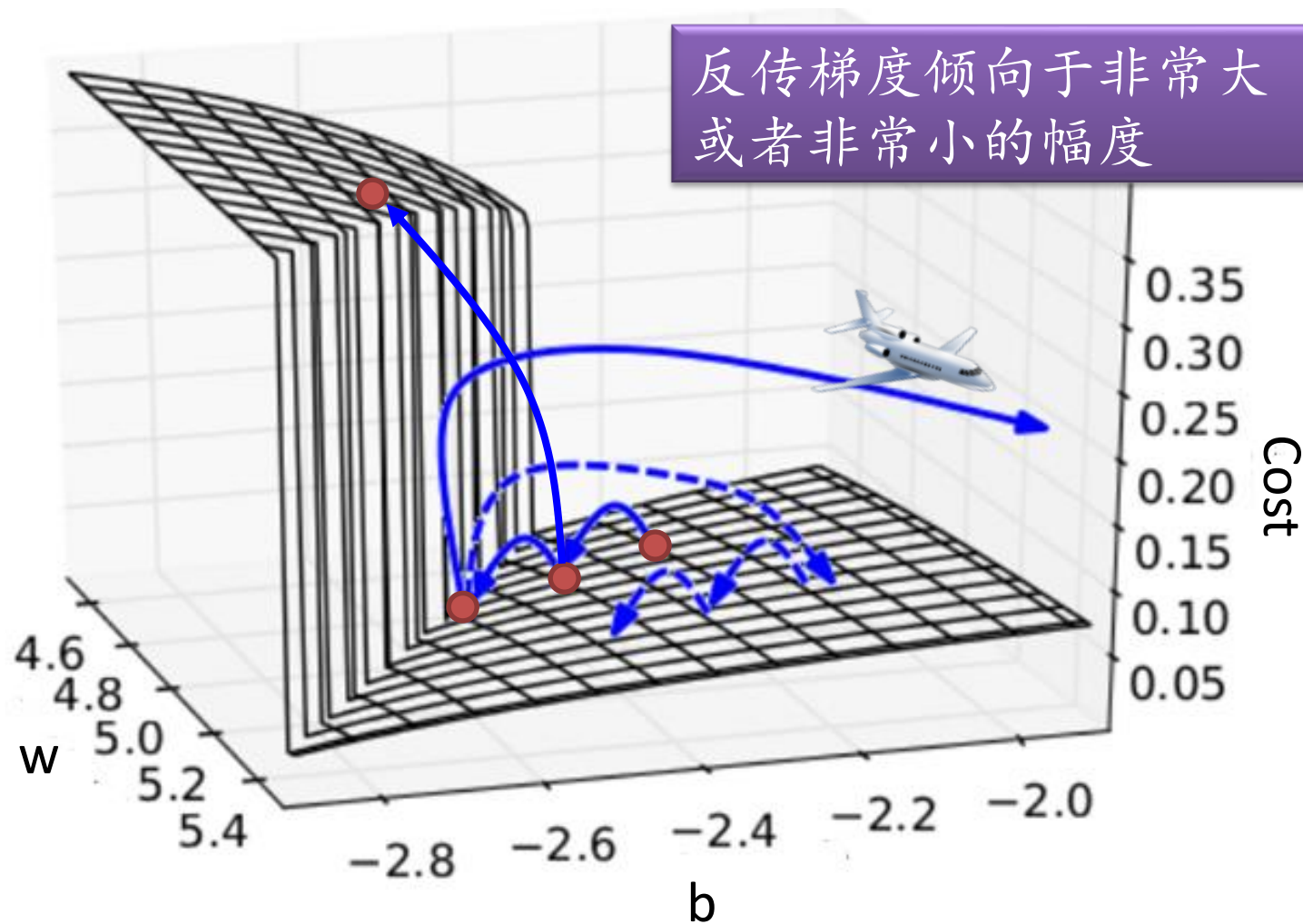
$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial L^t}{\partial \hat{\mathbf{y}}_t} \frac{\partial \hat{\mathbf{y}}_t}{\partial \mathbf{h}_t} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_{hh}}$$

$\tanh'(\dots) \cdot W_{hh} > 1$, 连乘项多则梯度爆炸

$\tanh'(\dots) \cdot W_{hh} < 1$, 连乘项多则梯度消失

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial L^t}{\partial \hat{\mathbf{y}}_t} \frac{\partial \hat{\mathbf{y}}_t}{\partial \mathbf{h}_t} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_{xh}}$$

6.2 循环神经网络：梯度消失与梯度爆炸

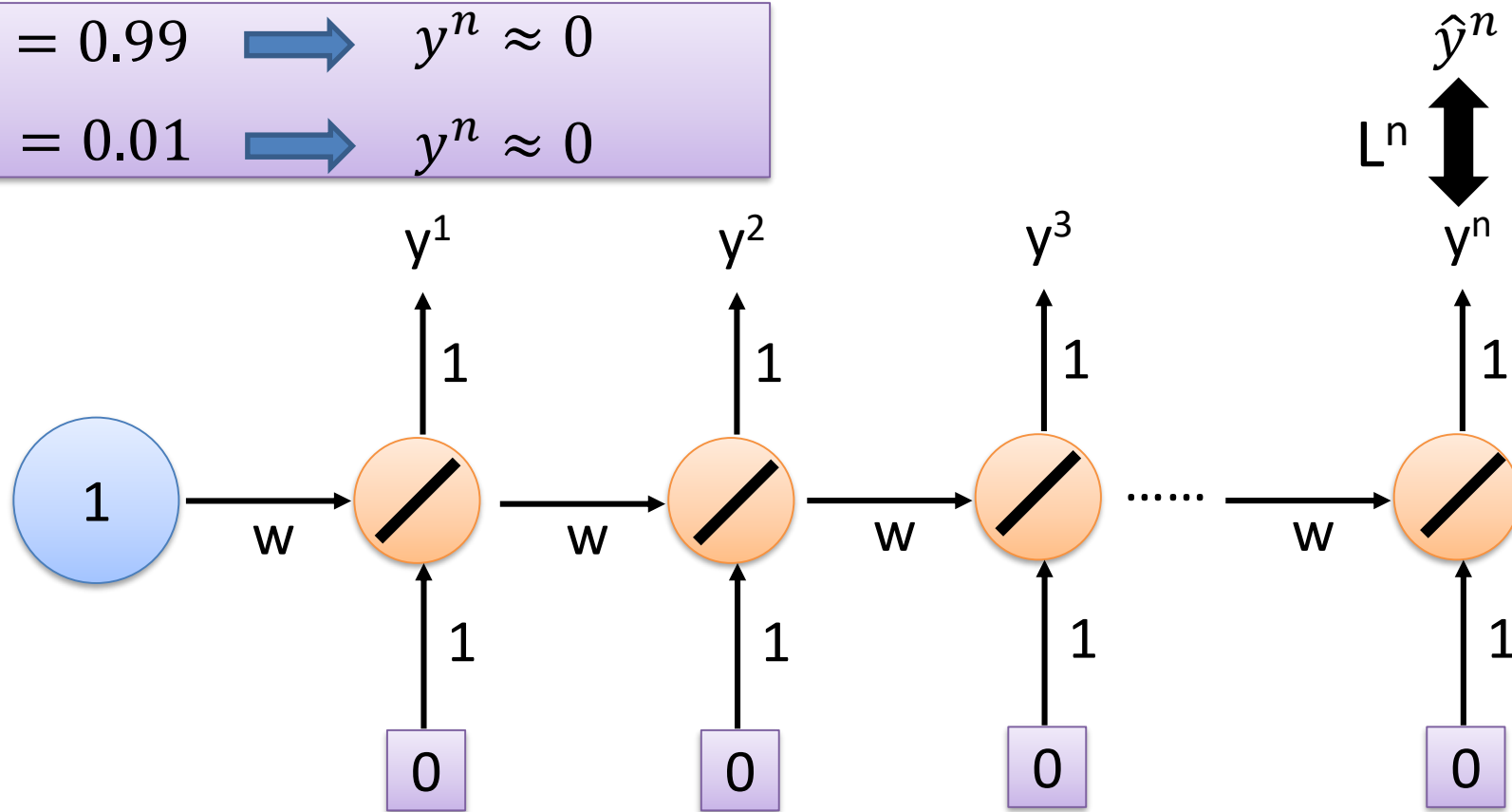


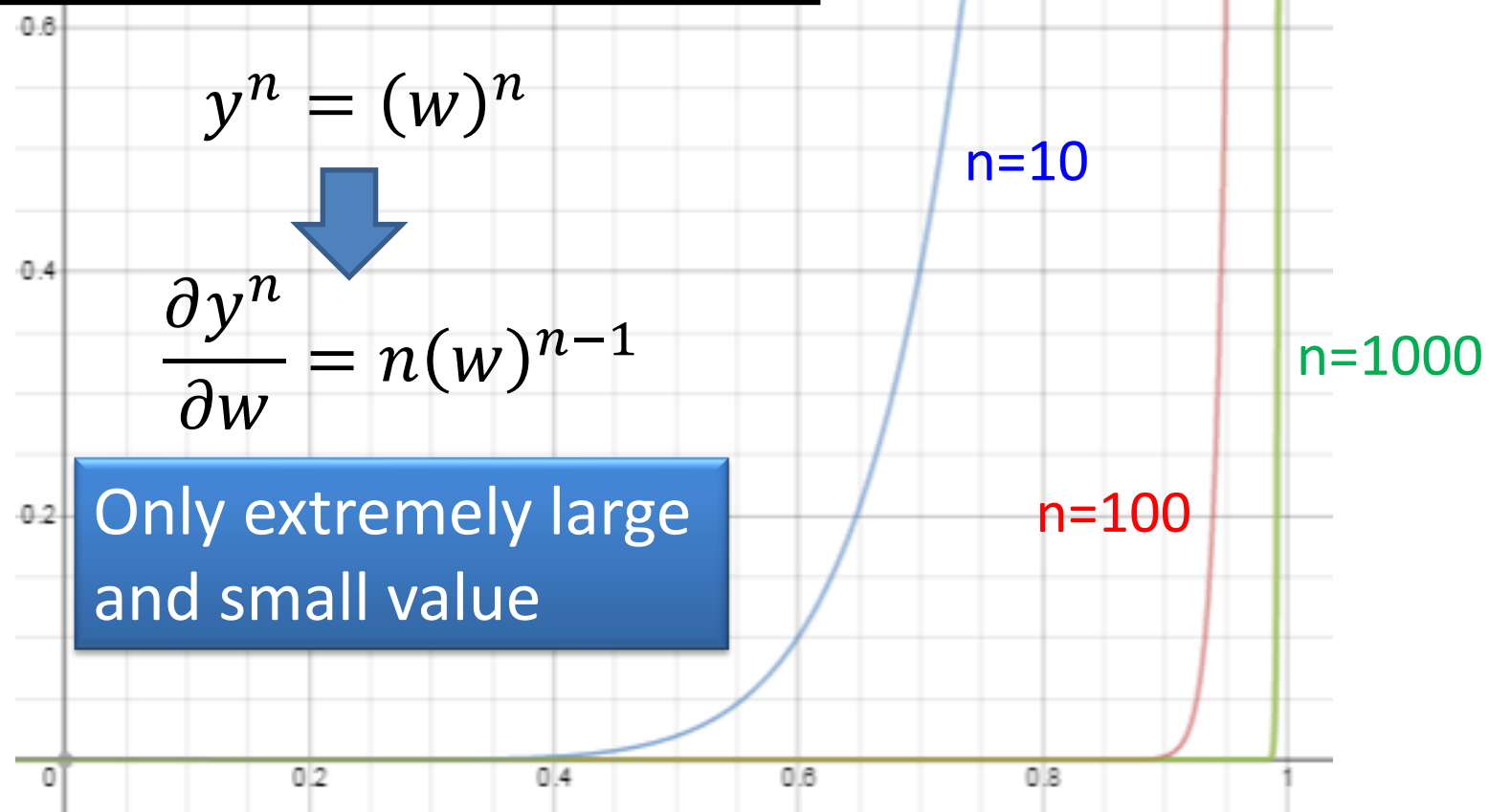
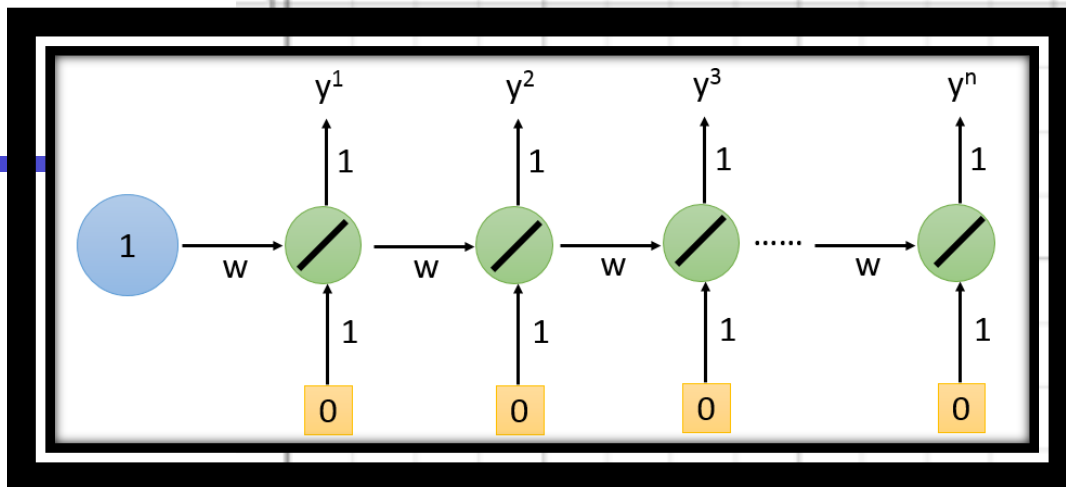
6.2 循环神经网络：梯度消失与梯度爆炸

长序列 $n = 1000$:

$w = 1$	$\longrightarrow$	$y^n = 1$
$w = 1.01$	$\longrightarrow$	$y^n \approx 20000$
$w = 0.99$	$\longrightarrow$	$y^n \approx 0$
$w = 0.01$	$\longrightarrow$	$y^n \approx 0$

$$\frac{\partial L^n}{\partial w} = \frac{\partial L^n}{\partial y^n} \frac{\partial y^n}{\partial w} \quad \frac{\partial y^n}{\partial w} \approx \frac{\Delta y^n}{\Delta w}$$





6.2 循环神经网络

► 梯度消失与梯度爆炸

◆ 解决方案1: gradient clipping

Algorithm 1 Pseudo-code for norm clipping the gradients whenever they explode

$$\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$$
$$\text{if } \|\hat{\mathbf{g}}\| \geq \textit{threshold} \text{ then}$$
$$\quad \hat{\mathbf{g}} \leftarrow \frac{\textit{threshold}}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}}$$
$$\text{end if}$$

◆ 解决方案2: 替换tanh激活函数为relu函数

◆ 解决方案3: 引入门控机制

6.2 循环神经网络 (pytorch)

CLASS `torch.nn.RNN(*args, **kwargs)`

[\[SOURCE\]](#)

Applies a multi-layer Elman RNN with `tanh` or `ReLU` non-linearity to an input sequence.

For each element in the input sequence, each layer computes the following function:

$$h_t = \tanh(W_{ih}x_t + b_{ih} + W_{hh}h_{(t-1)} + b_{hh})$$

where h_t is the hidden state at time t , x_t is the input at time t , and $h_{(t-1)}$ is the hidden state of the previous layer at time $t-1$ or the initial hidden state at time 0. If `nonlinearity` is `'relu'`, then `ReLU` is used instead of `tanh`.

Parameters

- **input_size** – The number of expected features in the input x
- **hidden_size** – The number of features in the hidden state h
- **num_layers** – Number of recurrent layers. E.g., setting `num_layers=2` would mean stacking two RNNs together to form a *stacked RNN*, with the second RNN taking in outputs of the first RNN and computing the final results. Default: 1

6.2 循环神经网络 (pytorch)

CLASS `torch.nn.RNN(*args, **kwargs)`

[\[SOURCE\]](#)

Applies a multi-layer Elman RNN with `tanh` or `ReLU` non-linearity to an input sequence.

For each element in the input sequence, each layer computes the following function:

$$h_t = \tanh(W_{ih}x_t + b_{ih} + W_{hh}h_{(t-1)} + b_{hh})$$

where h_t is the hidden state at time t , x_t is the input at time t , and $h_{(t-1)}$ is the hidden state of the previous layer at time $t-1$ or the initial hidden state at time 0. If `nonlinearity` is `'relu'`, then `ReLU` is used instead of `tanh`.

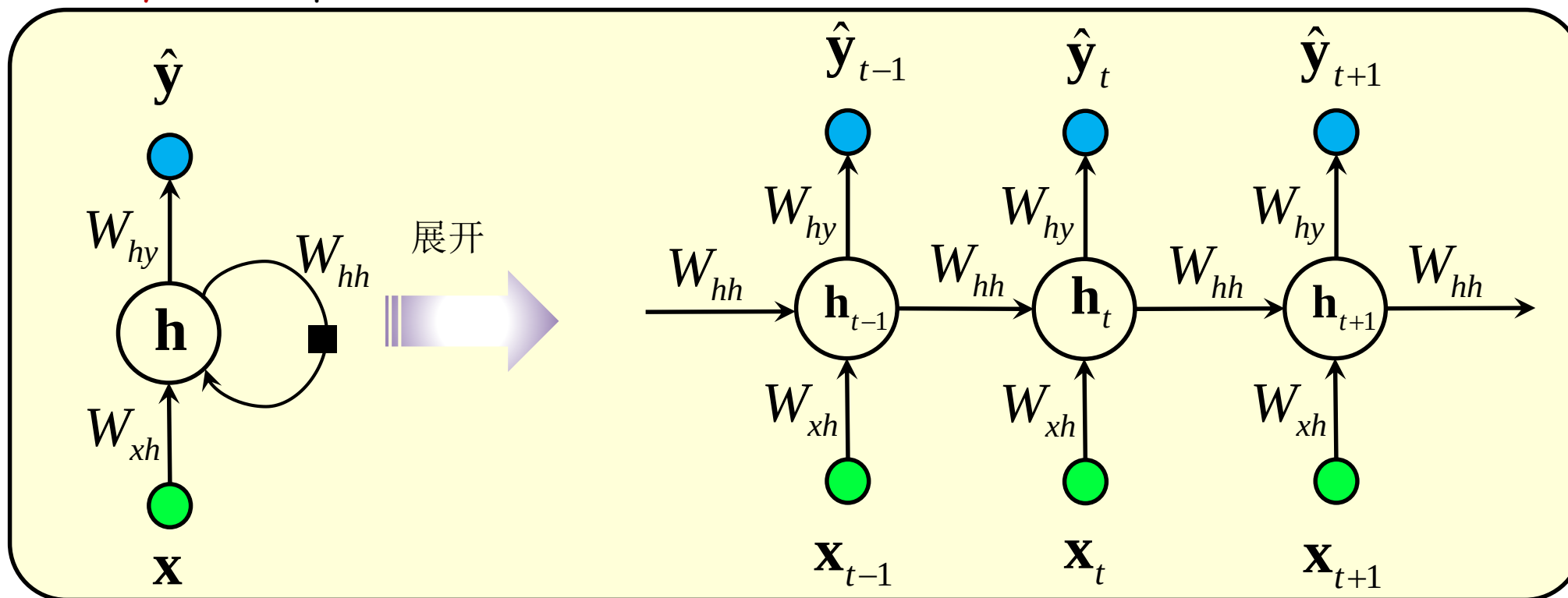
Parameters

- **input_size** – The number of expected features in the input x
- **hidden_size** – The number of features in the hidden state h
- **num_layers** – Number of recurrent layers. E.g., setting `num_layers=2` would mean stacking two RNNs together to form a *stacked RNN*, with the second RNN taking in outputs of the first RNN and computing the final results. Default: 1

6.2 循环神经网络：长时依赖

Mary and Henry, who helped a lot in old MacDonald's farm, were rewarded a little lamb.

Mary, who helped a lot in old MacDonald's farm, was rewarded a little lamb.



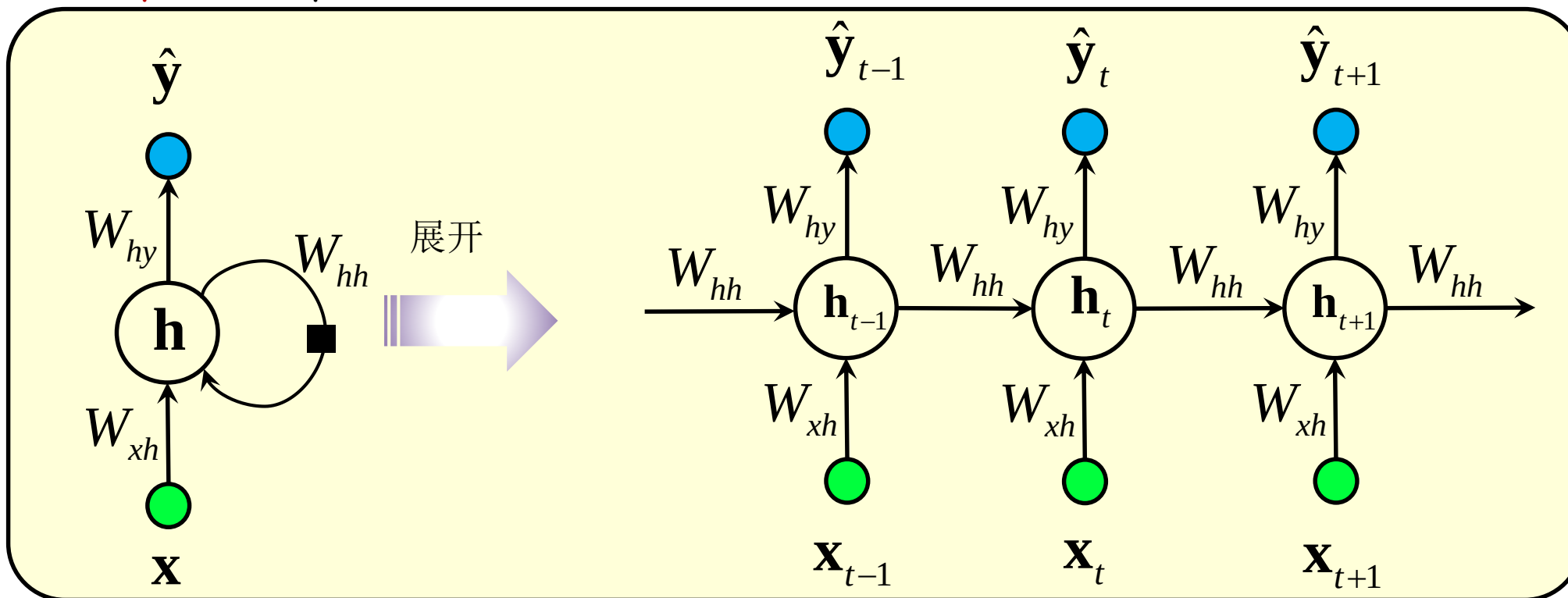
玛丽经常去老麦当劳家的农场帮忙，因此被奖励了一只小羊。

玛丽和亨利经常去老麦当劳家的农场帮忙，因此被奖励了一只小羊。

6.2 循环神经网络：长时依赖

Mary and Henry, who helped a lot in old MacDonald's farm, were rewarded a little lamb.

Mary, who helped a lot in old MacDonald's farm, was rewarded a little lamb.



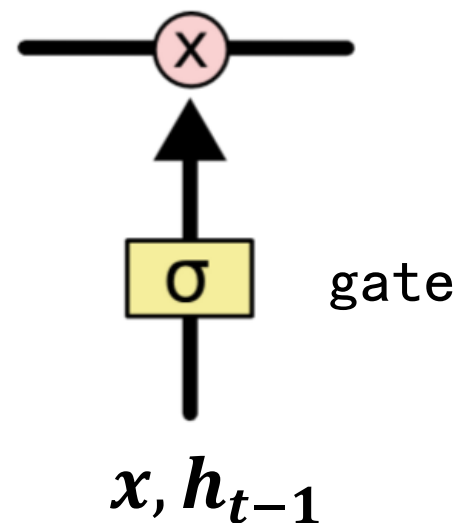
问题：在相关信息和预测词之间间隔不断增大时，RNN 会丧失学习到连接如此远的信息的能力。

6.2 循环神经网络：长时依赖

➤ 门控机制

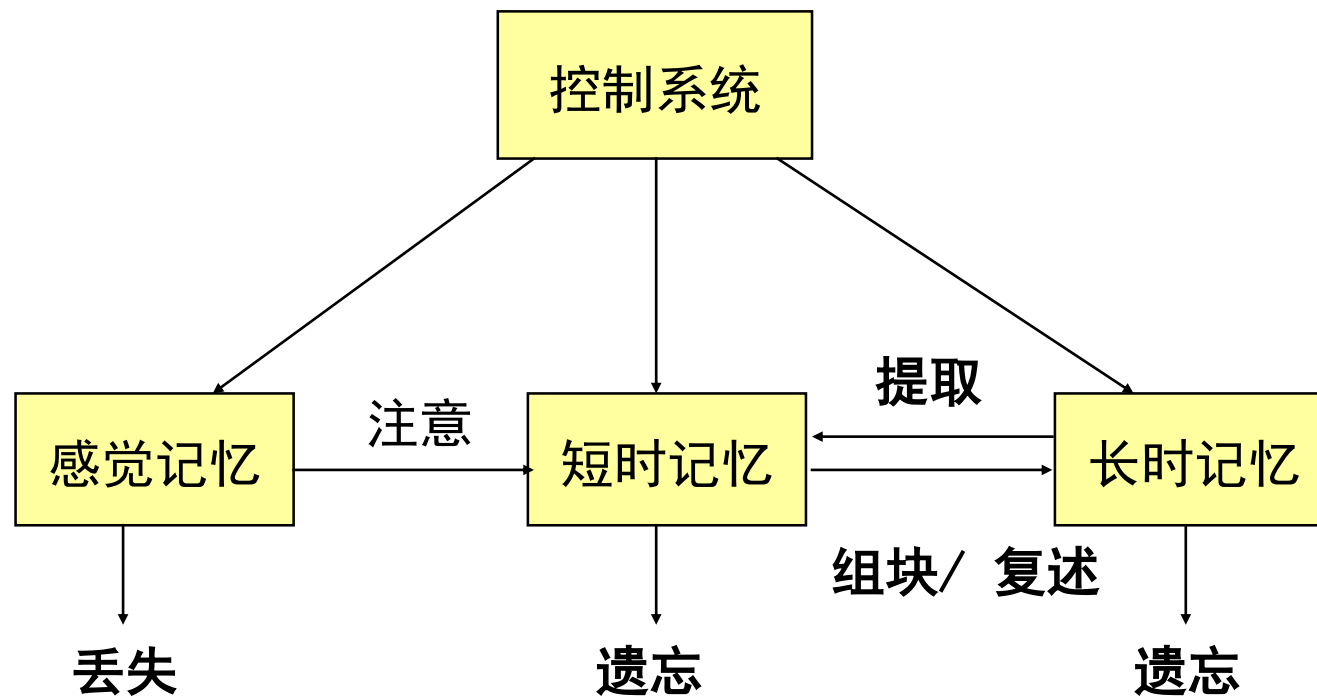
- ◆ 采用门（Gates）来控制数据的流动

$$gate = \sigma(\mathbf{w}_{xg}\mathbf{x}_t + \mathbf{w}_{hg}h_{t-1} + b_g)$$



6.2.2 长短时记忆网络

➤ 回顾：认知心理学中的记忆机制



三种记忆子系统与控制系统间的关系

6.2.2 长短时记忆网络

Long Short Term Memory (LSTM)

➤ Hochreiter et al. 1997年提出

输入节点: $g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g)$

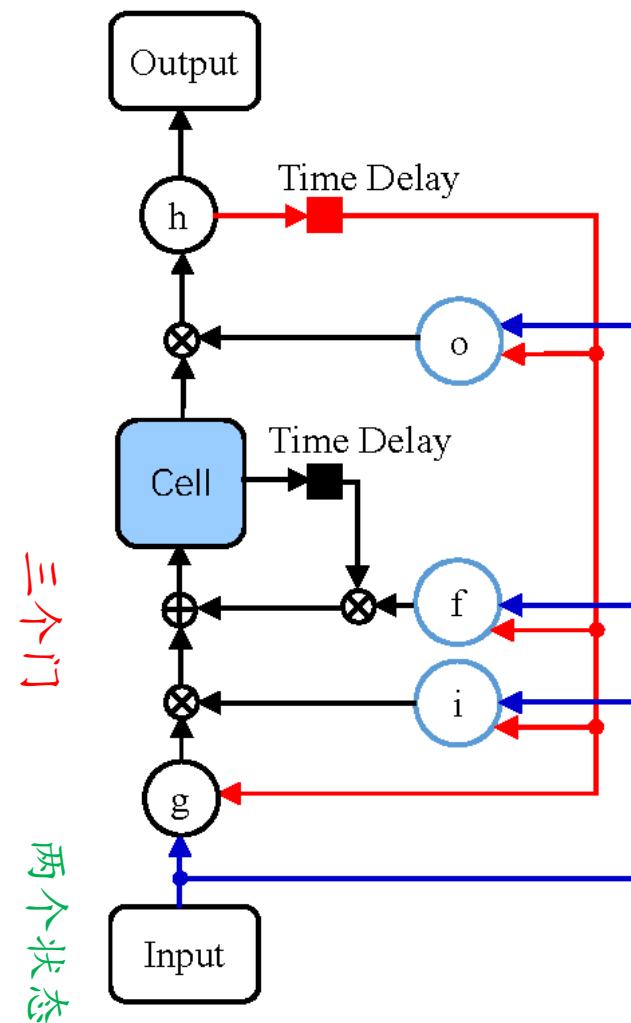
输入门: $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$

遗忘门: $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$

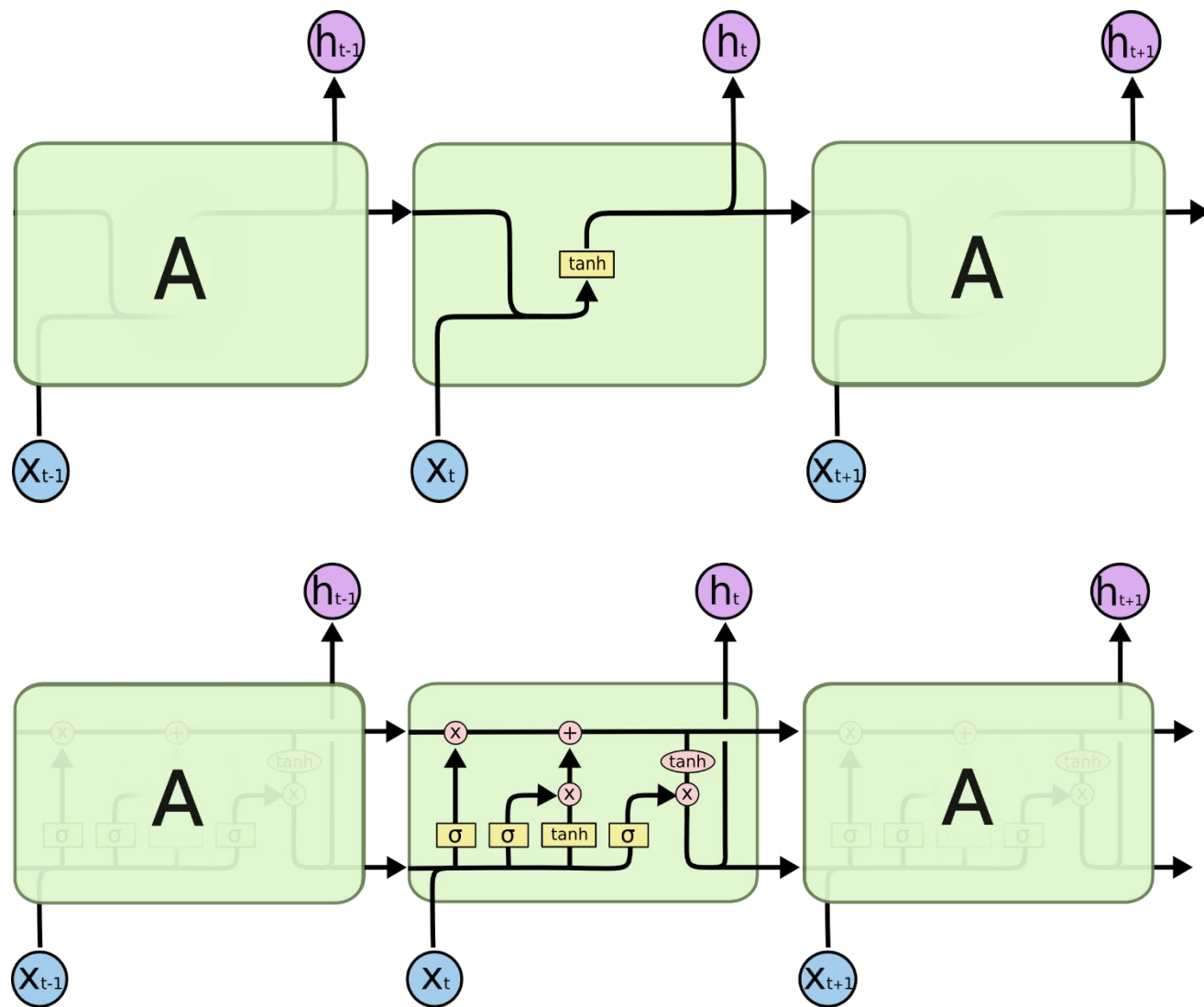
输出门: $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$

细胞状态: $c_t = i_t \odot g_t + f_t \odot c_{t-1}$

输出状态: $h_t = o_t \odot \tanh(c_t)$



6.2.2 长短时记忆网络



RNN

$$\mathbf{h}_t = \tanh(\mathbf{b}_h + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t)$$

LSTM

输入节点: $g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g)$

输入门: $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$

遗忘门: $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$

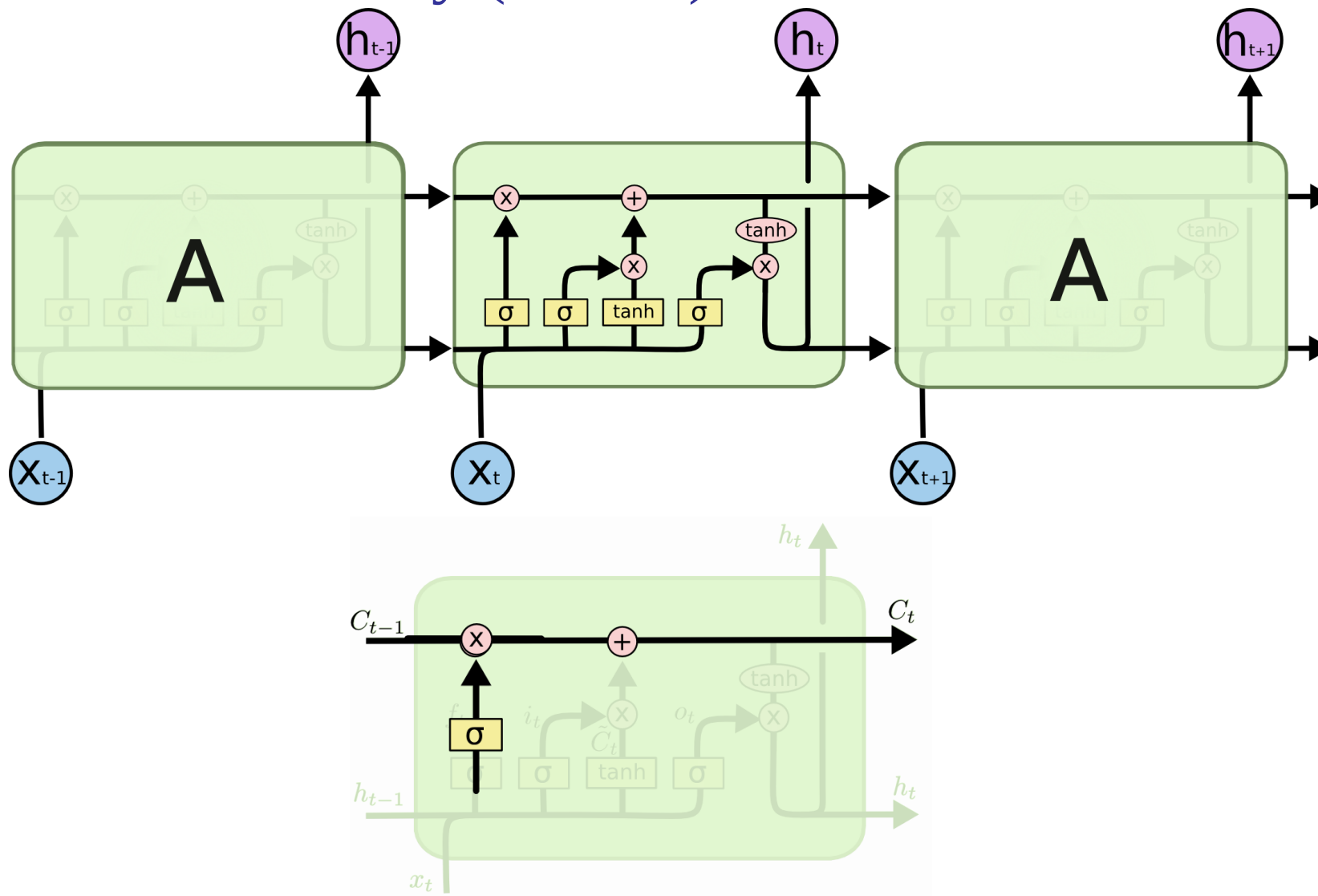
输出门: $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$

细胞状态: $c_t = i_t \odot g_t + f_t \odot c_{t-1}$

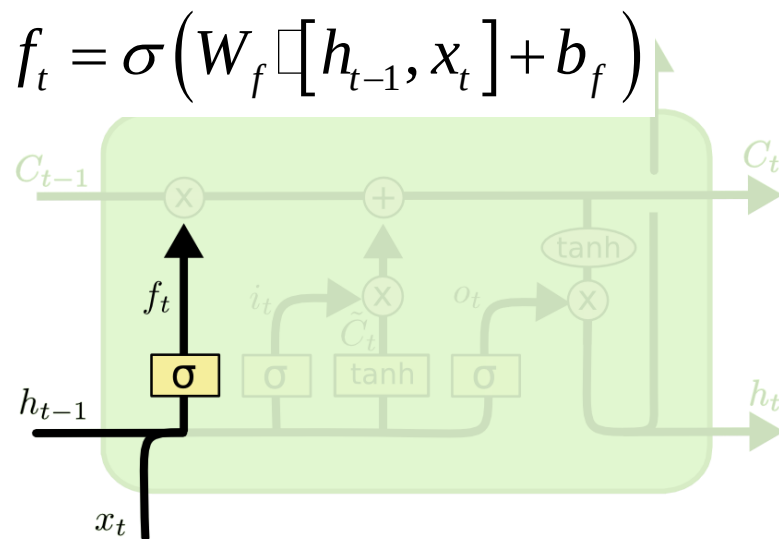
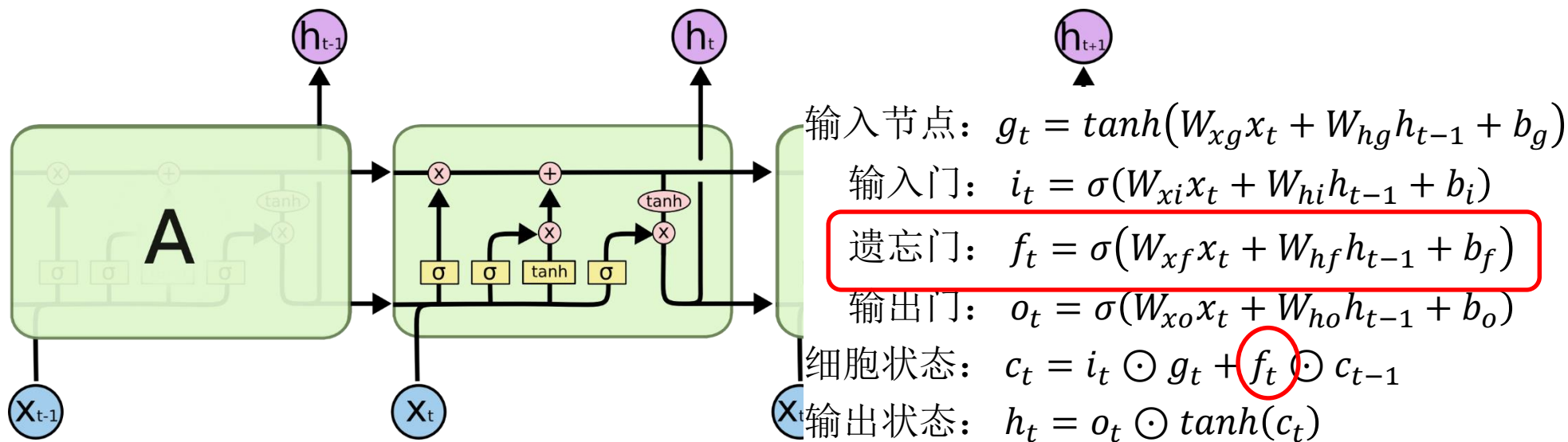
输出状态: $h_t = o_t \odot \tanh(c_t)$

6.2.2 长短时记忆网络

➤ Long Short Term Memory (LSTM)



6.2.2 长短时记忆网络

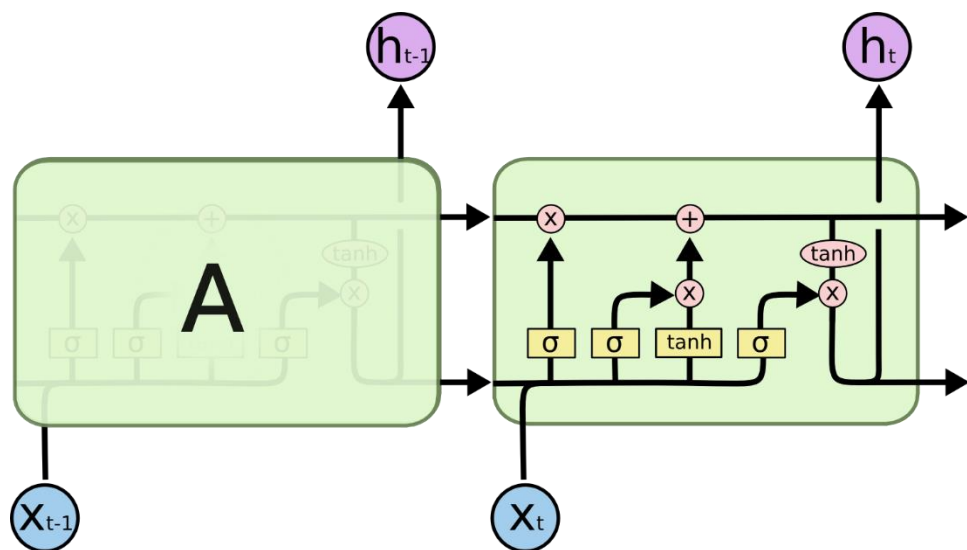


遗忘门 forget gate:

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$$

决定细胞状态 c_{t-1} 有多少信息将被遗忘;
 $f_t = 1$, 全部保留; $f_t = 0$, 全部遗忘。

6.2.2 长短时记忆网络



输入节点: $g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g)$

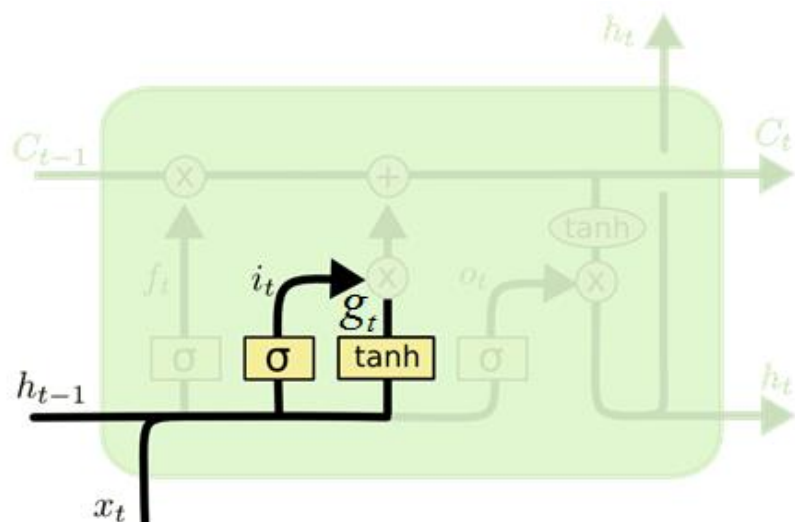
输入门: $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$

遗忘门: $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$

输出门: $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$

细胞状态: $c_t = i_t \odot g_t + f_t \odot c_{t-1}$

(输出状态: $h_t = o_t \odot \tanh(c_t)$)



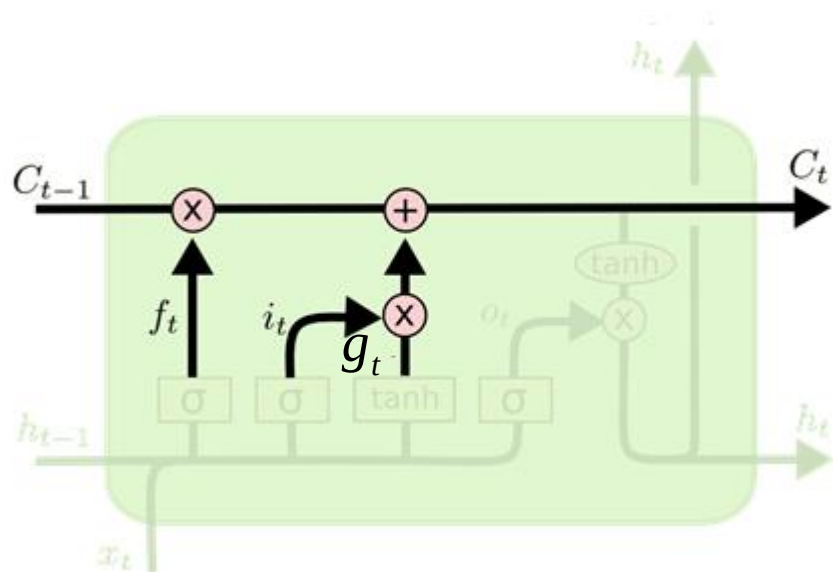
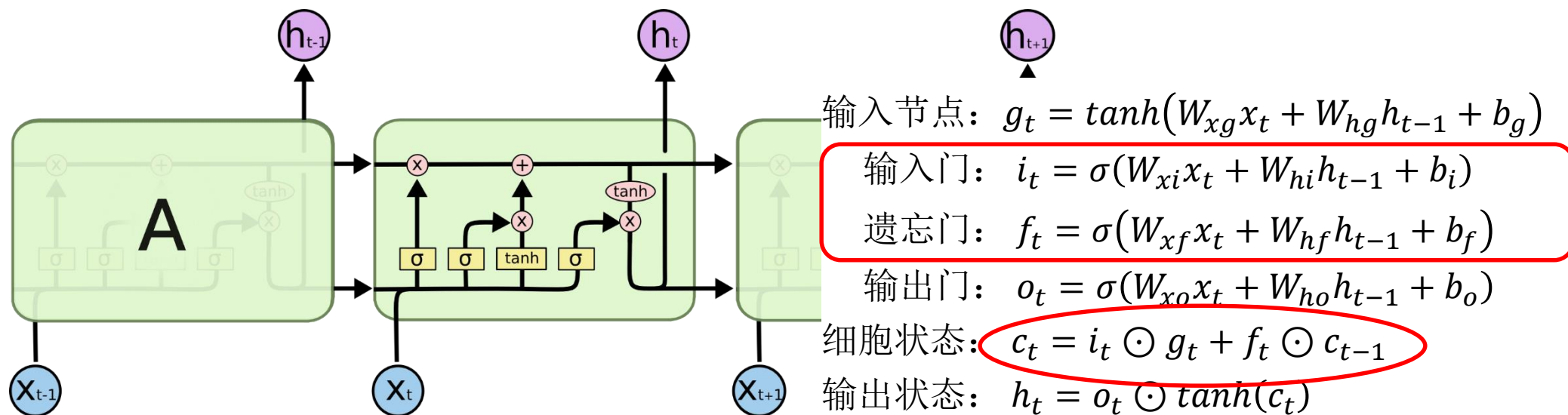
输入门 input gate:

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i)$$

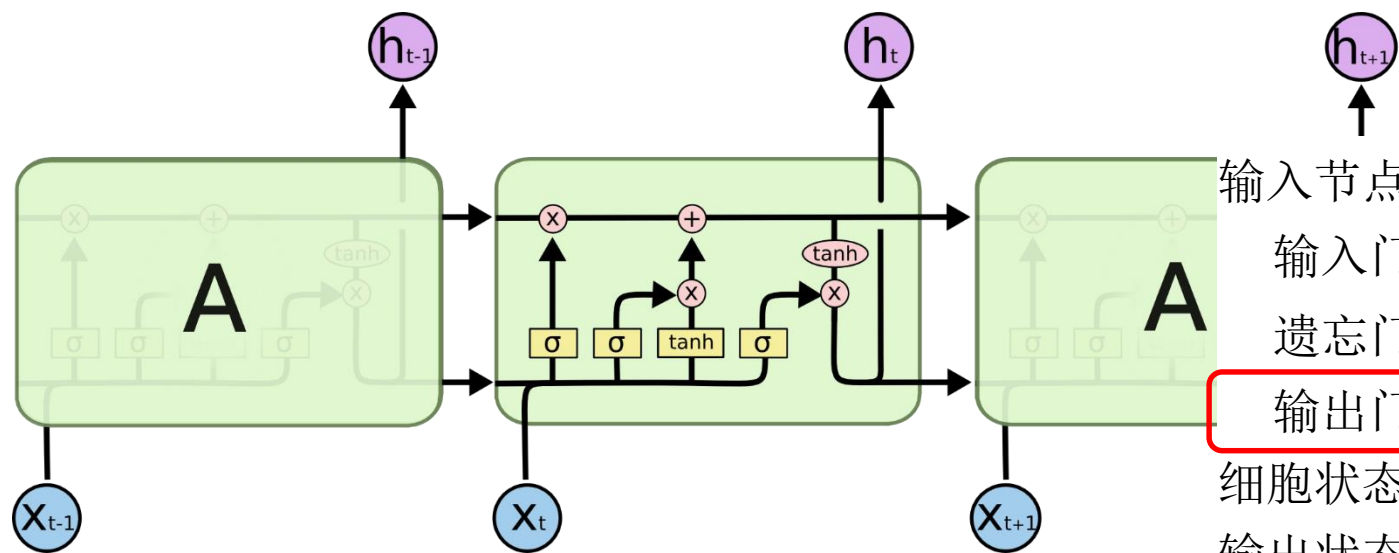
$$g_t = \tanh(W_g[x_t, h_{t-1}] + b_g)$$

决定输入节点 g_t 有多少信息将被登记;
 $i_t = 1$, 登记; $i_t = 0$, 丢失。

6.2.2 长短时记忆网络



6.2.2 长短时记忆网络



输入节点: $g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g)$

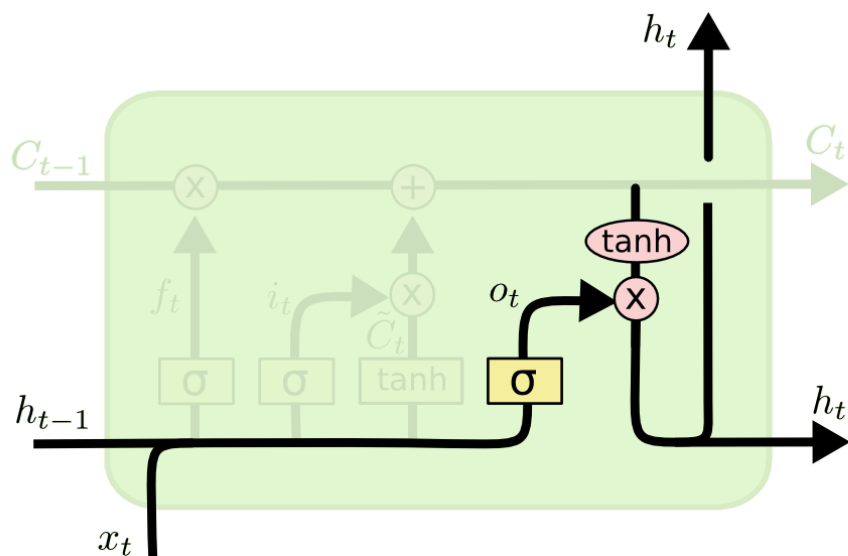
输入门: $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$

遗忘门: $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$

输出门: $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$

细胞状态: $c_t = i_t \odot g_t + f_t \odot c_{t-1}$

输出状态: $h_t = o_t \odot \tanh(c_t)$



输出门以及输出节点:

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

6.2.2 长短时记忆网络

输入节点: $g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g)$

输入门: $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$

遗忘门: $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$

输出门: $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$

细胞状态: $c_t = i_t \odot g_t + f_t \odot c_{t-1}$

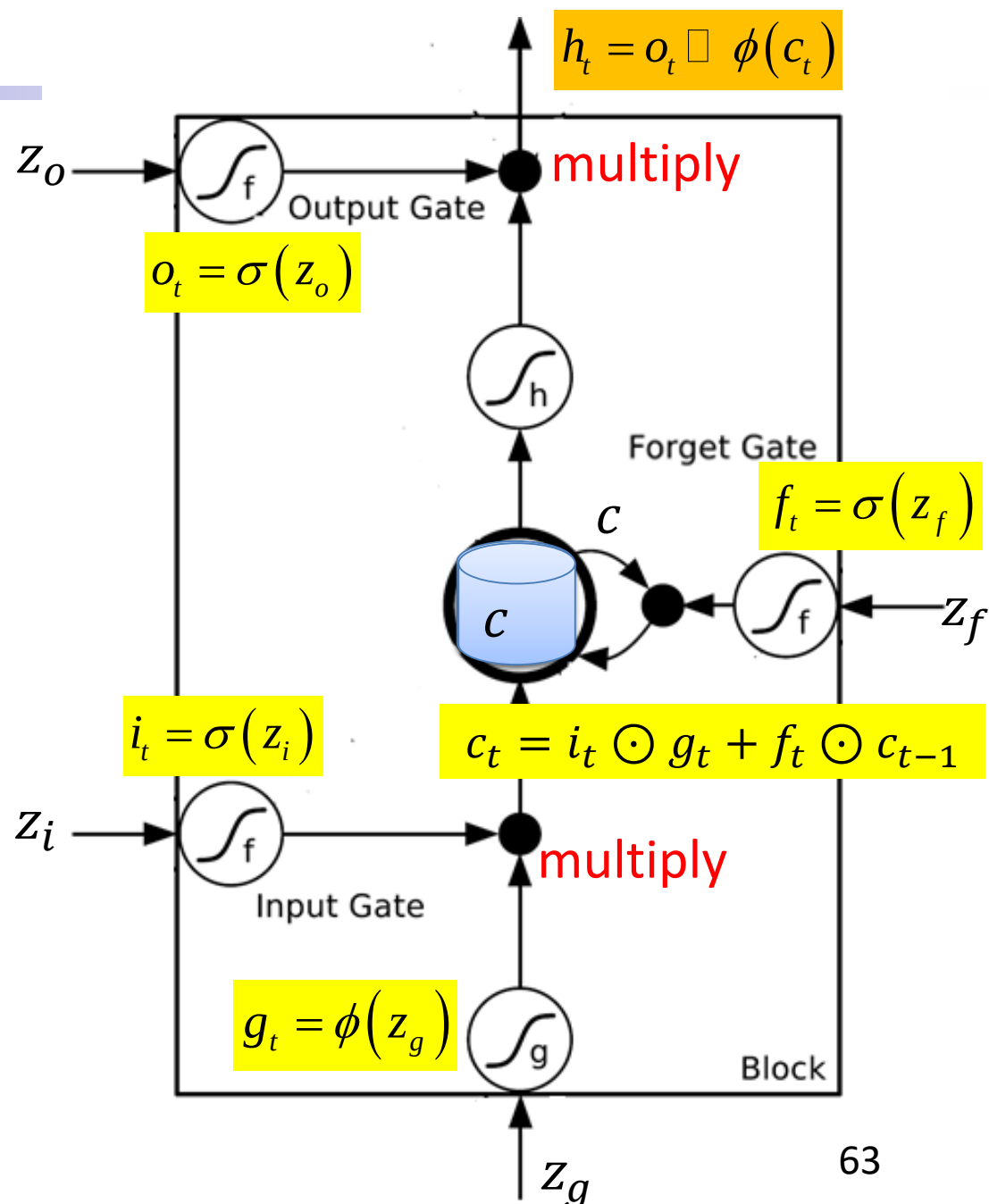
输出状态: $h_t = o_t \odot \tanh(c_t)$

$$z_g = W_g \cdot [h_{t-1}, x_t] + b_g$$

$$z_i = W_i \cdot [h_{t-1}, x_t] + b_i$$

$$z_f = W_f \cdot [h_{t-1}, x_t] + b_f$$

$$z_o = W_o \cdot [h_{t-1}, x_t] + b_o$$



6.2.2 长短时记忆网络

输入节点: $g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g)$

输入门: $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$

遗忘门: $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$

输出门: $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$

细胞状态: $c_t = i_t \odot g_t + f_t \odot c_{t-1}$

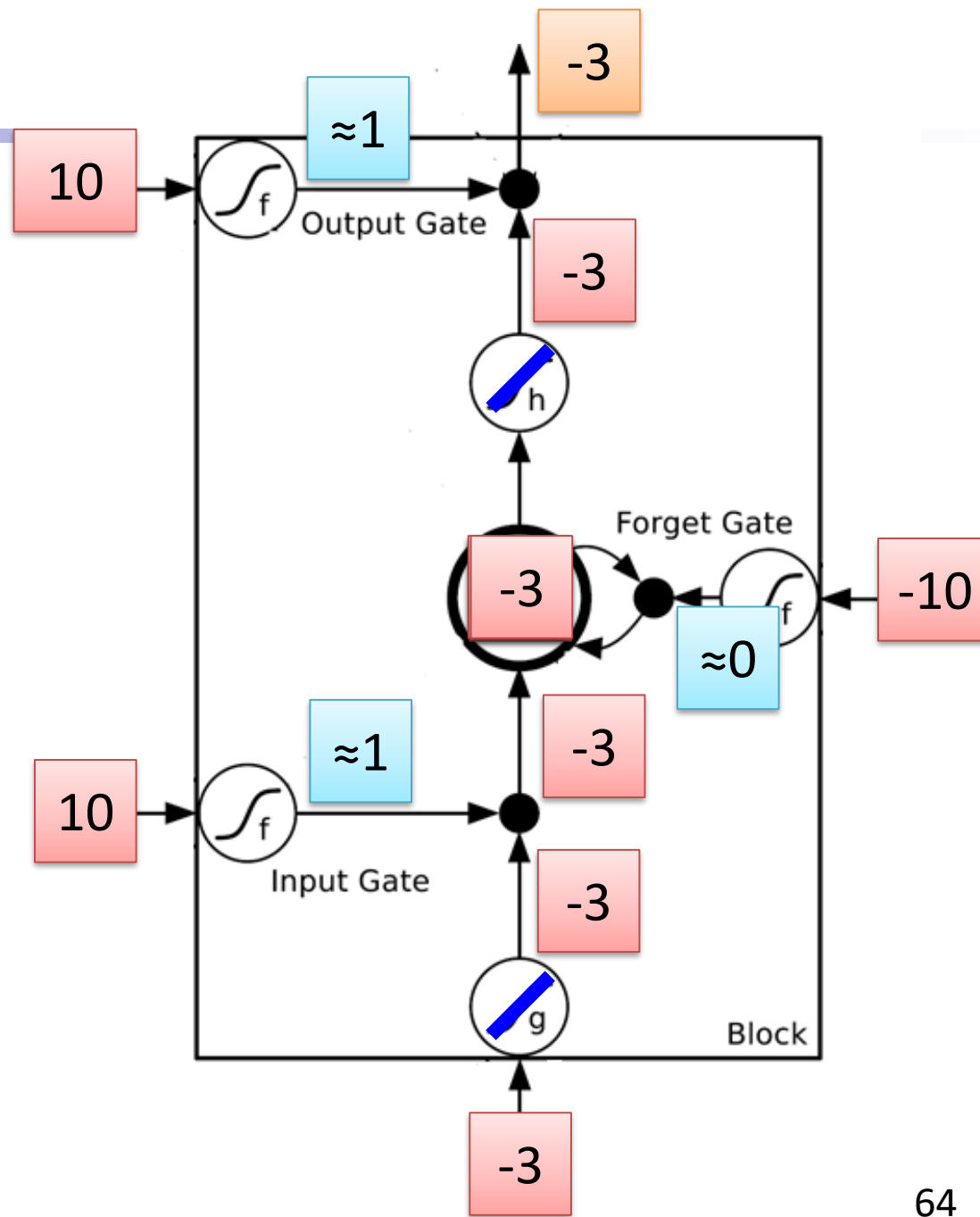
输出状态: $h_t = o_t \odot \tanh(c_t)$

$$z_g = W_g \cdot [h_{t-1}, x_t] + b_g$$

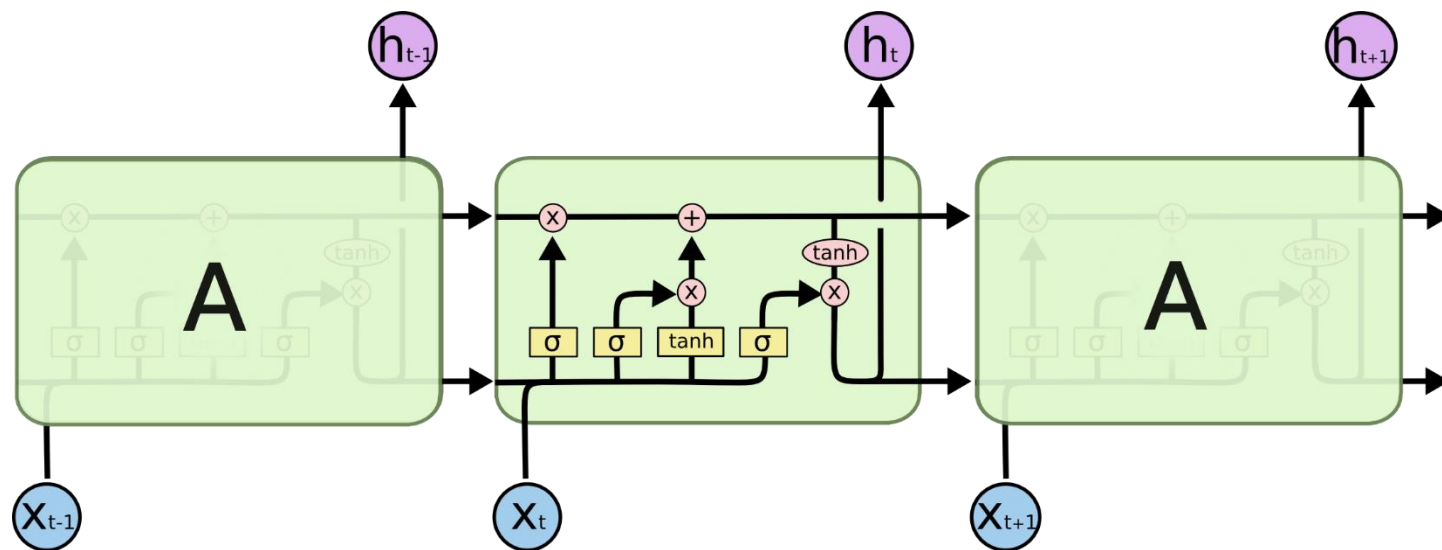
$$z_i = W_i \cdot [h_{t-1}, x_t] + b_i$$

$$z_f = W_f \cdot [h_{t-1}, x_t] + b_f$$

$$z_o = W_o \cdot [h_{t-1}, x_t] + b_o$$



6.2.2 长短时记忆网络



输入节点: $g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g)$

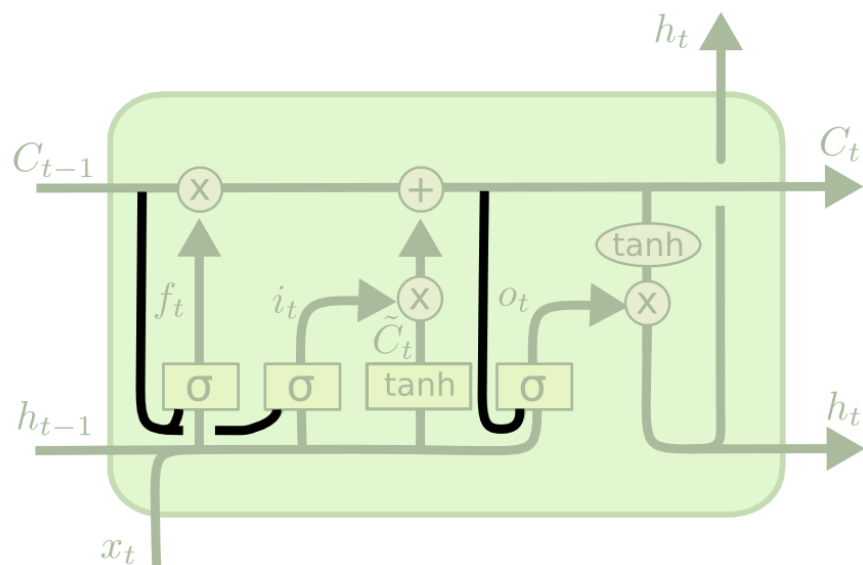
输入门: $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$

遗忘门: $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$

输出门: $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$

细胞状态: $c_t = i_t \odot g_t + f_t \odot c_{t-1}$

输出状态: $h_t = o_t \odot \tanh(c_t)$



$$f_t = \sigma(W_f[x_t, h_{t-1}, C_{t-1}] + b_f)$$

$$i_t = \sigma(W_i[x_t, h_{t-1}, C_{t-1}] + b_i)$$

$$o_t = \sigma(W_o[x_t, h_{t-1}, C_{t-1}] + b_o)$$

“Peephole connections”: let the gate layers look at the cell state

6.2.2 长短时记忆网络 LSTM (pytorch)

CLASS `torch.nn.LSTM(*args, **kwargs)`

[\[SOURCE\]](#)

Applies a multi-layer long short-term memory (LSTM) RNN to an input sequence.

For each element in the input sequence, each layer computes the following function:

$$\begin{aligned}i_t &= \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}) \\f_t &= \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}) \\g_t &= \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}) \\o_t &= \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}) \\c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\h_t &= o_t \odot \tanh(c_t)\end{aligned}$$

where h_t is the hidden state at time t , c_t is the cell state at time t , x_t is the input at time t , h_{t-1} is the hidden state of the layer at time $t-1$ or the initial hidden state at time 0, and i_t , f_t , g_t , o_t are the input, forget, cell, and output gates, respectively. σ is the sigmoid function, and $\odot$ is the Hadamard product.

In a multilayer LSTM, the input $x_t^{(l)}$ of the l -th layer ($l \geq 2$) is the hidden state $h_t^{(l-1)}$ of the previous layer multiplied by dropout $\delta_t^{(l-1)}$ where each $\delta_t^{(l-1)}$ is a Bernoulli random variable which

6.2.2 门控循环单元GRU

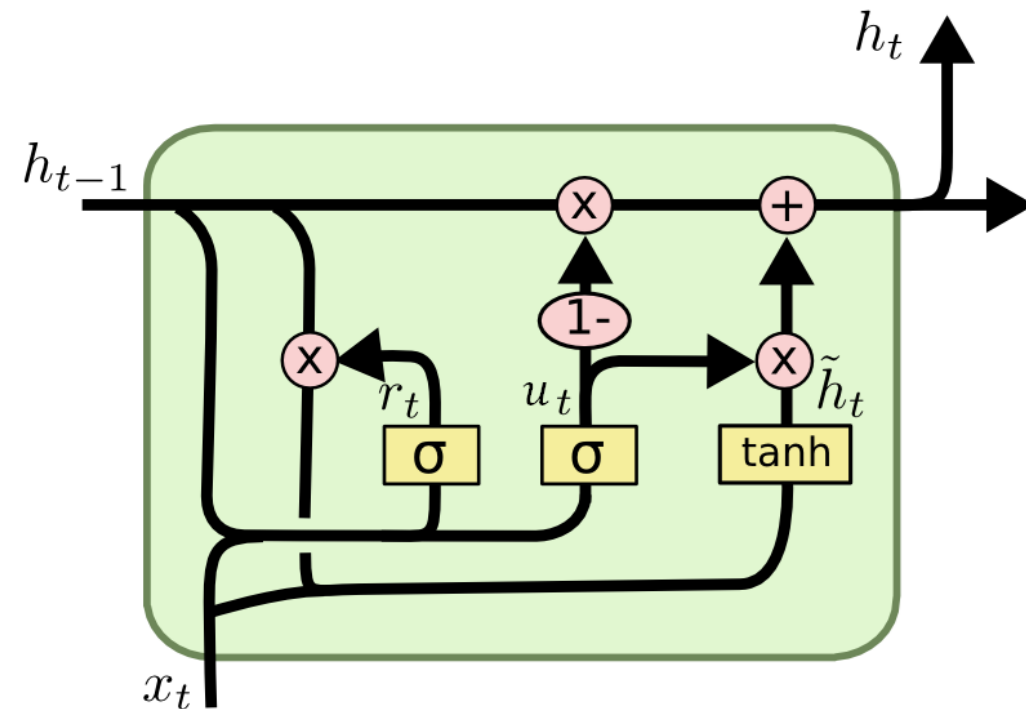
➤ Gated Recurrent Unit (Cho et al. 2014)

隐更新值: $\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h)$

隐含单元: $h_t = u_t \odot h_{t-1} + (1 - u_t) \odot \tilde{h}_t$

更新门: $u_t = \sigma(W_u \cdot [h_{t-1}, x_t] + b_u)$

重置门: $r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$



6.2.2 门控循环单元GRU

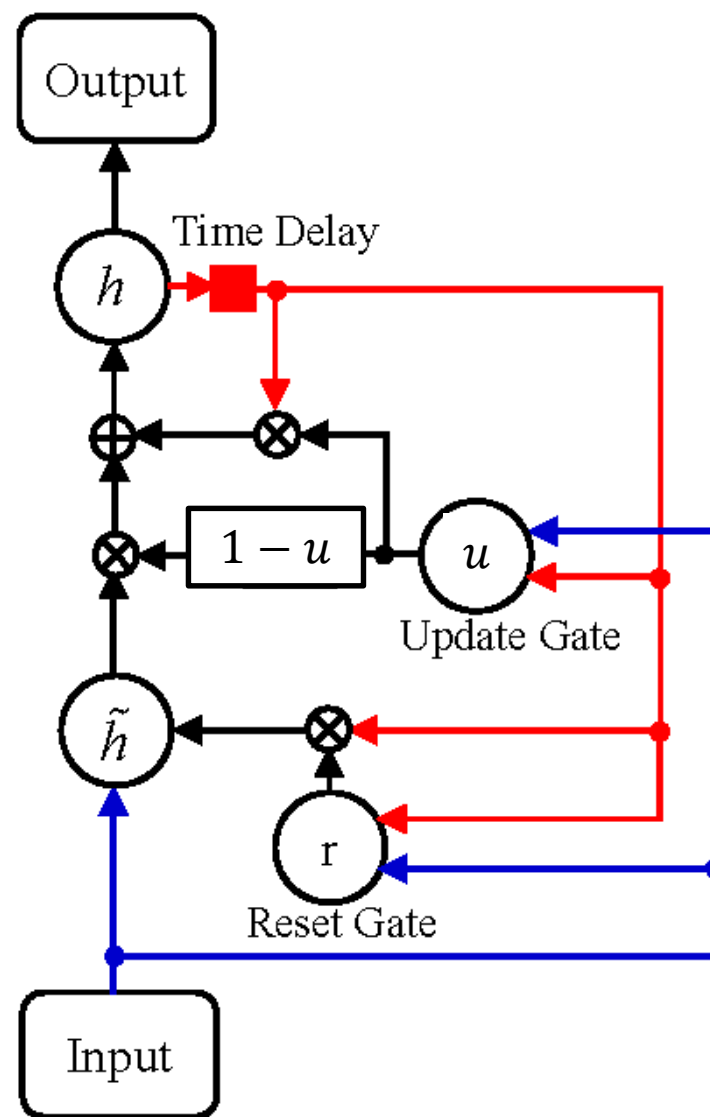
➤ Gated Recurrent Unit (Cho et al. 2014)

隐更新值: $\tilde{h}_t = \tanh(W_{xh}x_t + r_t \odot W_{hh}h_{t-1} + b_h)$

隐含单元: $h_t = u_t \odot h_{t-1} + (1 - u_t) \odot \tilde{h}_t$

更新门: $u_t = \sigma(W_{xu}x_t + W_{hu}h_{t-1} + b_u)$

重置门: $r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r)$



6.2.3 门控循环单元GRU (pytorch)

CLASS `torch.nn.GRU(*args, **kwargs)`

[\[SOURCE\]](#)

Applies a multi-layer gated recurrent unit (GRU) RNN to an input sequence.

For each element in the input sequence, each layer computes the following function:

$$\begin{aligned}r_t &= \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{(t-1)} + b_{hr}) \\z_t &= \sigma(W_{iz}x_t + b_{iz} + W_{hz}h_{(t-1)} + b_{hz}) \\n_t &= \tanh(W_{in}x_t + b_{in} + r_t * (W_{hn}h_{(t-1)} + b_{hn})) \\h_t &= (1 - z_t) * n_t + z_t * h_{(t-1)}\end{aligned}$$

where h_t is the hidden state at time t , x_t is the input at time t , $h_{(t-1)}$ is the hidden state of the layer at time $t-1$ or the initial hidden state at time 0, and r_t , z_t , n_t are the reset, update, and new gates, respectively. σ is the sigmoid function, and $*$ is the Hadamard product.

In a multilayer GRU, the input $x_t^{(l)}$ of the l -th layer ($l \geq 2$) is the hidden state $h_t^{(l-1)}$ of the previous layer multiplied by dropout $\delta_t^{(l-1)}$ where each $\delta_t^{(l-1)}$ is a Bernoulli random variable which is 0 with probability `dropout`.

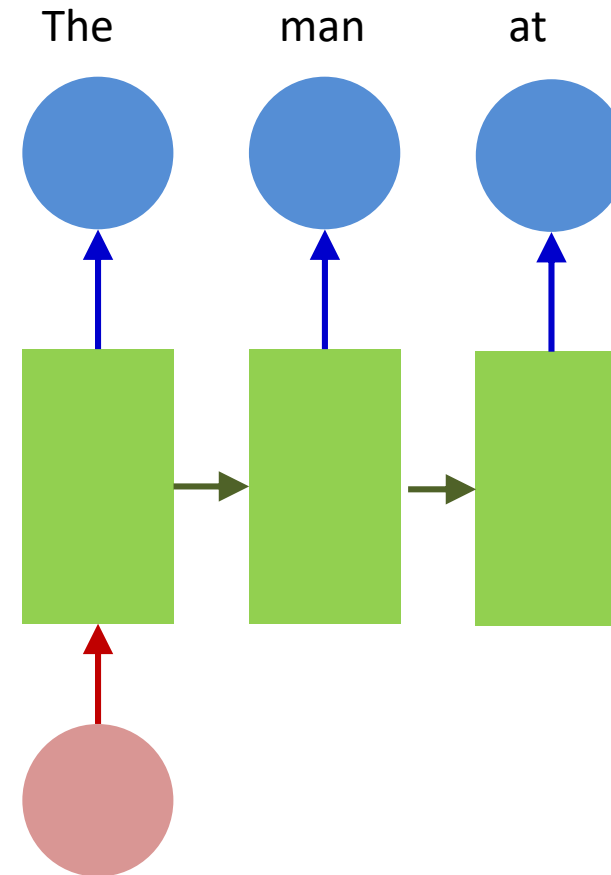
6.2.4 循环神经网络：应用

➤ One-to-many

- ◆ 图像描述 image captioning
- ◆ 从输入图片生成描述语句



The man at bat readies to swing at the pitch while the umpire looks on.

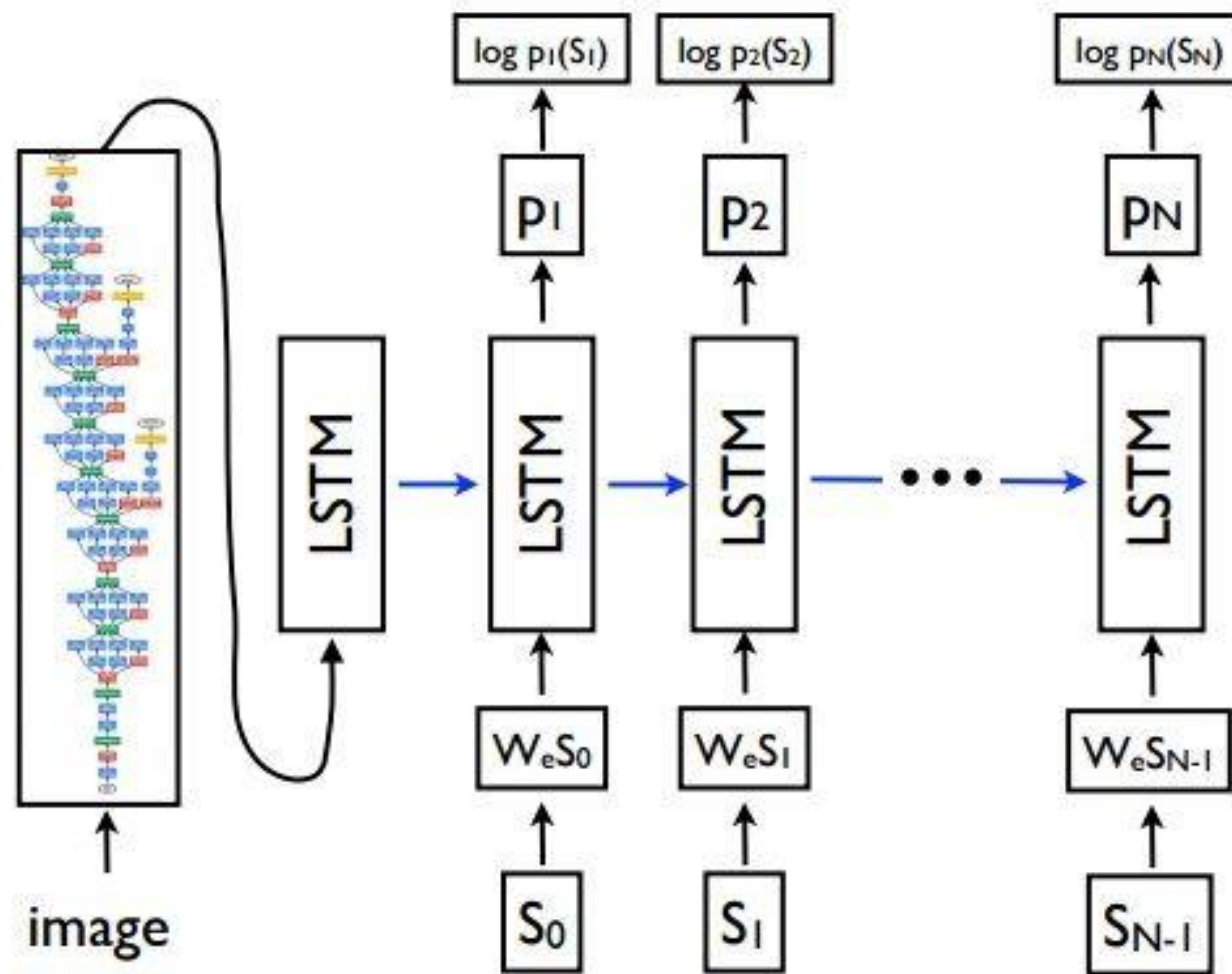


图像特征

6.2.4 循环神经网络：应用

➤ One-to-many

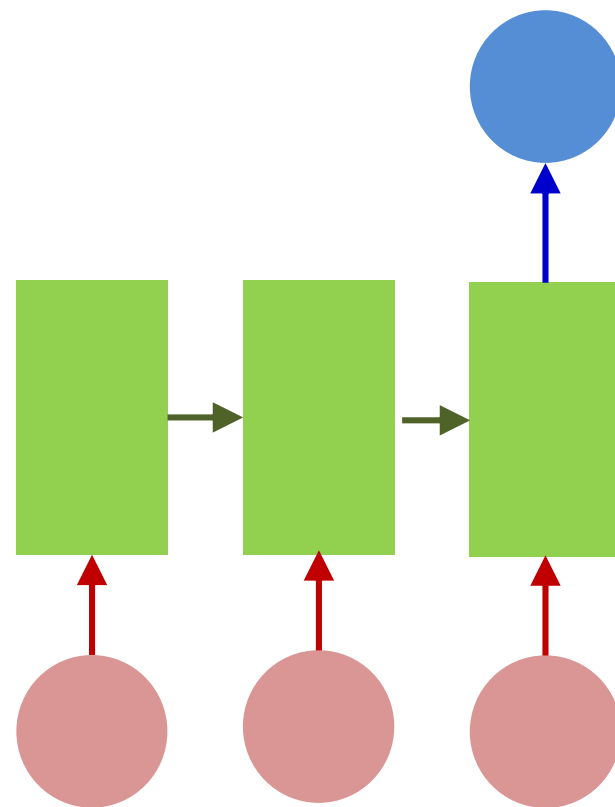
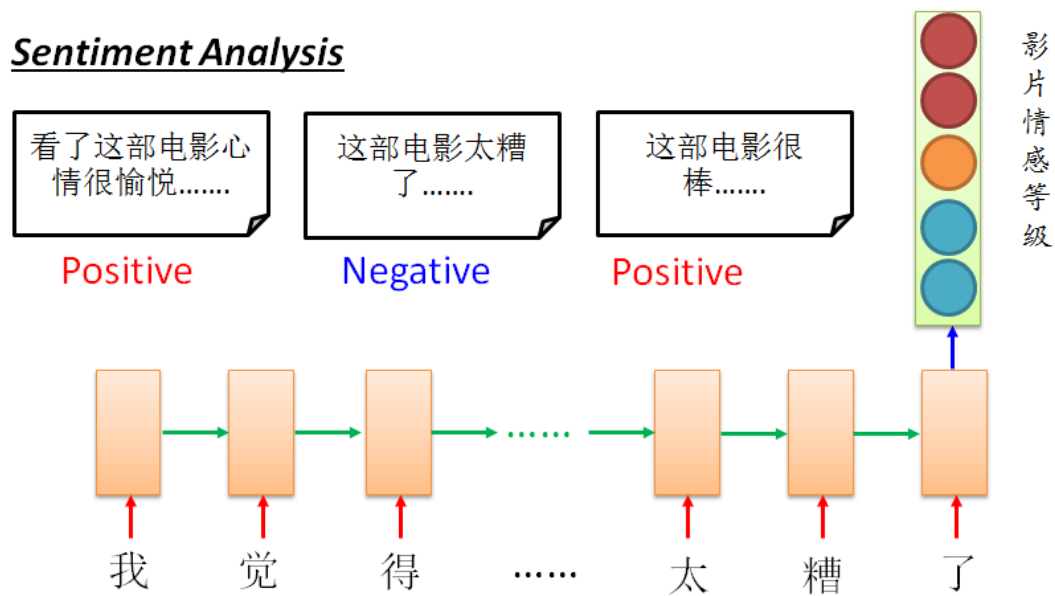
- ◆ 图像描述
- ◆ image captioning



6.2.4 循环神经网络：应用

➤ Many-to-one

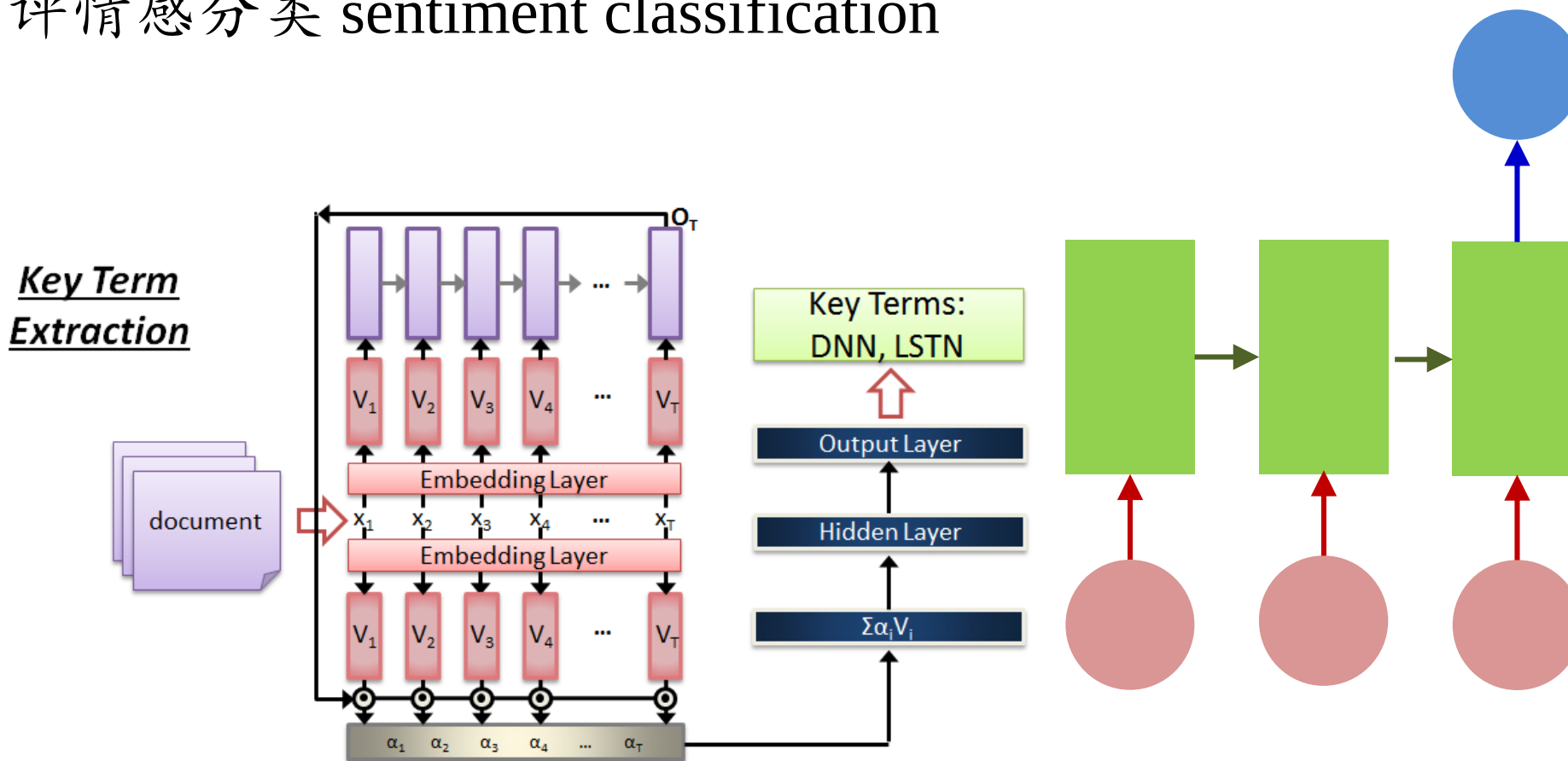
◆ 影评情感分类 sentiment classification



6.2.4 循环神经网络：应用

➤ Many-to-one

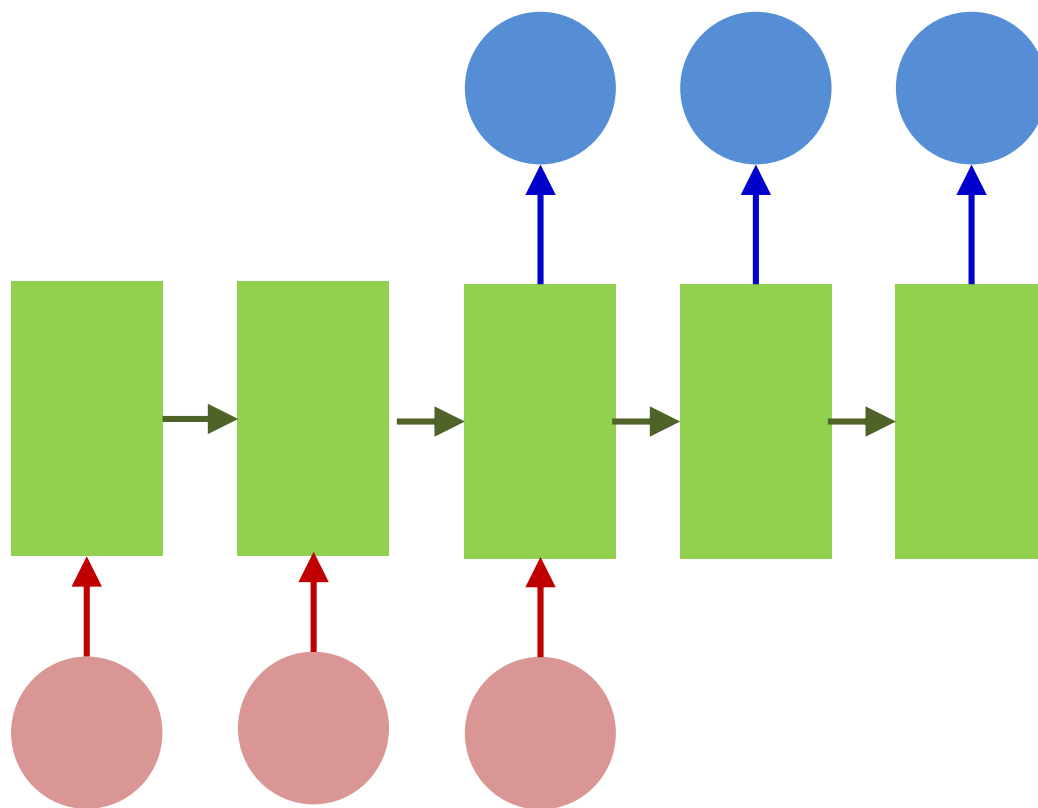
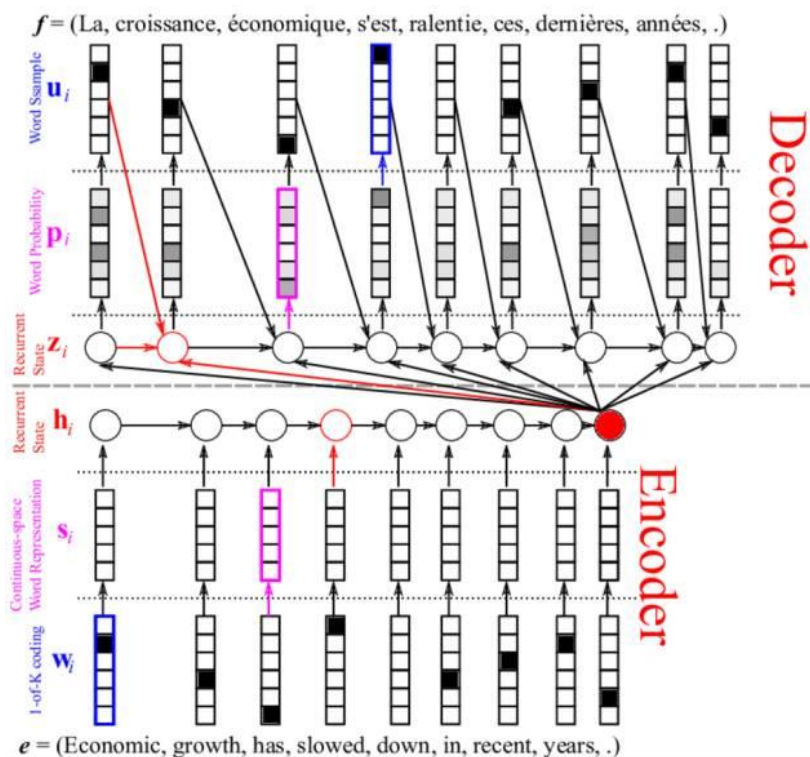
◆ 影评情感分类 sentiment classification



6.2.4 循环神经网络：应用

➤ Many-to-many

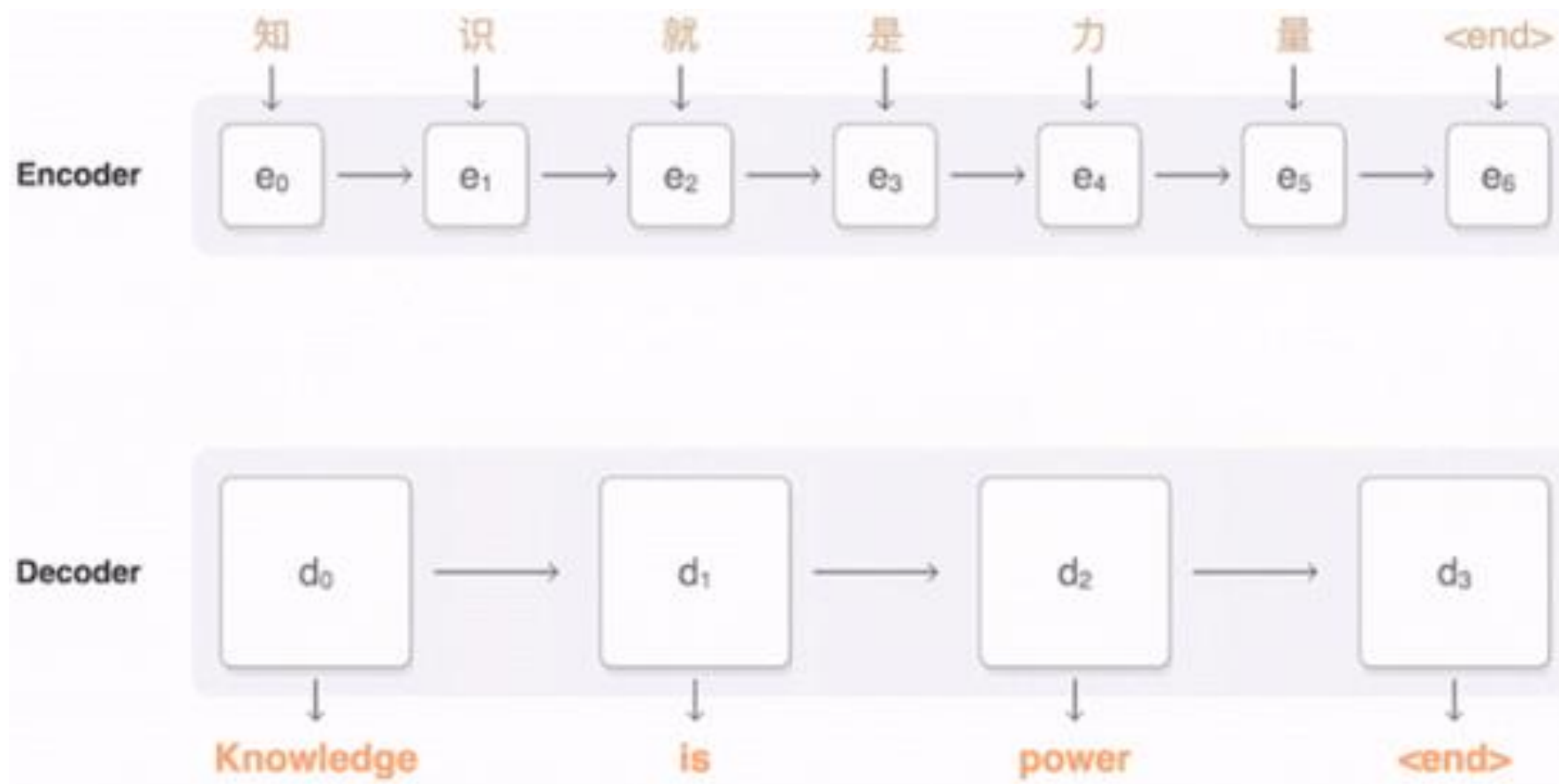
◆ 机器翻译 machine translation



6.2.4 循环神经网络：应用

➤ Many-to-many

◆ 机器翻译 machine translation



6.2.4 循环神经网络：应用

➤ Many-to-many

◆ 机器翻译 machine translation

Input sentence:	Translation (PBMT):	Translation (GNMT):	Translation (human):
李克強此行將啟動中加總理年度對話機制，與加拿大總理杜魯多舉行兩國總理首次年度對話。	Li Keqiang premier added this line to start the annual dialogue mechanism with the Canadian Prime Minister Trudeau two prime ministers held its first	Li Keqiang will start the annual dialogue mechanism with Prime Minister Trudeau of Canada and hold the first annual dialogue between the two premiers.	Li Keqiang will initiate the annual dialogue mechanism between premiers of China and Canada during this visit, and hold the first annual dialogue with Premier Trudeau of Canada.

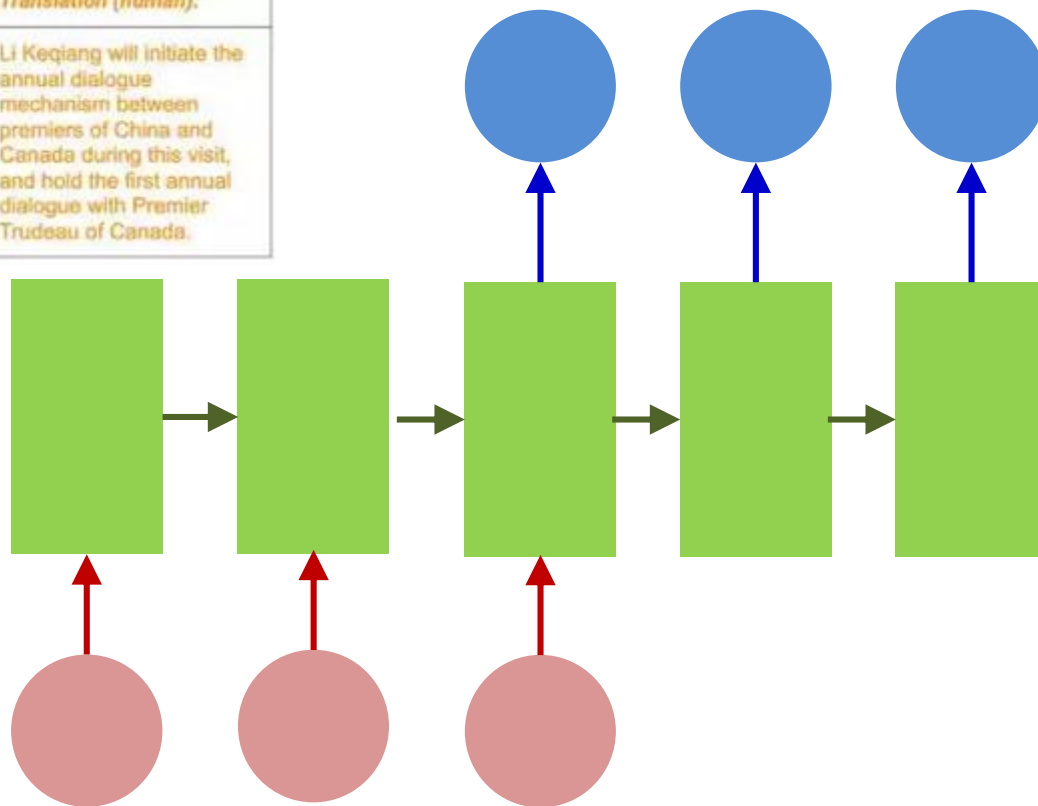
简体中文 (已检测) ⇌ 英语

清华大学电子工程系 ⇌ Department of Electronic Engineering, Tsinghua University

简体中文 (已检测) ⇌ 阿拉伯语

清华大学电子工程系 ⇌ قسم الهندسة الكهرونية ، جامعة تسينغها

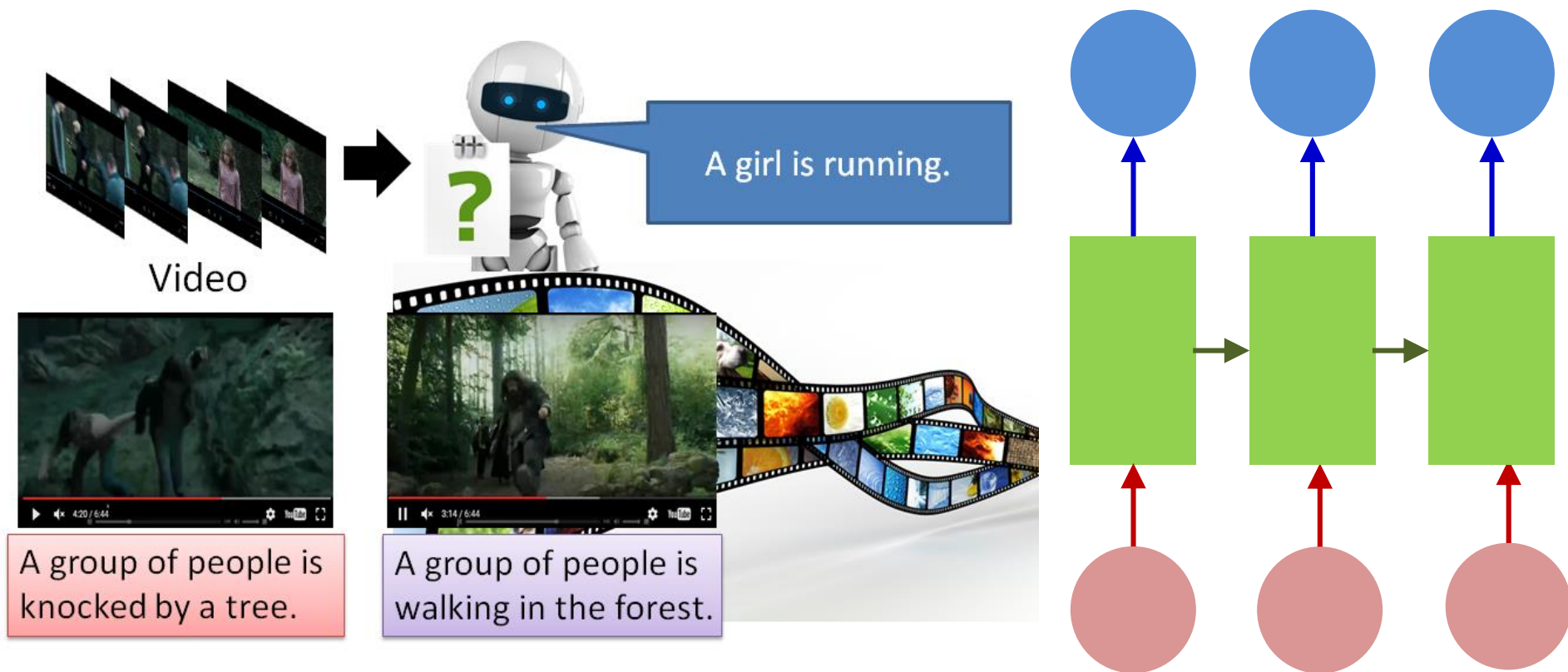
数据来源: 微软翻译



6.2.4 循环神经网络：应用

► Many-to-many

- ◆ 视频标签 video classification on frame level



以下()不属于LSTM引入的门

- ☐ A 输入门
- ☒ B 更新门
- ☐ C 输出门
- ☐ D 遗忘门

提交

谢谢大家！

欢迎提出宝贵意见！